Software Re-Engineering

Lecture 11

Automating the Development Environment

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2021

- Every developer has an array of different tools they need in order to complete their work.
- It can be quite frustrating to have to rebuild or recreate my development environment over and over again

  Whenever I start a new project, I generally have to download and install the tools I may need. This tends to take quite a bit of time to ensure the development environment will work seamlessly, because I generally have to:

  •Find the software, which entails finding the website, and reading the installation instructions.

  •Download it, or run the relevant commands that will download it.

  •Install it.

  •Ensure that it works to the project's specification by running a few trial commands.

  •Customise it further (Only necessary for certain tools).



Vagrant

Vagrant is an open-source utility that works on top of Virtual machine solutions like Virtualbox, HyperV, VMware, and also Docker

“Vagrant enables users to create and configure lightweight, reproducible, and portable development environments.”

primary goal seems to be *improving the workflow of using VMs, so that it is more convenient, faster and easily shareable*.

Vagrant integrates with many VM providers and exports a simpler API for working with the actual VMs, including setting them up, cloning them and of course using them.



Why vagrant?



- Prior to Vagrant, the preferred method of working on a web application was to install and configure all the software you needed (e.g., Apache, MySQL, RabbitMQ, etc.) locally on your development machine.
- In the days when web applications were almost certainly just PHP and MySQL alone, this made a lot of sense. Once installed, both technologies are generally “set and forget” and they just work. They also work well with multiple projects due to easy virtual hosts in Apache and database isolation with MySQL.
- But today, modern web applications have a lot more moving parts, a lot more options for underlying technologies, and a lot more overall complexity. While PHP and MySQL are still very popular, they no longer clearly dominate the market, and languages like JavaScript, Python, and Ruby are quickly becoming the dynamic languages of choice. For databases, there are now a handful of choices that are equally correct in their valid use case (e.g., MySQL, PostgreSQL, Redis, Riak, Cassandra, etc.). In addition to just languages and databases, there are multiple web servers, application servers, and backend services each with their own use cases (e.g., Apache, Nginx, Unicorn, Thin, RabbitMQ, Solr, etc.)



There are other issues with the local install practice as well…

- Everything must be installed manually, by a human.
- Configuration is even more difficult than installation. Once the software is installed, it has to be configured. Developers are unlikely to know how to configure it to best resemble production. Misconfigured software can lead to functionality that appears to work in development, but doesn’t work the same in production.
- With manually set up development environments, there is a stark difference between development and production. This leads to a lot of “works on my machine” bugs (i.e., bugs that don’t appear on a developer’s machine, but appear in production). Ideally, development would mirror production identically.



- It’s difficult to keep development environments in sync with other team members, or to bring a new team member on board.
- Because each developer is responsible for a separate development environment, it is easy for the environments to quickly get out of sync.
- And when new team members are hired, they are personally responsible for manually setting up an entire complex development environment.
- The result is confusion and long onboarding times before new team members can actually start working on the project itself.



- Vagrant is a modern solution to all of these problems. While not strictly required, Vagrant encourages automation to set up your development environments using shell scripts or configuration management software.
- Vagrant allows you to work with the same operating system that is running in production, whether your physical development machine is running Linux, Mac OS X, or Windows.
- Because Vagrant puts your development environment into a virtual machine, working with multiple projects is easy, because each project just get its own virtual machine.
- Finally, working with a team is easier than ever, since you can share the virtual machine image.
- And bringing a new team member on board is as simple as telling them to build their Vagrant machine with a single command.

Background

Virtualisation: ability to run an operating system within another operating system every day

VirtualBox is an open source (GPL v2) virtualization platform that works on almost any base OS.

Your virtual machines will be hosted in this on top of your base OS, sharing its resources.



- The vagrant command lets you manage your VMs (Vagrant machines) (starting them, stopping them, destroying unneeded VMs, etc.), and you can login to a VM simply by typing vagrant ssh.
- The main benefits of using Vagrant are as follows.
  - It makes it easy to automate the setup of a development environment inside a VM
  - Each VM is isolated from the host machine and other VMs, so you don't need to worry about version conflicts when you have many different projects set up on the same machine. If one project needs Python 2.6, Ruby 2.8.1 and PostgreSQL 9.1, while another needs Python 2.7, Ruby 2.0 and PostgreSQL 9.3, it can be tricky to set everything up on your development machine. But if each project lives in a separate VM, it can make life easier.

Why Vagrant is important for your development team

- Because as team leader you have to standardize the environment where the application is going to run so the developers can have the testing environment beforehand. It also allows new developers joining in to the team to have the work space up and running without spending hours installing and tuning all the required extensions your project is going to use.
- This does not imply the developer need to change the OS of their flavor since Vagrant takes advantage of virtualization in the developer's computer.
- Once Vagrant is installed and the project configuration is ready the developer is all set to start working in an environment shared with the others developers in team.



Real-world use cases for Vagrant

1.Development Environments: Let's say you want identical development environments for all the developers in a team. In this case, you can create a common Vagrantfile with all the configs required to provision the VM with development environment dependencies (Software). And then share the Vagrantfile with all the developers. This way, there will not be any change in the underlying software or configs used. Some companies host Vagrant boxes in a common location for different approved software so everyone in the organization can use it.

2.Sandboxed environments: A sandbox is an isolated testing environment that enables users to run programs or open files without affecting the application, system or platform on which they run. If you want a sandboxed environment for testing, you could use Vagrant to spin up and destroy VMs in minutes.



Vagrantfile

Using Vagrant, you can easily create virtual development environments from exiting VM images and have all the VM configs in a configuration file called Vagrantfile. To put it simply, you define everything you need in a VM in the Vagrantfile, and Vagrant will take care of configuring those in the VM.



Download Vagrant and VirtualBox

Vagrant 2.2.3_x86_64 download (Windows 64 bit)
VirtualBox 6.0.4 download (Windows)



Installing the Megabits and Pieces

To get started, go ahead and install these core tools:

- VirtualBox (the software that creates virtual machines)
- Vagrant (the software that deploys virtual machines and runs provisioning scripts)

Start your first Vagrant project by creating a project folder which will house the various configurations for each of your VMs. You'll use the command line to run Vagrant commands from within these folders.

Create a project folder,

E:\Vagrant\



The primary commands to get started are **vagrant init** and **vagrant up**

- Open a command line and change directories to your test project folder.
- Run vagrant init and a new vagrant file will be created in the current directory called “Vagrantfile” (no extension) which holds a basic starter configuration.

- Open Vagrantfile in a text editor like Notepad++
  - study the comments before each configuration value.
  - You can set shared folders between guest and host, turn on port forwarding, set the hostname, and more.
  - Notice that all but maybe one line is commented out: Vagrant has a default configuration it will use even without any settings changed in here.



- This Vagrantfile doesn’t point to any sort of virtual machine yet, so go ahead and delete the Vagrantfile you just created and let’s find a “base box” to use.
- **Base box** contain the bare minimum required for Vagrant to function
- Browse the Vagrant Cloud for base boxes you might like to use. For this example, we’ll use “ubuntu/trusty64” which is the “official” base box for Ubuntu Server 14.04 LTS (Trusty Tahr).
- To have Vagrant automatically use this base box, type this:

vagrant init ubuntu/trusty64

- Now that you have a Vagrantfile with a base box configured, you can spin up the VM with this amazing command:

vagrant up

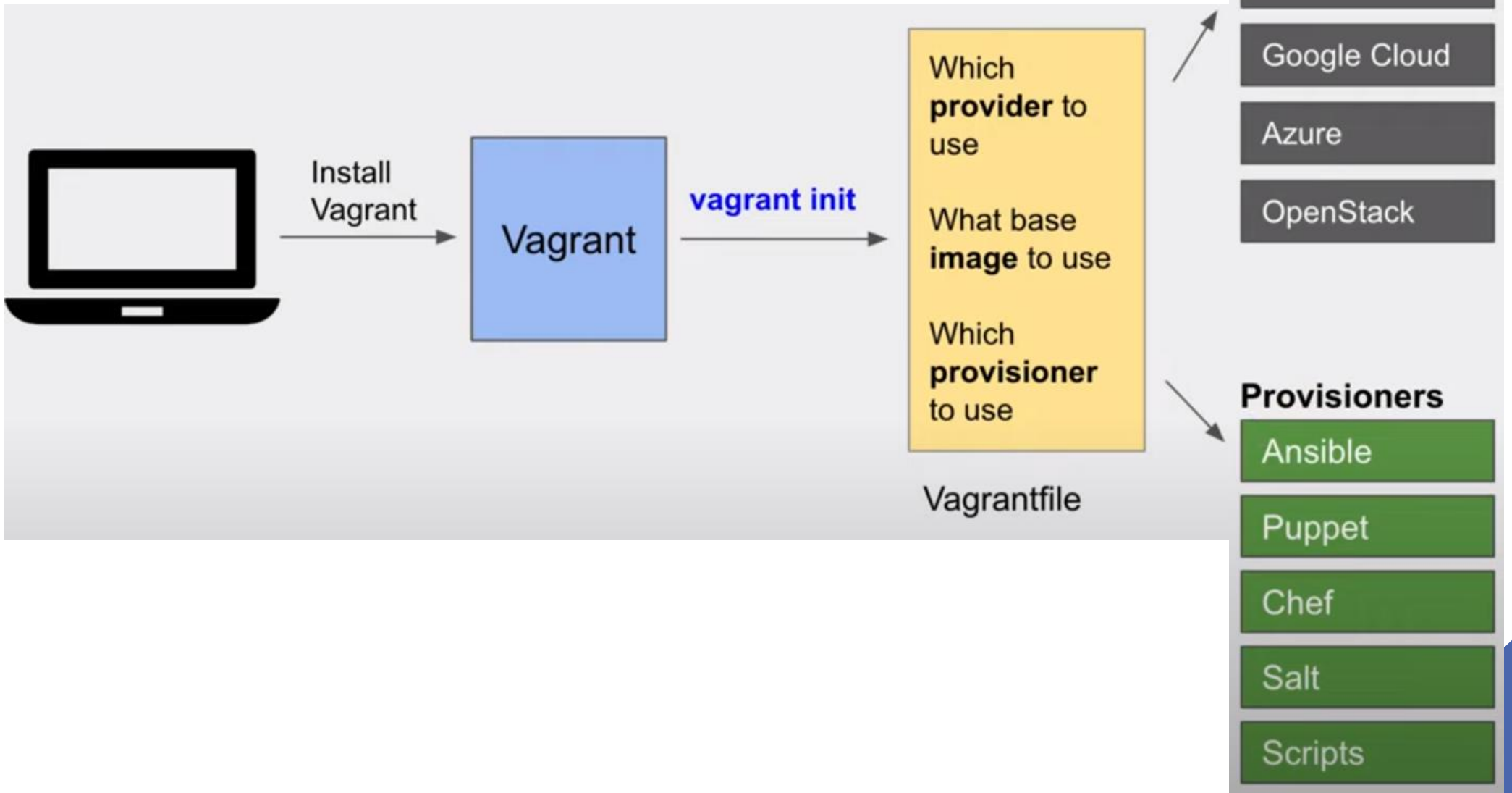
This command creates and configures guest machines according to your Vagrant file.

vagrant up [name|id]

- **name -** Name of machine defined in Vagrantfile. Using name to specify the Vagrant machine to act on must be done from within a Vagrant project (directory where the Vagrantfile exists).

- **id -** Machine id found with vagrant global-status. Using id allows you to call vagrant up id from any directory.




Install
Vagrant
Vagrant
vagrant init
Which
provider to
use
What base
image to use
Which
provisioner
to use
Vagrantfile
Providers
Virtualbox
VMWare
AWS Cloud
Google Cloud
Azure
OpenStack
Provisioners
Ansible
Puppet
Chef
Salt
Scripts

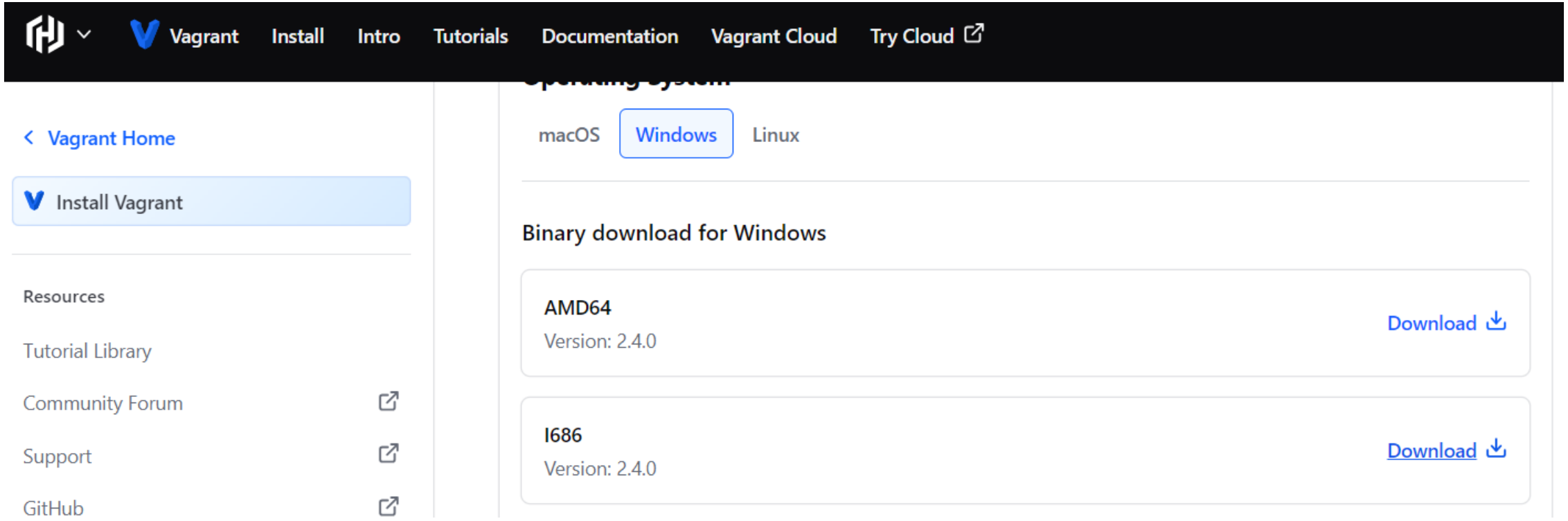
Provisioners in Vagrant allow you to automatically install software, alter configurations, and more on the machine as part of the vagrant up process.

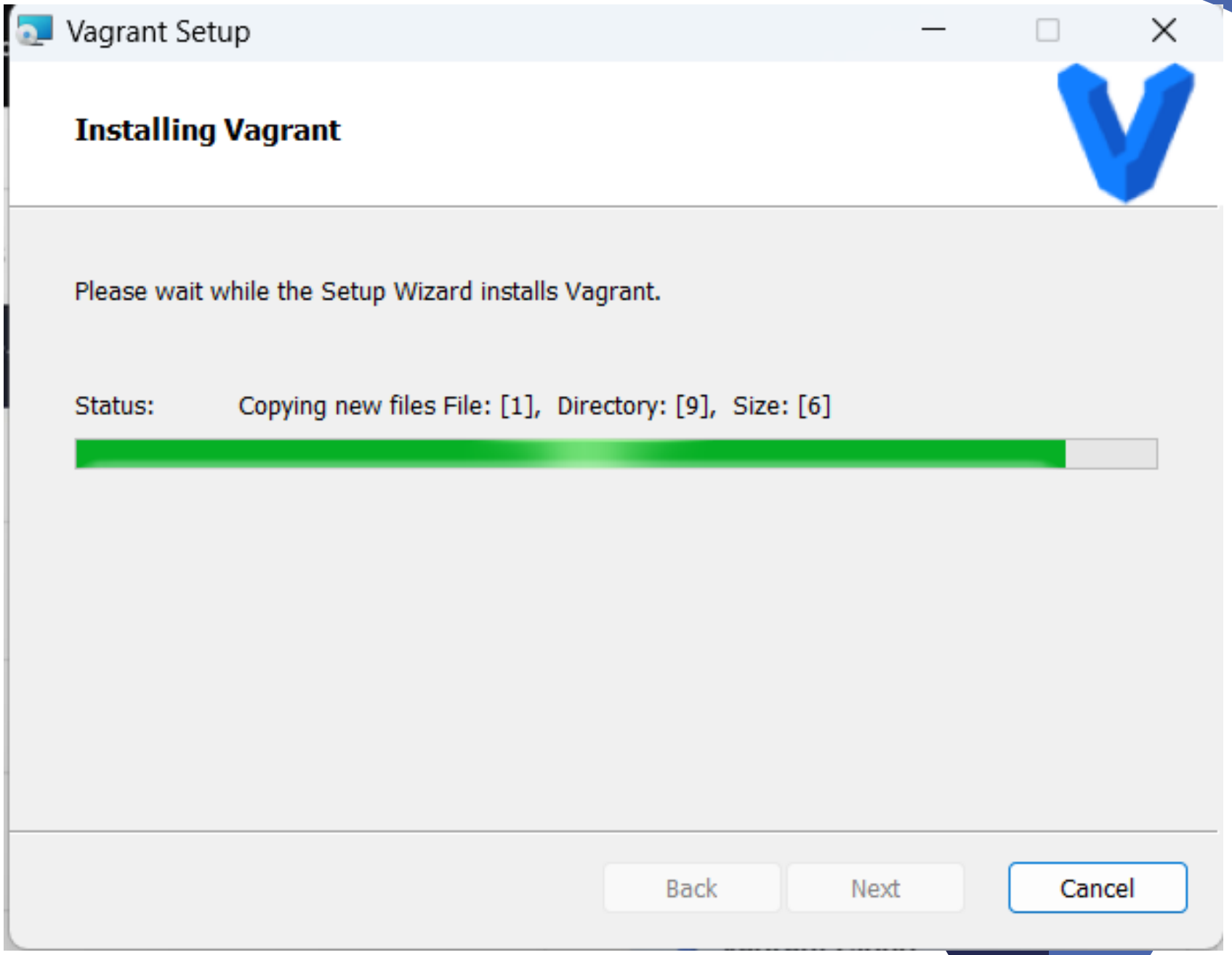
Can use shell scripts or any provisioner



Hands on

- Visit https://developer.hashicorp.com/vagrant/install




Vagrant Setup
Installing Vagrant
Please wait while the Setup Wizard installs Vagrant.
Status: Copying new files File: [1], Directory: [9], Size: [6]
Back
Next
Cancel

```
C:\Users\Natalia Farhan>vagrant --version
Vagrant 2.4.0

C:\Users\Natalia Farhan>
```

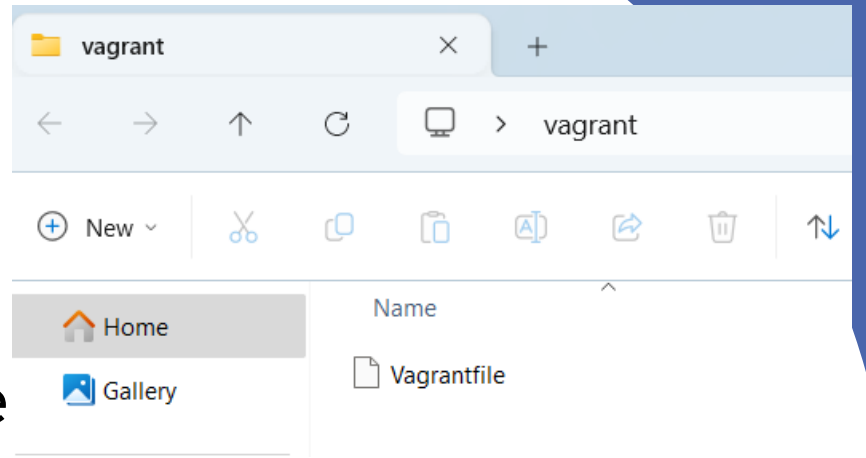
Download virtualbox

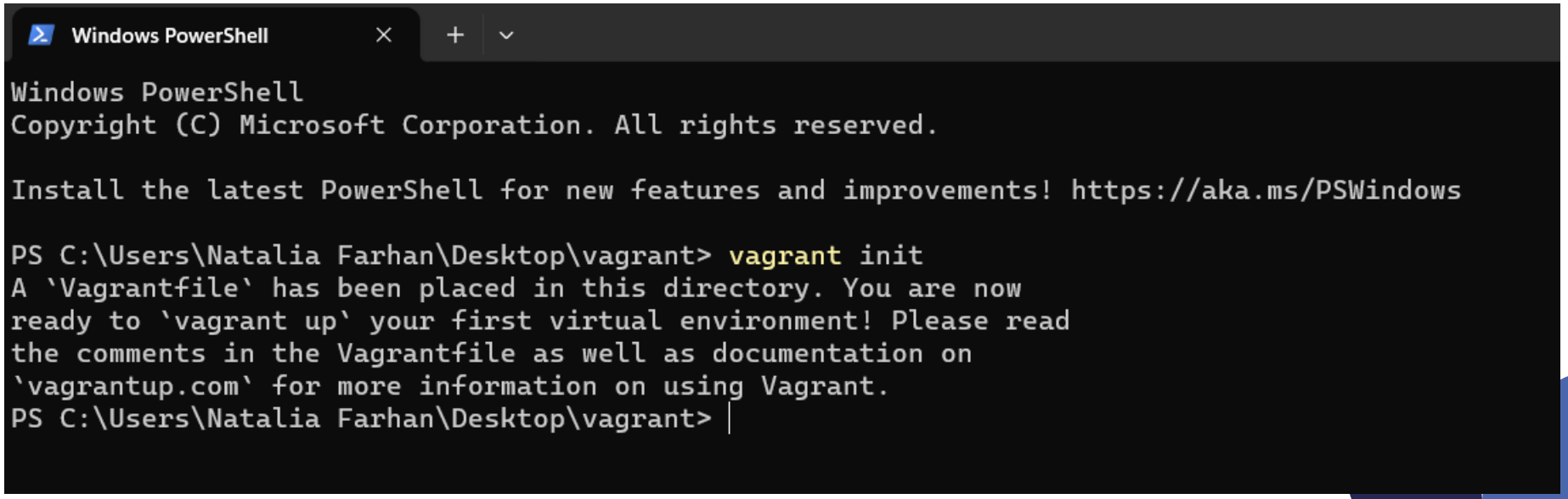
https://www.virtualbox.org/wiki/Downloads

Next step..

Create a folder (e.g. vagrant on Desktop)

Open that folder in cmd (terminal) and type vagrant init





Uncommented line

Vagrantfile

File Edit View

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

# All Vagrant configuration is done below. The "2" in Vagrant.configure
# configures the configuration version (we support older styles for
# backwards compatibility). Please don't change it unless you know what
# you're doing.
Vagrant.configure("2") do |config|
  # The most common configuration options are documented and commented below.
  # For a complete reference, please see the online documentation at
  # https://docs.vagrantup.com.

  # Every Vagrant development environment requires a box. You can search for
  # boxes at https://vagrantcloud.com/search.
  config.vm.box = "base"

  # Disable automatic box update checking. If you disable this, then
  # boxes will only be checked for updates when the user runs
  # `vagrant box outdated`. This is not recommended.
  # config.vm.box_check_update = false

  # Create a forwarded port mapping which allows access to a specific port
  # within the machine from a port on the host machine. In the example below,
  # accessing "localhost:8080" will access port 80 on the guest machine.
  # NOTE: This will enable public access to the opened port
  # config.vm.network "forwarded_port", guest: 80, host: 8080

  # Create a forwarded port mapping which allows access to a specific port
  # within the machine from a port on the host machine and only allow access
  # via 127.0.0.1 to disable public access
  # config.vm.network "forwarded_port", guest: 80, host: 8080, host_ip: "127.0.0.1"

  # Create a private network, which allows host-only access to the machine
  # using a specific IP.
  # config.vm.network "private_network", ip: "192.168.33.10"

  # Create a public network, which generally matched to bridged network.
  # Bridged networks make the machine appear as another physical device on
  # your network.
  # config.vm.network "public_network"
```

Next step..

Choose a box

https://app.vagrantup.com/boxes/search

a box is a pre-configured virtual machine image that you can use as a startng point for your vitual machine



HashiCorp Vagrant Cloud

Search | Pricing | Vagrant | Help | Create an Account | Sign In

Discover Vagrant Boxes

ubuntu

Provider | Architecture any | Sort by Downloads

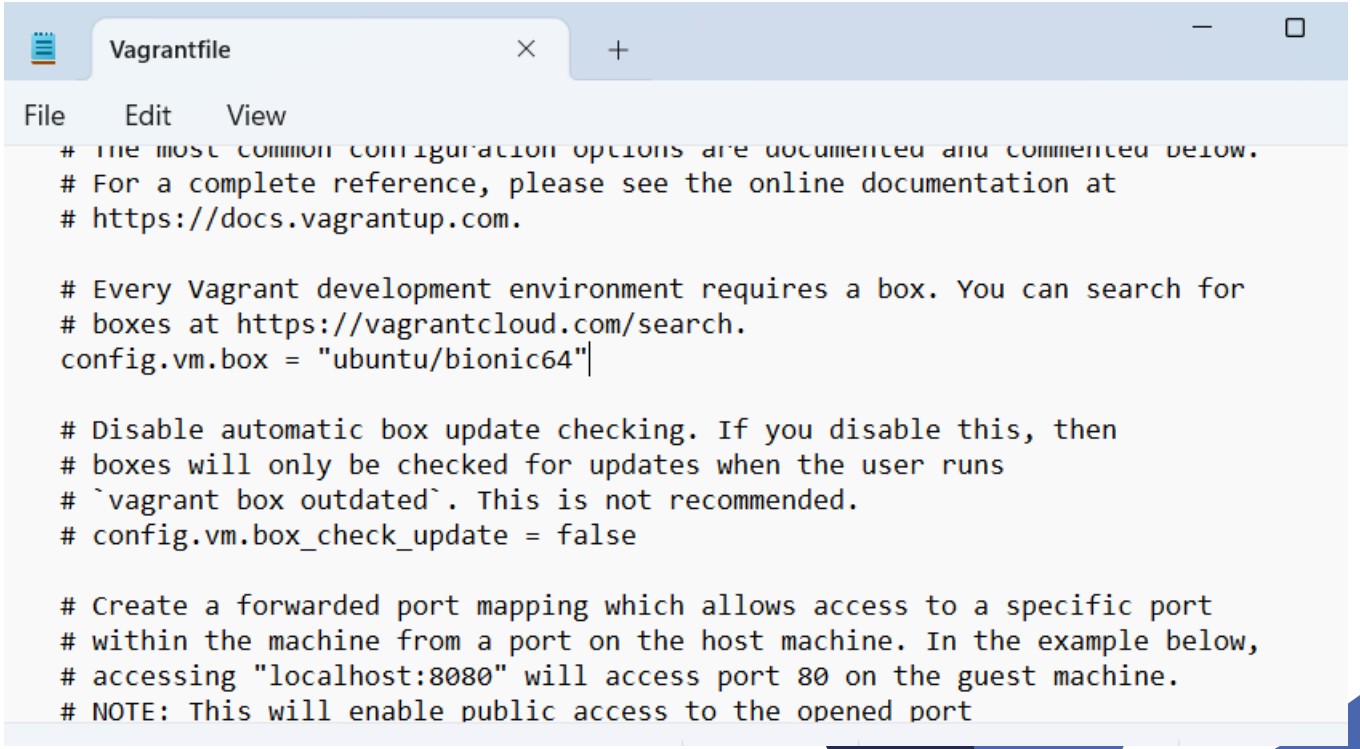
ubuntu/trusty64 20190514.0.0 Official Ubuntu Server 14.04 LTS (Trusty Tahr) builds (End of standard support)	virtualbox	Downloads 30,745,002	Released about 4 years ago
hashicorp/precise64 1.1.0 A standard Ubuntu 12.04 LTS 64-bit box.	hyperv, virtualbox, vmware_fusion	Downloads 6,810,647	Released over 9 years ago
ubuntu/xenial64 20211001.0.0 Official Ubuntu 16.04 LTS (Xenial Xerus) builds (End of standard support)	virtualbox	Downloads 3,592,814	Released about 2 years ago

ubuntu / bionic64 Vagrant box

How to use this box with Vagrant:

Vagrantfile | New

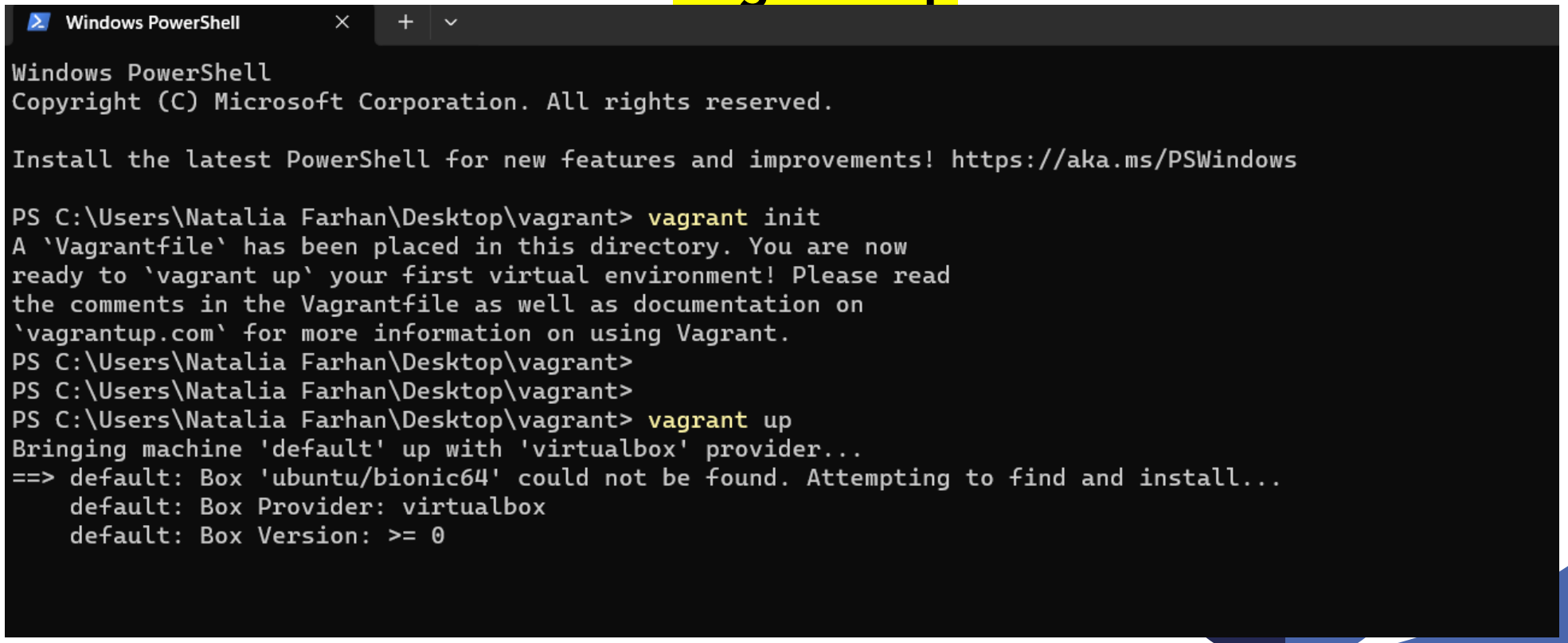
```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"
end
```



Next step...

Start virtual machine using command

vagrant up



Type command:

Vagrant box list

Next step..

Add SSH to virtual machine
Vagrant ssh

whoami
pwd (parent working directory)
ls –l

This is where boxes go:
C:\Users\Natalia Farhan\.vagrant.d\boxes

This will SSH into a running Vagrant machine and give you access to a shell.

To Dos..

Try following vagrant commands

- vagrant up: Bring a box online.
- vagrant status: Show current box status.
- vagrant suspend: Pause the current box.
- vagrant resume: Resume the current box.
- vagrant halt: Shutdown the current box.
- vagrant destroy: Destroy the current box. By running this command, you will lose any data stored on the box.
- vagrant snapshot: Take a snapshot of the current box.

- vagrant box list -- see a list of all installed boxes on your computer
- vagrant box add <name> <url> -- download a box image to your computer
- vagrant box outdated -- check for updates vagrant box update
- vagrant box remove <name> -- deletes a box from the machine
- vagrant package -- packages a running virtualbox env in a reusable box

Vagrant Environment Setup



Objective: Create a Vagrant environment that includes multiple virtual machines (VMs) with specific configurations.

Requirements:

1.Install Vagrant and VirtualBox on your machine.

2.Create a directory for your Vagrant project.



Task 1: Single VM Setup Create a Vagrantfile that sets up a single virtual machine with the following specifications:

- Operating System: Ubuntu 20.04
- 512 MB RAM
- 1 CPU core
- Static IP address: 192.168.33.10

```
# Vagrantfile

Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"

  config.vm.define "web_server" do |web_server|
    web_server.vm.hostname = "web-server"
    web_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.10”

    web_server.vm.provider "virtualbox" do |vb|
      vb.memory = "512"
      vb.cpus = 1
    end
  end
end
```

Task 2: Multi-VM Setup Extend the existing Vagrantfile to include a second virtual machine:

- Operating System: CentOS 7
- 1 GB RAM
- 2 CPU cores
- Static IP address: 192.168.33.11

```
# Vagrantfile
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"

  config.vm.define "web_server" do |web_server|
    web_server.vm.hostname = "web-server"
    web_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.10”
    web_server.vm.provider "virtualbox" do |vb|
      vb.memory = "512"
      vb.cpus = 1
    end
  end

  config.vm.define "app_server" do |app_server|
    app_server.vm.box = "centos/7"
    app_server.vm.hostname = "app-server"
    app_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.11”

    app_server.vm.provider "virtualbox" do |vb|
      vb.memory = "1024"
      vb.cpus = 2
    end
  end
end
```

Task 3: Provisioning Scripts Create shell provisioning scripts for each VM that install and configure the following:

- **web_server:** Nginx server
- **app_server:** Apache server

You can provision your VM using shell scripts, Ansible, Puppet, or other provisioners. For simplicity, let's create a basic shell script.

Create a file named web_server.sh and app_server.sh in your project directory

```
# Vagrantfile
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"

  config.vm.define "web_server" do |web_server|
    web_server.vm.hostname = "web-server"
    web_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.10”
    web_server.vm.provider "virtualbox" do |vb|
      vb.memory = "512"
      vb.cpus = 1
    end
    web_server.vm.provision "shell", path: "provision/web_server.sh"
  end
  config.vm.define "app_server" do |app_server|
    app_server.vm.box = "centos/7"
    app_server.vm.hostname = "app-server"
    app_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.11”

    app_server.vm.provider "virtualbox" do |vb|
      vb.memory = "1024"
      vb.cpus = 2
    end
    app_server.vm.provision "shell", path: "provision/app_server.sh"
  end
end
```

web_server.sh:

```
#!/bin/bash

# Install Nginx
sudo apt-get update
sudo apt-get install -y nginx

# Start Nginx
sudo service nginx start
```

app_server.sh:

```
#!/bin/bash

# Install Apache
sudo yum install -y httpd

# Start Apache
sudo service httpd start
```

Software Re-Engineering

Lecture 12

Vagrant with Ansible

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

- Though Vagrant handles the generation and basic imaging of virtual machines to form isolated development environments, it doesn't do the provisioning of these virtual machines.
- Provisioning is the concept of preparing and equipping a machine to allow it to provide (new) services to its users
- So, it's up to us to handle the installation and configuration of the packages we want to work.
- Fortunately, Vagrant easily integrates with existing provisioners such as Chef, Puppet, and Ansible.



- Once a virtual machine is generated and a provisioner is ready, we can quickly start a development environment on our machine without having to worry about finding, installing, and configuring project dependencies.

- Any developer can pickup a project and start working on it with little work with the same environment as everyone else.



- Ansible communicates over normal SSH channels in order to retrieve information from remote machines, issue commands, and copy files.
- Ansible can interact with clients through either command line tools or through its configuration scripts called Playbooks.
- It uses a very simple language (YAML, in the form of Ansible Playbooks) that allow you to describe your automation jobs in a way that approaches plain English.
- YAML stands for Yet Another Markup Language.
- A *Playbook* is simply a text-file that defines which *roles* get assigned to which machines

What Problem It Solves:

NASA needed to move roughly 65 applications from a traditional hardware-based data center to a cloud-based environment for better agility and cost savings. The rapid timeline resulted in many applications being migrated ‘as-is’ to a cloud environment. This created an environment spanning multiple virtual private clouds (VPCs) and AWS accounts that could not be easily managed. Even simple things, like ensuring every system administrator had access to every server, or simple patching, were extremely burdensome.

NASA used the Ansible to manage and schedule the cloud environment.

As a result of implementing the Ansible, NASA is better equipped to manage its AWS environment.

This allowed NASA to provide better operations and security to its clients. It has also increased efficiency as a team.

- Updating nasa.gov went from over 1 hour to under 5 minutes
- Patching updates went from a multi-day process to 45 minutes
- Achieving near real-time RAM and disk monitoring
- Provisioning OS Accounts across an entire environment in under 10 minutes
- Application stack set up from 1–2 hours to under 10 minutes.

How Ansible works?

- In Ansible, there are two categories of computers: the ***control node*** and ***managed nodes*****.**
- The control node is a computer that runs Ansible.
- There must be at least one control node, although a backup control node may also exist.
- A managed node is any device being managed by the control node.



- Ansible works by connecting to nodes (clients, servers, or whatever you're configuring) on a network, and then sending a small program called an Ansible module to that node.
- Ansible executes these modules over SSH and removes them when finished.

- The only requirement for this interaction is that your Ansible control node has login access to the managed nodes.
- SSH keys are the most common way to provide access, but other forms of authentication are also supported.



- The units of code that Ansible executes on the managed nodes are called modules.
- Each module is invoked by a task, and an ordered list of tasks together forms a playbook.
- Users write playbooks with tasks and modules to define the desired state of the system.



Basic Concepts & Terms

Host: A remote machine managed by Ansible.

Group: Several hosts grouped together that share a common attribute.

Inventory: A collection of all the hosts and groups that Ansible manages.

Modules: Units of code that Ansible sends to the remote nodes for execution.

Tasks: Units of action that combine a module and its arguments along with some other parameters.

Playbooks: An ordered list of tasks along with its necessary parameters that define a recipe to configure a system.

Roles: Redistributable units of organization that allow users to share automation code easier.

YAML: A popular and simple data format that is very clean and understandable by humans



All YAML files (regardless of their association with Ansible or not) can optionally begin with --- and end with .... This is part of the YAML format and indicates the start and end of a document.

All members of a list are lines beginning at the same indentation level starting with a "- " (a dash and a space):

```
---
# A list of tasty fruits
- Apple
- Orange
- Strawberry
- Mango
...
```



A dictionary is represented in a simple key: value form (the colon must be followed by a space):

```
# An employee record
martin:
  name: Martin D'vloper
  job: Developer
  skill: Elite
```

More complicated data structures are possible, such as lists of dictionaries, dictionaries whose values are lists or a mix of both:

```
# Employee records
- martin:
    name: Martin D'vloper
    job: Developer
    skills:
      - python
      - perl
      - pascal
- tabitha:
    name: Tabitha Bitumen
    job: Developer
    skills:
      - lisp
      - fortran
      - erlang
```

you can also specify a boolean value (true/false) in several forms:

create_key: true
needs_agent: false
knows_oop: True
likes_emacs: TRUE
uses_cvs: false



What Are Inventories In Ansible?

- Static lines of servers
- Dynamic list of servers: AWS, Azure, GCP, etc.
- The headings in brackets are group names

```
1   ## Ansible Hosts File for Lab
2
3   [CSR-Routers]
4   CSR-1 ansible_host=192.168.1.210
5   CSR-2 ansible_host=192.168.1.211
6   CSR-3 ansible_host=192.168.1.212
7
8   [SWITCHES]
9   2960 ansible_host=192.168.1.250
```

Here’s that same basic inventory file in YAML format:

```
ungrouped:
  hosts:
    mail.example.com:
webservers:
  hosts:
    foo.example.com:
    bar.example.com:
dbservers:
  hosts:
    one.example.com:
    two.example.com:
    three.example.com:
```

Default groups

- Even if you do not define any groups in your inventory file, Ansible creates two default groups: **all** and **ungrouped.**
- The **all** group contains every host. The **ungrouped** group contains all hosts that don’t have another group aside from all.
- Every host will always belong to **at least 2 groups** (all and ungrouped or all and some other group).
- For example, in the basic inventory above, the host **mail.example.c**om belongs to the all group and the ungrouped group; the host **two.example.com** belongs to the all group and the dbservers group.

Ansible Roles

1.Roles are a way to group tasks together into one container. We could have a role for setting up MySQL, another one for configuring ip tables.

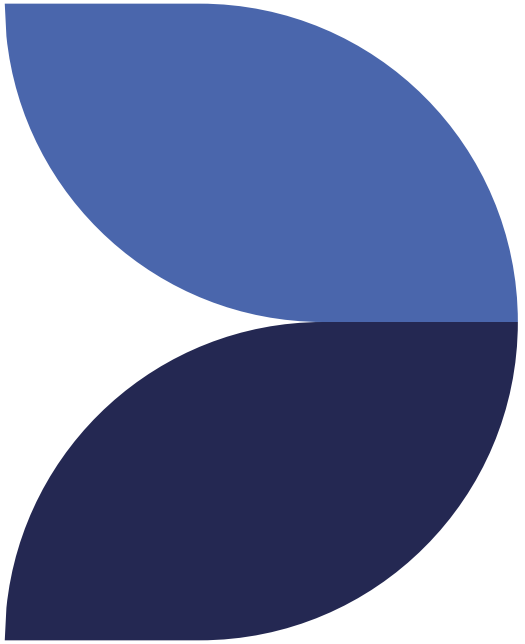
2.Roles make it easy to configure hosts. Any role can be performed on any host or group of hosts such as:

- *hosts: all*
- *roles:*
- *role_1*
- *role_2*



Create a Playbook

Let us start by writing a sample YAML file.



```
---
- hosts: all
  tasks:
    - name: Print message
      debug:
        msg: Hello Ansible World
```

- Tasks are defined as a list under the name tasks inside a play, at the same level as the hosts directive that defines the targets for that play. The name property defines the output that will be printed out when that task is about to be executed.

- The example task invokes the debug module, which allows you to display messages in a play. These messages can be used to show debug information such as the contents of a variable or the output message returned by a command, for instance.

- Each module has its own set of options and properties. The debug module expects a property named msg containing the message to be printed out. Pay special attention to the indentation (2 spaces), since msg must be a property inside debug.

```
---
- hosts: all
  vars:
    - username: YOUR_USERNAME
    - home: /home/YOUR_USERNAME
  tasks:
    - name: print variables
      debug:
        msg: "Username: {{ username }}, Home dir: {{ home
}}"
```

hosts: Specifies the target servers where the tasks will be executed.
become: yes: Instructs Ansible to run the tasks with elevated privileges (usually using sudo or root).
tasks: This section contains a list of tasks that Ansible will execute sequentially.

run the playbook using the following command.

$ansible-playbook <filename>.yaml

Installing Required Packages

git, nginx, sqlite3, mcrypt, and a couple of php5-* packages.

```
---
- hosts: php
  sudo: yes

  tasks:
  - name: install packages
    apt: name={{ item }} update_cache=yes state=latest
    with_items:
      - git
      - mcrypt
      - nginx
      - php5-cli
      - php5-curl
      - php5-fpm
      - php5-intl
      - php5-json
      - php5-mcrypt
      - php5-sqlite
      - sqlite3
```

```
---
- hosts: php
  sudo: yes
```

hosts: php

•This means the playbook will run on all hosts in the **php group** (as defined in your inventory file).

•Example: If your inventory has this:

```
[php]
192.168.1.100
192.168.1.101
```

An **inventory file** is where you define the list of **hosts (machines/servers)** that Ansible will manage. It tells Ansible *"These are the systems I want to run my playbooks on."*

Ansible inventory files can be in different formats:

Format	Common Extension	Description
INI	.ini or no extension	Simple format with groups and hostnames
YAML	.yml or .yaml	Preferred for dynamic or structured configuration
JSON	.json	Used in dynamic inventories or APIs

Static Inventory (INI Format)

```
[web]
192.168.1.10
192.168.1.11

[db]
dbserver1.example.com
dbserver2.example.com
```

Save it as:
inventory or inventory.ini

Static Inventory (YAML Format)

```
all:
  children:
    web:
      hosts:
        192.168.1.10:
        192.168.1.11:
    db:
      hosts:
        dbserver1.example.com:
        dbserver2.example.com:
```

Sudo: yes
Or become: yes

This line tells Ansible to run tasks as superuser (root) using sudo.

```
---
- hosts: php
  sudo: yes

  tasks:
   - name: install packages
     apt: name={{ item }} update_cache=yes
state=latest
     with_items:
       - git
       - mcrypt
       - nginx
       - php5-cli
       - php5-curl
       - php5-fpm
       - php5-intl
       - php5-json
       - php5-mcrypt
       - php5-sqlite
       - sqlite3
```

tasks:This begins the list of actions (tasks) that Ansible will perform on the target hosts.

- name: install packages

This is a human-friendly description of the task.

```
apt: name={{ item }} update_cache=yes
state=latest
    with_items:
      - git
      - mcrypt
      - nginx
      - php5-cli
      - php5-curl
      - php5-fpm
      - php5-intl
      - php5-json
      - php5-mcrypt
      - php5-sqlite
      - sqlite3
```

•This task uses the **apt module**, which is used to install packages on Debian/Ubuntu-based systems.

Breakdown of parameters:

•name={{ item }}: Installs the current item in the loop (see with_items below).

•update_cache=yes: Runs apt-get update before installing to get the latest package lists.

•state=latest: Ensures the package is installed and **up to date** with the latest version.

with_items:

•This is a **loop** in Ansible.

•It means: **Repeat this task for each item in the list** below.

Let's suppose we want to install the latest OpenSSL version in our ubuntu server. these are the command we would execute in our server

```
apt update
apt install openssl
(or)
apt-get update
apt-get install openssl
```

With an ansible apt module,

```
---
 - name: Ansible apt module examples
   hosts: web
   become: true
   tasks:
    - name: Ansible Update Cache and install openssl
      apt:
        name: openssl
        state: present
        update_cache: yes
```

An outdated cache can lead apt to miss updates or to fail to install a package altogether (because the version it wants is no longer available from the repositories).

update_cache=yes tells Ansible's apt module to refresh the caches before applying whatever change is necessary (if any).

Let us suppose we want to upgrade the OpenSSL version installed in our system and this is the command we would execute on our servers

```
# Update the Cache
apt update
# Upgrade the package openssl
apt upgrade openssl (or)
apt-get  update
apt-get upgrade openssl
```

```
---
 - name: Ansible apt module examples
   hosts: web
   become: true
   tasks:
    - name: Ansible Update Cache and upgrade openssl
      apt:
        name: openssl
        state: latest
        update_cache: yes
```

If you have not noticed it already. the **state: latest** parameter would make sure that the package is upgraded and stay the latest.

This is the playbook we are going to use to install / setup a LAMP (LinuxApacheMysqlPhp) stack on our Debian/Ubuntu server

```
- name: Ansible apt module examples
  hosts: web
  become: yes
  tasks:
    - name: Ansible apt to install multiple packages - LAMP
      register: updatesys
      apt:
        update_cache: yes
        name:
          - firewalld
          - apache2
          - mariadb-server
          - php
          - php-mysql
        state: present
```

let's see how to specify the package version with ansible apt module.

```
---
 - name: Ansible apt module examples
   hosts: web
   become: true
   tasks:
    - name: Ansible apt install tomcat Version 8.0.32
      register: updatesys
      apt:
        update_cache: yes
        name: tomcat8=8.0.32-1ubuntu1.11
        state: present
```

here is the sample playbook to validate if few packages are installed or not.

```
---
 - name: Ansible validate if the packages are installed
   hosts: web
   become: true
   tasks:

    - name: Gather Package facts
      package_facts:
        manager: auto

    - name: Validating if the package is installed or not
      debug:
        msg: "{{item}} is installed"
      when: '"{{item}}" in ansible_facts.packages'
      with_items:
        - apache2
        - nginx
        - filebeat
```

prints only apache2 is installed

It clears the useless packages from cache and remove the dependency tools which are no longer required.

```
- name: Remove useless packages from the cache
  apt:
    autoclean: yes

- name: Remove dependencies that are no longer required
  apt:
    autoremove: yes
```

```
- name: Clone git repository
  git: >
    dest=/var/www/laravel
    repo=https://github.com/laravel/laravel.git
    update=no
  sudo: yes
  sudo_user: www-data
```

Loops

Ansible allows us to iterate over a set of items in a task to execute it multiple times with different parameters without rewriting it. For example, to create several files, we would use a task that iterates over a list of directory names instead of writing five tasks with the same module.

To iterate over a simple list of items, use the loop keyword. We can reference the current value with the loop variable item.

```
- name: "Create some files"
  ansible.builtin.file:
    state: touch
    path: /tmp/{{ item }}
  loop:
    - example_file1
    - example_file2
    - example_file3
```

It’s also possible to iterate over dictionaries:

```
- name: "Create some files with dictionaries"
  ansible.builtin.file:
    state: touch
    path: "/tmp/{{ item.filename }}"
    mode: "{{ item.mode }}"
  loop:
    - { filename: 'example_file1', mode: '755'}
    - { filename: 'example_file2', mode: '775'}
    - { filename: 'example_file3', mode: '777'}
```

Install and configure a web server

The first thing that the automation should handle is the installation and configuration of NGINX, the chosen web server. The **roles/webserver/tasks/main.yml** file shown below is a straightforward way to accomplish this:

```
---
- name: Install packages
  package:
    name: "{{ webserver_packages }}"
    state: latest

- name: Add base NGINX configuration file
  copy:
    dest: /etc/nginx/nginx.conf
    src: etc/nginx/nginx.conf
    owner: root
    group: root
    mode: 0644
  notify: Reload NGINX
```

The webserver_packages variable is defined in the **roles/webserver/vars/main.yml file**:

```
---

webserver_packages:
  - nginx
```

State as '**Present**' and '**Installed**' are used interchangeably. They both do the same thing i.e. it will ensure that a desired package in your case 'yum' is installed.

Whereas State as '**Latest**' means in addition to installation, it will go ahead and update if it is not of the latest available version.

Whenever you are building your stack/app or working on production, it is always advisable to use '**Present**' or '**Installed**' state.

- **Vagrant** uses a Vagrantfile, which is written in **Ruby**.
- **Ansible** uses .yml or .yaml files, which follow the **YAML** syntax.

Example: Vagrant calling Ansible

Here's a minimal Vagrantfile (Ruby) that uses Ansible (YAML) for provisioning:

Vagrantfile

ruby

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"

  config.vm.provision "ansible" do |ansible|
    ansible.playbook = "provision.yml"
  end
end
```

provision.yml (Ansible playbook)

yaml

```
---
- name: Provision VM with Apache
  hosts: all
  become: yes
  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present
        update_cache: yes
```

What happens?

1. vagrant up — starts a VM using the Ubuntu box
2. Vagrant checks the provisioner (here, Ansible)
3. Vagrant runs ansible-playbook provision.yml automatically **inside or against the VM**
4. Ansible provisions the VM — installing Apache in this case

To Dos

Try deploying simple web application on some web server using ansbile

Docker

- Docker is an open platform that enables users to develop, ship, and run applications with ease.
- Docker software is packaged as containers- a docker standardized unit.
- These containers have all the elements, such as system tools, libraries, runtime, etc required by the software.
- Docker container is an executable package of applications and its dependencies bundled together



- Difference from virtual machine:
  - With a virtual machine, **the memory usage is very high**, whereas, in a Docker environment, memory usage is very low.
  - In terms of portability, virtual machines just are not ideal. They’re still dependent on the host operating system, and a lot of problems can happen when you use virtual machines for **portability**.
  - In contrast, Docker was designed for portability. You can actually build solutions in a Docker container, and the solution is guaranteed to work as you have built it no matter where it’s hosted.



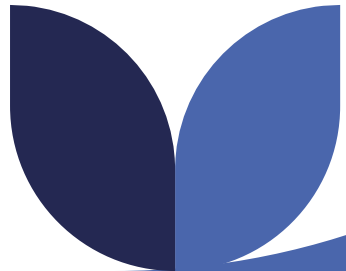
- One of the other challenges of using a virtual machine is that if you have unused memory within the environment, you cannot reallocate it.
- If you set up an environment that has 9 gigabytes of memory, and 6 of those gigabytes are free, you cannot do anything with that unused memory.
- With Docker, if you have free memory, you can reallocate and reuse it across other containers used within the Docker environment.
- The boot-up time for a virtual machine is about a few minutes, in contrast to the milliseconds it takes for a Docker environment to boot up.



Vagrant is more resource heavy

- Because Vagrant uses existing virtualization software under the hood (it can work on top of VirtualBox, VMWare and Hyper-V among others), it seems that Docker has an advantage in being more lightweight as it uses Linux kernel features instead of hypervisors.
- But if it requires Linux, how come Docker also runs on Windows and macOS? As it turns out, Docker Desktop for Windows and for Mac use virtualization (Hyper-V and HyperKit respectively) to run the Linux kernel, so it makes Docker lose its lightweight appeal

Vagrant is available on all major platforms i.e. Windows, macOS and Linux, and its installation is fairly straightforward. However, it requires us to have a hypervisor already installed - VirtualBox is probably the best option as it's free, open-source, well supported and runs on all previously mentioned systems



Why Vagrant?

Ideal for Legacy Applications

"Vagrant works better when dealing with legacy software that can’t be containerized easily or doesn’t run well in minimal Linux containers."

Many **older/legacy applications** require:

- A full OS environment
- Specific kernel versions or systemd
- GUI components or services that don’t run well in containers

Example:
You’re working on a legacy **Java web app** from 2009 that:
•Requires **Ubuntu 16.04**
•Needs **Apache2**, **Java 7**, and an old **MySQL 5.5**
•Has **init.d** scripts instead of systemd

Docker Struggles:
•Alpine or slim containers don’t support init.d properly
•Finding or building a container image with Java 7 and MySQL 5.5 is painful
•OS-level services don’t behave normally inside a container

Vagrant Solution

You now have a **real Ubuntu 16.04 VM**, behaving just like a physical server.

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/xenial64"
  config.vm.provision "shell", inline: <<-SHELL
    sudo apt-get update
    sudo apt-get install -y openjdk-7-jdk mysql-server apache2
  SHELL
end
```

<<-SHELL ... SHELL

Everything between <<-SHELL and SHELL is treated as a multi-line string.The name SHELL is just a label — it could be SCRIPT, EOF, or anything.The important part is that the opening and closing marker must match.

This is a **Ruby feature** called a **"heredoc"**

```
<<-MARKER
  multi-line string content
MARKER
```

telling Vagrant:
“Run these shell commands **inside the VM** right after it's created.”

Isolation from Host System

"Vagrant VMs are fully isolated; nothing leaks into your host system unless you configure shared folders. Docker shares the host kernel, which can introduce conflicts."

- Docker containers share your host’s kernel and networking
- This can lead to port conflicts, file permission issues, or even security concerns
- Vagrant, by using VirtualBox or another hypervisor, gives each VM its own virtual hardware (memory, disk, network, etc.)

Example:
You have two projects:
•Project A runs **PostgreSQL 13**
•Project B runs **PostgreSQL 9.6** (old version)

Docker Risk:
Running both containers on Docker on the same host:
•Might conflict on ports (e.g., both want port 5432)
•Might use shared volumes with mismatched permissions
•Share same kernel — risky if an image has an exploit

Vagrant Solution:

Each project runs in its **own isolated VM**:

```
# Project A - PostgreSQL 13
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/focal64"
  config.vm.network "private_network", ip: "192.168.56.10"
  config.vm.provision "shell", inline: <<-SHELL
    sudo apt install -y postgresql-13
  SHELL
end
```

```
# Project B - PostgreSQL 9.6
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"
  config.vm.network "private_network", ip: "192.168.56.11"
  config.vm.provision "shell", inline: <<-SHELL
    sudo apt install -y postgresql-9.6
  SHELL
end
```

Each VM has its own kernel, IP, and environment. No port clash, no kernel issues.

Software Re-Engineering

Lecture 13

Reverse engineering I – Basics and Introduction to assembly language

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Reverse engineering, also called backwards engineering or back engineering, is the process by which an artificial object is deconstructed to reveal its designs, architecture, code, or to extract knowledge from the object.

It’s similar to disassembling the parts of a vehicle to understand the basic functioning of the machine and internal parts etc. and thereafter making appropriate adjustments to give rise to a better performing or superior vehicle.

- In software design, reverse engineering enables the developer or programmer to add new features to the existing software with or without knowing the source code. Different techniques are used to incorporate new features into the existing software.
- Reverse engineering is also very beneficial in software testing, as most of the virus programmers don’t leave behind instructions on how they wrote the code, what they have set out to accomplish etc. Reverse engineering helps the testers to study the virus and other malware code.
- The third category where reverse engineering is widely used is in software security. Reverse engineering techniques are used to make sure that the system does not have any major vulnerabilities and security flaws. The main purpose of reverse engineering is to make the system robust so as to protect it from spywares and hackers. Infact, this can be taken a step forward to Ethical hacking, whereby you try to hack your own system to identify vulnerabilities.

Reverse Engineering Tools

The process of reverse engineering is accomplished by making use of some tools that are categorized into debuggers or disassemblers, hex editors, monitoring and decompile tools

Disassemblers – A disassembler is used to convert binary code into assembly code and also used to extract strings, imported and exported functions, libraries etc. The disassemblers convert the machine language into a user-friendly format. There are different dissemblers that specialize in certain things.

Debuggers – This tool expands the functionality of a disassembler by supporting the CPU registers, the hex duping of the program, view of stack etc. Using debuggers, the programmers can set breakpoints and edit the code at run time. Debuggers analyse the binary in a similar way as the disassemblers and allow the reverser to step through the code by running one line at a time to investigate the results.

With a disassembler you can take a binary and see exactly how it functions (static analysis). Whereas, with a debugger we can step through, break and edit the assembly while it is executing (dynamic analysis).

debugger gives you live execution details, such as variables (like enemy positions on the map), values in memory, etc.

Hex Editors – These editors allow the binary to be viewed in the editor and change it as per the requirements of the software.

Decompile-To decompile means to convert executable or ready-to-run program code -- sometimes called object code -- into some form of higher-level programming language that humans can easily understand. Decompilation is a type of reverse-engineering that performs the opposite operations of a compiler.

A disassembler generates code in a low-level language, while a decompiler generates code in a higher-level programming language.

- The reverse engineering learning process is similar to that of foreign language acquisition for adults.
- The first phase of learning a foreign language begins with an introduction to letters in the alphabet, which are used to construct words with well-defined semantics.
- The next phase involves understanding the grammatical rules governing how words are glued together to produce a proper sentence.
- After being accustomed to these rules, one then learns how to stitch multiple sentences together to articulate complex thoughts.
- Eventually it reaches the point where the learner can read large books written in different styles and still understand the thoughts therein

- In reverse engineering, the language is the architecture and assembly language.
- A word is an assembly instruction. Paragraphs are sequences of assembly instructions.
- A book is a program. However, to fully understand a book, the reader needs to know more than just vocabulary and grammar.
- These additional elements include structure and style of prose, unwritten rules of writing, and others

Books:

- **Practical Reverse Engineering: X86, X64, ARM, Windows Kernel, Reversing Tools, and Obfuscation** by Alexandre Gazet, Bruce Dang, and Elias Bachaalany
- **The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition** by Chris Eagle

Basics and background knowledge

x86 and x64

- x86 supports the concept of privilege separation through an abstraction called ring level.
- The processor supports four ring levels, numbered from 0 to 3. (Rings 1 and 2 are not commonly used so they are not discussed here.)
- Ring 0 is the highest privilege level and can modify all system settings.
- Ring 3 is the lowest privileged level and can only read/modify a subset of system settings.
- Hence, modern operating systems typically implement user/kernel privilege separation by having user-mode applications run in ring 3, and the kernel in ring 0

- “byte” refers to a one-byte integer (suffix b),
- “word” refers to a two-byte integer (suffix w), “doubleword” refers to a four-byte integer (suffix l), and
- “quadword” refers to an eight-byte value (suffix q).

Processor operations mostly involve processing data. This data can be stored in memory and accessed from thereon. However, reading data from and storing data into memory slows down the processor, as it involves complicated processes of sending the data request across the control bus and into the memory storage unit and getting the data through the same channel.

To speed up the processor operations, the processor includes some internal memory storage locations, called registers.

Memory Segments

- To reference a memory location its logical address has to be specified. The logical address is written as:

Segment number:offset

- For example, A43F:3487h means offset 3487h within segment A43Fh.

Program Segments

Machine language programs usually have 3 different parts. Each of these parts is stored in different memory segments:

Instructions: This is the code part and is stored in the code segment

Data: This is the data part which is manipulated by the code and is stored in the data segment

Stack: The stack is a special memory buffer outside the CPU that is maintained by the CPU as a temporary holding area for addresses and data. It is organized as Last-In-First-Out (LIFO) buffer and is used by the CPU to implement procedure calls. This data structure is stored in the stack segment

- The segment numbers for the code segment, the data segment, and the stack segment are stored in the segment registers CS, DS, and SS, respectively

- A fourth segment register, the ES, or extra segment is provided for programs that need to access a second data segment

- Segment registers cannot be used in arithmetic operations

Addressing modes

When writing assembly programs, the instructions being written require operands. There are several ways to specify the location of the operands. These are called the addressing modes. Depending on where an operand used in instruction is located, the following are the various addressing modes available.

Register addressing mode: When an operand is in a register, it is called register addressing mode. This is the most efficient way of specifying operands as memory access is not required and operands are within the processor. Note the following example, where both the operands in the instruction are process registers.

```
MOV EAX, ECX
```

In the preceding instruction, the contents of the source operand (ECX) will be copied to the destination operand (EAX).

Contd..

Immediate addressing mode: When data is specified as part of the instruction itself, it is called immediate addressing mode. Note how the following instruction contains a constant value as the second operand.

```
mov EAX, 2
```

In the preceding instruction, the value 2 will be stored in the source operand, which is EAX register.

Memory addressing mode: When an operand refer to a memory address, it is memory-addressing mode. As the name suggests, operating in this mode requires access to the memory. Note the following example, where a value referenced by the address of EAX is being moved into the register EBX.

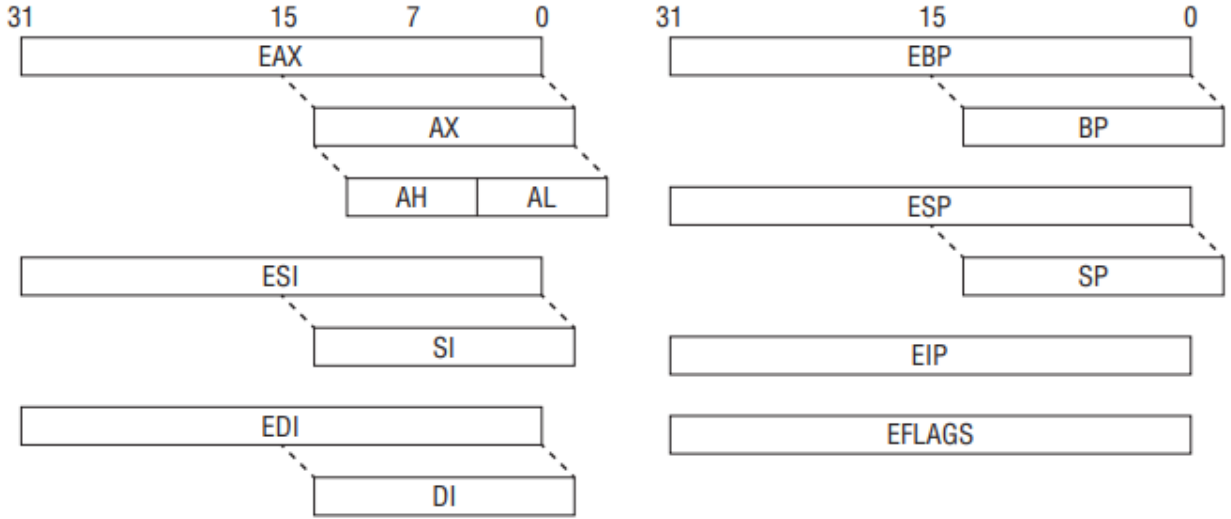
```
MOV EBX, [EAX]
```

Register Set and Data Types

- The x86 architecture provides ten 32-bit registers.
- These registers are grouped into **general purpose, control,** and **segment registers**.
- The general-purpose registers are further divided into **data, pointer,** and **index registers.**
- Data Registers: Data registers are used for arithmetic, logical and other operations. There are four data registers as listed below.

  EAX, EBX, ECX and EDX

It is possible to use a 32-bit register and access the lower half of the data by the corresponding 16-bit register name. For example, the lower 16 bits of EAX can be accessed by using AX. Similarly, the lower two bytes can be individually accessed by using the 8-bit register names. For example, the lower byte of AX can be accessed as AL and the upper byte as AH.



AX (accumulator): This is the accumulator register (it can hold an operand to an arithmetic operation and the result of the operation).

For example: If **1100110001101100** is moved to AX, then the value of AL would be:

0 1 1 0 1 1 0 0

and the value of AH would be:

1 1 0 0 1 1 0 0

MOV AX, 10
ADD AX, AX

In this example, the MOV instruction moves the value 10 into the AX register. The ADD instruction then adds the value in the AX register to itself, effectively doubling it.

BX (base register): This register can hold the address of a procedure or a variable. It can also perform arithmetic and data movement. Again as with the AX, this register can be addressed as a 16-bit value BX, as an 8-bit value BH corresponding to the higher 8-bits of BX, or as BL corresponding to the lower 8-bits of BX.

CX (counter): This register acts as a counter for repeating or looping instructions. These instructions automatically repeat and decrement CX. Similar to AX and BX, it is possible to address the 16-bit value as CX, the upper 8-bits value as CH or the lower 8-bits as CL.

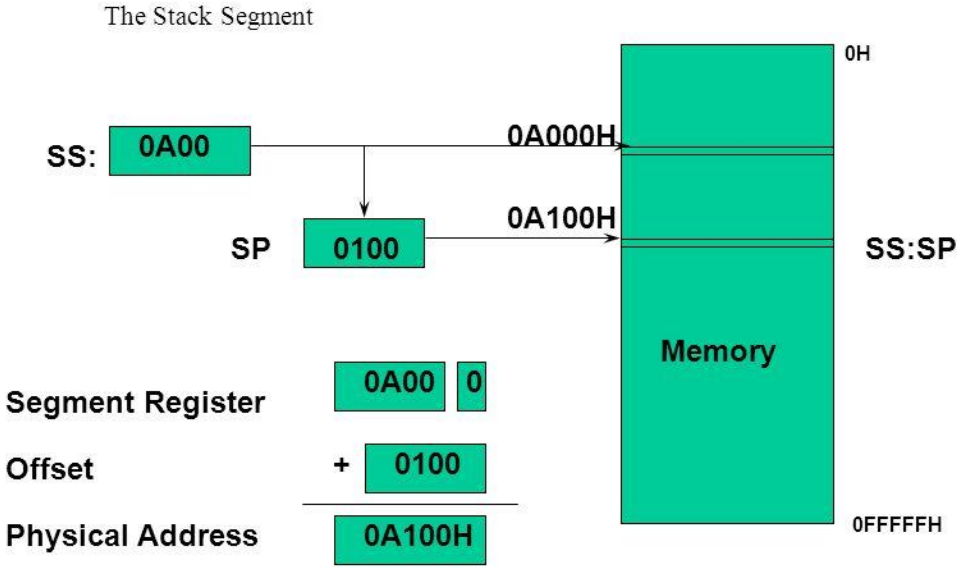
DX (data): This register has a special role in multiply and divide operations. When multiplying, for example, it holds the upper 16-bits of the product. Again, the 16-bit value can be addressed as DX, the upper 8-bits value as DH or the lower 8-bits as DL.

Pointer and Index Registers

- These registers contain the offsets of data and instructions
- The offset refers to the distance of a variable, label, or instruction from its base segment

SP (stack pointer):

- Contains an assumed offset value of the top of the stack
- Combined with the SS (Stack segment) register form the complete logical address of the top of the stack
- The stack is maintained as a LIFO with
  - ❑ its bottom at the start of the stack segment (specified by the SS segment register)
  - ❑ The top of the stack (the location of the last data in the stack) is specified by the offset stored in the SP register



The offset is given by the SP register.
The stack is always referenced with respect to the stack segment register.
The stack grows toward decreasing memory locations.
The SP points to the last or top item on the stack.

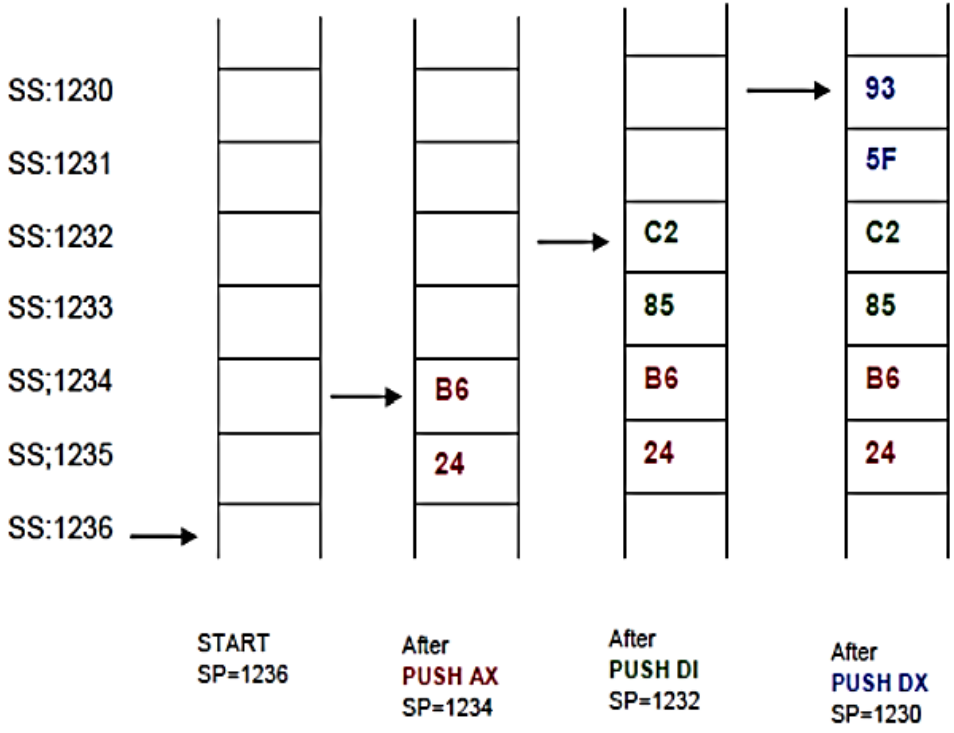
PUSH - pre-decrement the SP
POP - post-increment the SP

BP (base pointer):

- points to the bottom of the stack. This is used to set up and manage information on the stack during a procedure call.
- BP can be used as a base address in the stack area to determine the operand address on the stack

Example: SP=1236H, AX=24B6H, DI=85C2H, and DX=5F93H, show the contents of the stack as each of the following instructions is executed.

```
PUSH    AX
PUSH    DI
PUSH    DX
```



Solution:

Note that in 80x86 the lower byte of the register is stored to the lower address. **(Little Endian Convention)**

SI (source index):

- SI is the source index, used as a pointer to the current character being read in a string instruction (LODS, MOVS, or CMPS). It is also available as an offset to add to BX or BP when doing indirect addressing; for example, the instruction MOV [BX+SI], AX copies the contents of AX into the memory location whose address is the sum of the contents of BX and SI.

DI (destination index):

- DI is the destination index, used as a pointer to the current character being written or compared in a string instruction. It is also available as an offset, just like SI.

IP (Instruction Pointer):

- This is the register used for accessing instructions
- IP register gets updated by the control unit every time an instruction is executed such that it will always point to the next instruction
- Unlike other registers, the IP can not be manipulated by instructions (i.e. it cannot appear as an operand in any instruction)

In terms of **data movement** between registers and memory, the movement can be classified into five general methods:

- Immediate to register
- Register to register
- Immediate to memory
- Register to memory and vice versa
- Memory to memory

- Instructions operate on values that come from registers or main memory. The most common instruction for moving data is MOV. The simplest usage is to move a register or immediate to register.

- Data movement instructions destroy the previous contents of the destination.
- A **load** instruction loads a register from memory.
- A **store** instruction stores the contents of a register into memory.
- A **transfer** instruction loads a register from another register.
- In processors that have separate names for different kinds of data moves, a memory to memory data move might be specially designated as a “move” instruction.

x86 uses square brackets ([]) to indicate memory access

```
02: 8B 08              mov   ecx, [eax]
; set ECX to the value at address EAX
03: 89 18              mov   [eax], ebx
; set the memory at address EAX to EBX
04: 89 46 34           mov   [esi+34h], eax
; set the memory address at (ESI+34) to EAX
05: 8B 46 34           mov   eax, [esi+34h]
; set EAX to the value at address (EAX+34)
06: 8B 14 01           mov   edx, [ecx+eax]
; set EDX to the value at address (ECX+EAX)
```

1. Register to Register

```
mov ax, 10 # move 10 to register 'ax'
mov bx, 5  # move 5 to register 'bx'
add ax, bx
```

2. Memory to Register

```
mov ax, 10  # move 10 to register 'ax'
add ax, [num1] # access value stored at location num1 and store sum
               # again in the register 'ax'
```

3. Register to Memory

```
mov ax, 10
add [num1], ax
```

4. Constant to Register

```
mov ax, 10
add ax, 5 # add 5 to the register 'ax' value and store the sum in 'ax'
```

5. Constant to Memory

```
add [num1], 5 # add 5 to the value stored at num1
```

- The inc and dec instructions can be used to increment or decrement the contents of their respective operands by one
- mul performs an unsigned multiplication of the source operand and the accumulator.\

operation source

- If the source operand is a byte (8 bits), we multiply it by the value stored in the register AL. The result is returned in the registers AH and AL. The higher half of the 16-bit result is stored in AH and the lower half in AL. This means that if the result is small enough to be represented in 8 bits, AH would contain 0.

```
# multiplication - 8 bit source
mov al, 5
mov bl, 10
mul bl      # result in ax

# refreshing registers
mov ax, 0
mov bx, 0
```

operation source

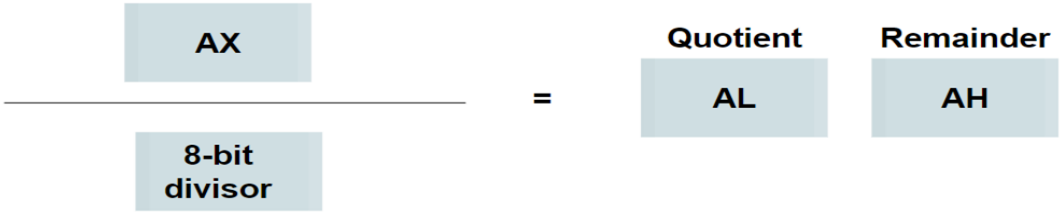
- If the source operand is a word (16 bits), then we multiply it by the value stored in the register AX and the result is returned in the registers DX and AX. The higher half of the 32-bit result is stored in DX and the lower half in AX.

```
# multiplication - 16 bit source
mov ax, 5
mov bx, 10
mul bx      # result in dx:ax

# refreshing registers
mov ax, 0
mov bx, 0
```

- div performs an integer division of the accumulator and the source operand. The result consists of an integer quotient and remainder.

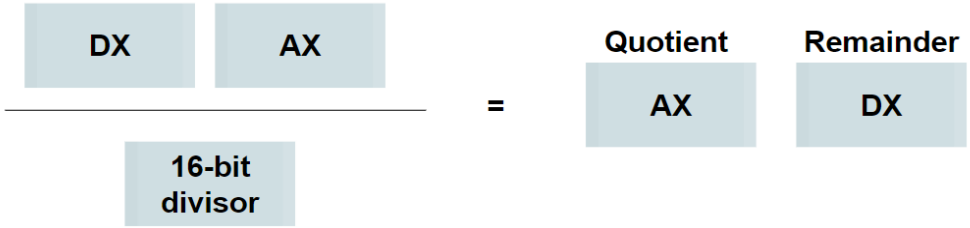
- source operand is a byte, the 16-bit number stored in AX is divided by the operand. The 8-bit quotient is stored in AL and the 8-bit remainder in AH.




```
# divison - 8 bit source
mov al, 23
mov bl, 4
div bl      # quotient in al, remainder in ah

# refreshing registers
mov ax, 0
mov bx, 0
```

- If the source operand is a word, the 32-bit number stored in DX : AX is divided by the operand. The higher half of the number is stored in DX and the lower in AX. The 16-bit quotient is stored in AX and the remainder in DX.




```
# divison - 16 bit source
mov ax, 23
mov bx, 4
div bx      # quotient in ax, remainder in dx

mov ax, 0x4c00  # terminate program
int 0x21
```

a program to show arithmetic instructions in AL

```
# addition
mov ax, 5   # load number in ax
mov bx, 2   # load number in bx
add ax, bx  # accumulate sum in ax

# subtraction
mov cx, 10
mov dx, 3
sub cx, dx  # accumulate difference in cx

# refreshing registers
mov ax, 0
mov bx, 0
mov cx, 0
mov dx, 0
```

Stack Operations and Function Invocation

- Push means to put something on top of the stack;
- pop means to remove an item from the top.
- Concretely speaking, on x86, a stack is a contiguous memory region pointed to by ESP and it grows downwards.
- Push/pop operations are done through the PUSH/POP instructions and they implicitly modify ESP.
- The PUSH instruction decrements ESP and then writes data at the location pointed to by ESP;
- POP reads the data and increments ESP.

push ax ; Pushes the value of the AX register onto the stack

pop ax ; Pops the top value from the stack into the AX register

Conditional Instructions

CMP Instruction

The CMP instruction compares two operands. It is generally used in conditional execution. It is used along with the conditional jump instruction for decision making.

Syntax

CMP destination, source

CMP compares two numeric data fields. The destination operand could be either in register or in memory. The source operand could be a constant (immediate) data, register or memory.

Example

```
CMP DX, 00  ; Compare the DX value with zero
JE  L7      ; If yes, then jump to label L7
.
.
L7: ...
```

```
INC     EDX
CMP     EDX, 10 ; Compares whether the counter has reached 10
JLE     LP1     ; If it is less than or equal to 10, then jump to LP1
```

Unconditional Jump

JMP label

Example

The following code snippet illustrates the JMP instruction −

```
MOV  AX, 00    ; Initializing AX to 0
MOV  BX, 00    ; Initializing BX to 0
MOV  CX, 01    ; Initializing CX to 1
L20:
ADD  AX, 01    ; Increment AX
ADD  BX, AX    ; Add AX to BX
SHL  CX, 1     ; shift left CX, this in turn doubles the CX value
JMP  L20       ; repeats the statements
```

Conditional Jump

Instruction	Description	Flags tested
JE/JZ	Jump Equal or Jump Zero	ZF
JNE/JNZ	Jump not Equal or Jump Not Zero	ZF
JG/JNLE	Jump Greater or Jump Not Less/Equal	OF, SF, ZF
JGE/JNL	Jump Greater/Equal or Jump Not Less	OF, SF
JL/JNGE	Jump Less or Jump Not Greater/Equal	OF, SF
JLE/JNG	Jump Less/Equal or Jump Not Greater	OF, SF, ZF

Instruction	Description	Flags tested
JE/JZ	Jump Equal or Jump Zero	ZF
JNE/JNZ	Jump not Equal or Jump Not Zero	ZF
JG/JNLE	Jump Greater or Jump Not Less/Equal	OF, SF, ZF
JGE/JNL	Jump Greater/Equal or Jump Not Less	OF, SF
JL/JNGE	Jump Less or Jump Not Greater/Equal	OF, SF
JLE/JNG	Jump Less/Equal or Jump Not Greater	OF, SF, ZF

The assembly language uses JMP instruction to implement **loops**. However, the processor set can use the LOOP instruction to implement loops conveniently.

- The loop instruction always extracts the number of iterations from the ECX register.
- The loop instruction decrements the value of ECX and compares it with zero.
- If the value in ECX is not equal to zero, the program jumps to the L1 label; otherwise, the program exits the loop.

```
mov ECX 5    ; store the number of iterations in ECX register
L1:
(loop code)
loop l1
```

Readings:

- **Chapter 1, Practical Reverse Engineering: X86, X64, ARM, Windows Kernel, Reversing Tools, and Obfuscation** by Alexandre Gazet, Bruce Dang, and Elias Bachaalany

Software Re-Engineering

Lecture 14

Reverse engineering II – Introduction to ARM, moving btw Assembly and c code

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Advanced RISC Machine

- Arm is RISC (Reduced Instruction Set Computing) based, while x86 is CISC (Complex Instruction Set Computing).
- Arm’s CPU instructions are reasonably atomic, with a very close correlation between the number of instructions and micro-ops. CISC, by comparison, offers many more instructions, many of which execute multiple operations (such as optimized math and data movement).
- This leads to better performance but more power consumption in decoding these complex instructions.

LDR and STR

```
01: 03 68          LDR          R3, [R0] ; R3 = *R0
02: 23 60          STR          R3, [R4] ; *R4 = R3;
```

For the instruction in line 1, R0 is the base register and R3 is the destination; it loads the word value at address R0 into R3.

In line 2, R4 is the base register and R3 is the destination; it takes the value in R3 and stores at the memory address R4.

At a fundamental level, the LDR/STR instructions take a base register and an offset

STR Ra, [Rb, imm]
Rb is the base register, and imm is the offset to be added to Rb

functional assembly program:

```
.data          /* the .data section is dynamically created and its addresses cannot be easily predicted */
var1: .word 3  /* variable 1 in memory */
var2: .word 4  /* variable 2 in memory */

.text          /* start of the text (code) section */
.global _start

_start:
    ldr r0, adr_var1  @ load the memory address of var1 via label adr_var1 into R0
    ldr r1, adr_var2  @ load the memory address of var2 via label adr_var2 into R1
    ldr r2, [r0]      @ load the value (0x03) at memory address found in R0 to register R2
    str r2, [r1]      @ store the value found in R2 (0x03) to the memory address found in R1
    bkpt

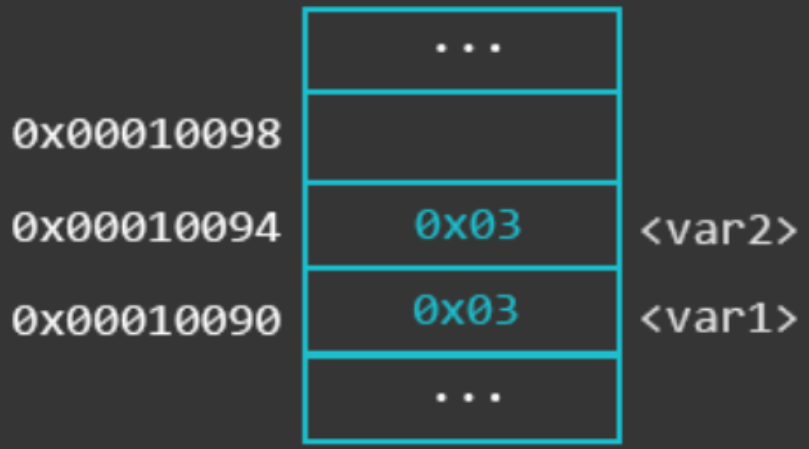
adr_var1: .word var1  /* address to var1 stored here */
adr_var2: .word var2  /* address to var2 stored here */
```

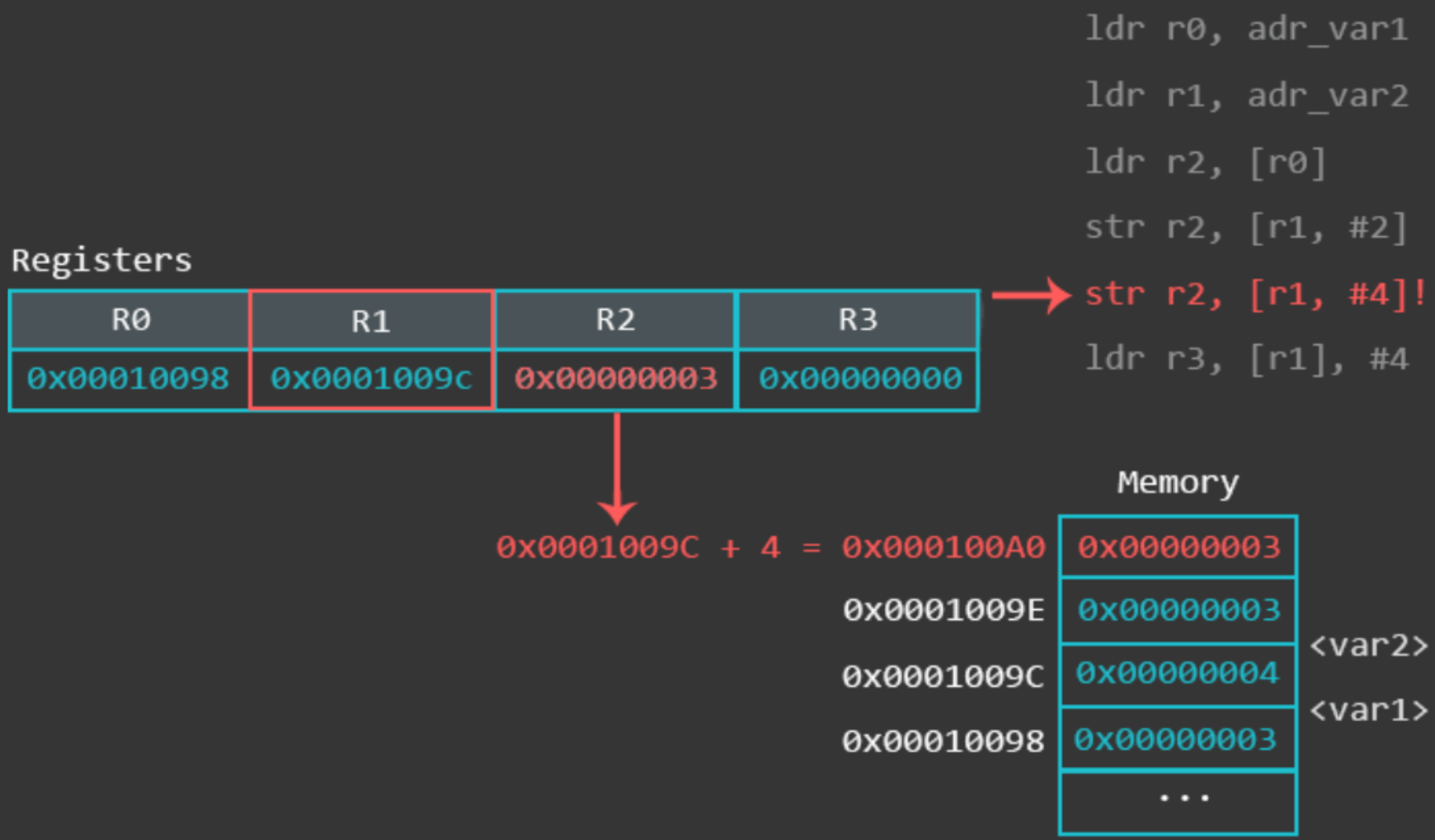
```
ldr   r0, addr_var1
ldr   r1, addr_var2
ldr   r2, [r0]
str   r2, [r1]
```

Registers

R0	R1	R2
0x00010090	0x00010094	0x00000003

Memory

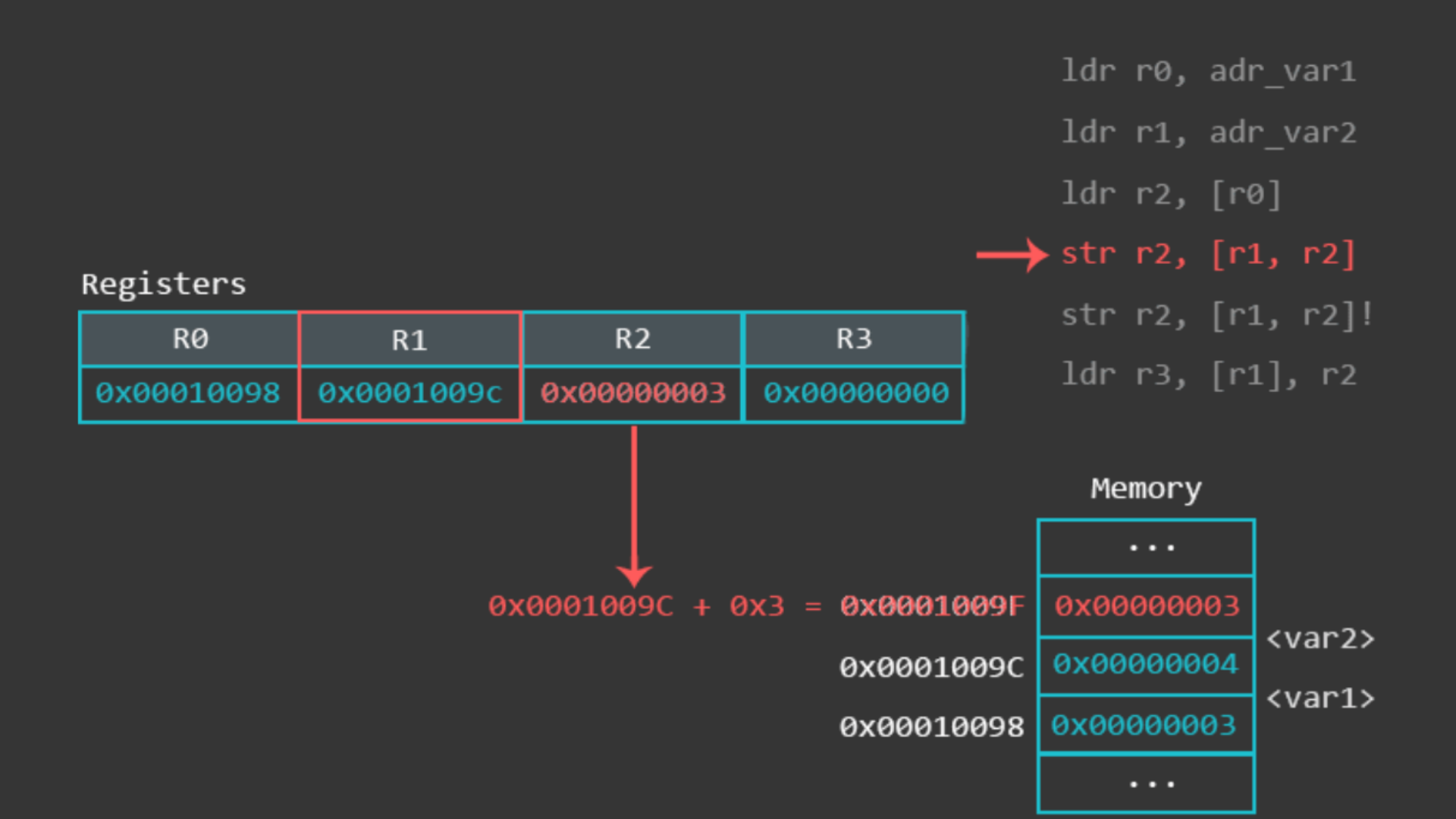



ldr r0, adr_var1
ldr r1, adr_var2
ldr r2, [r0]
str r2, [r1, #2]
str r2, [r1, #4]!
ldr r3, [r1], #4
Registers
R0
R1
R2
R3
0x00010098
0x0001009c
0x00000003
0x00000000
0x0001009C + 4 = 0x000100A0
Memory
0x00000003
0x0001009E
0x00000003
<var2>
0x0001009C
0x00000004
<var1>
0x00010098
0x00000003
...

Register as the offset.

```
STR    Ra, [Rb, Rc]
LDR    Ra, [Rb, Rc]
```

```
_start:
    ldr r0, adr_var1  @ load the memory address of var1 via label adr_var1 to R0
    ldr r1, adr_var2  @ load the memory address of var2 via label adr_var2 to R1
    ldr r2, [r0]      @ load the value (0x03) at memory address found in R0 to R2
    str r2, [r1, r2]  @ address mode: offset. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 (0x03). Base register unmodified.
    str r2, [r1, r2]! @ address mode: pre-indexed. Store value found in R2 (0x03) to the memory address found in R1 with the offset R2 (0x03). Base register modified: R1 = R1+R2.
    ldr r3, [r1], r2  @ address mode: post-indexed. Load value at memory address found in R1 to register R3. Then modify base register: R1 = R1+R2.
    bx lr
```


ldr r0, adr_var1
ldr r1, adr_var2
ldr r2, [r0]
str r2, [r1, r2]
str r2, [r1, r2]!
ldr r3, [r1], r2
Registers
R0
R1
R2
R3
0x00010098
0x0001009c
0x00000003
0x00000000
0x0001009C + 0x3 = 0x0001009F
Memory
...
0x00000003
<var2>
0x0001009C
0x00000004
<var1>
0x00010098
0x00000003
...

Scaled register as the offset

LDR Ra, [Rb, Rc, <shifter>]
STR Ra, [Rb, Rc, <shifter>]

```
_start:
    ldr r0, adr_var1        @ load the memory address of var1 via label adr_var1 to R0
    ldr r1, adr_var2        @ load the memory address of var2 via label adr_var2 to R1
    ldr r2, [r0]          @ load the value (0x03) at memory address found in R0 to R2
    str r2, [r1, r2, LSL#2]  @ address mode: offset. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 left-shifted by 2. Base register (R1) unmodified.
    str r2, [r1, r2, LSL#2]! @ address mode: pre-indexed. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 left-shifted by 2. Base register modified: R1 = R1 + R2<<2
    ldr r3, [r1], r2, LSL#2  @ address mode: post-indexed. Load value at memory address found in R1 to the register R3. Then modifiy base register: R1 = R1 + R2<<2
```

```
ldr r0, adr_var1
ldr r1, adr_var2
ldr r2, [r0]
str r2, [r1, r2, LSL#2]
[...]
```

Registers

R0	R1	R2
0x00010098	0x0001009c	0x00000003

1. 0x00000003 (hex) = 0000 0011 (binary)
2. 0000 0011 LSL#2 = 0000 1100
3. 0000 1100 = 0xC (hex)
4. 0x0001009c + 0xC = 0x00100a8

   base offset

Memory

0x000100a8	0x00000003	
	...	
0x0001009c	0x00000004	<var2>
0x00010098	0x00000003	<var1>
	...	

SUMMARY

Three offset modes in LDR/STR:

- offset mode uses an immediate as offset

```
ldr   r3, [r1, #4]
```

- offset mode uses a register as offset

```
ldr   r3, [r1, r2]
```

- offset mode uses a scaled register as offset

```
ldr   r3, [r1, r2, LSL#2]
```

Functions and Function Invocation

The Basics of Function Calls

At its core, calling a function in assembly involves several key steps:

1. **Preparing Arguments**: Load function arguments into specified registers or onto the stack.
2. **Branching**: Use a branch instruction to jump to the function's entry point.
3. **Function Execution**: The called function executes, possibly modifying registers or the stack.
4. **Returning**: The function returns control to the caller, potentially passing back a return value.
5. **Clean-Up**: Any necessary clean-up tasks are performed, such as restoring registers or adjusting the stack.

Function arguments can be passed in registers or on the stack, depending on the calling convention. Commonly used registers for argument passing in ARM are R0 to R3. If more arguments are required, additional arguments are typically passed on the stack.

Here’s an example of passing arguments in registers:

```
MOV     R0, #5        ; Argument 1: R0 = 5
    MOV     R1, #10       ; Argument 2: R1 = 10
    BL      my_function   ; Call the function
```

Branching

The branch instruction BL (Branch with Link) is used to jump to the function's entry point. It stores the return address in the link register (LR or R14) so that the function can return control to the caller.

```
BL      my_function    ; Call the function, LR stores the return address
```

Function Execution

Within the called function, arguments are accessed based on the calling convention. The function uses these arguments as needed, potentially modifying registers, and executes its code.

```
my_function:
    ADD     R2, R0, R1   ; Add arguments R0 and R1, result in R2
    ...
    BX      LR           ; Return using the value in LR
```

Returning

The BX (Branch and Exchange) instruction is used to return control to the caller. It branches to the address stored in the link register (LR), which holds the return address.

```
my_function:
    ...
    BX      LR          ; Return using the value in LR
```

Clean-Up

After returning from the function, any necessary clean-up tasks must be performed. This may include restoring registers or adjusting the stack, depending on the calling convention and the function’s requirements.

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

Entering two arguments in R0 and R1 respectively. Registers R0-R3 are for arguments. If more than 4 arguments then the arguments are pushed to stack

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

Push to save state. Otherwise when add function updates r0 and r1 and the control gets back to the instruction /line of code present after func call; we missed out to original values of r0 and r1. the values have been overwritten

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

BL: branch with link is used to call a routine

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

This adds up r0 and r1 and puts the result in r0. The result is always stored in r0.

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

BX branch and exchange causes a branch to an address specified by register. Link register LR stores address of the instruction mentioned after function call…i.e. address of instruction “mov r2,r0”

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

This is done to safe state. i.e. save value r0 returned by function. Because in upcoming lines we are going to reload original values of r0 and r1

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

reload original values of r0 and r1… retrieved the state

```
int main()
{
 int res = 0;
 int a = 1;
 int b = 2;
 res = max(a, b);
 return res;
}
```

```
main:
        push   {r11, lr}    /* Start of the prologue. Saving Frame Pointer and LR onto the stack
*/
        add    r11, sp, #0  /* Setting up the bottom of the stack frame */
        sub    sp, sp, #16  /* End of the prologue. Allocating some buffer on the stack */
        mov    r0, #1       /* setting up local variables (a=1). This also serves as setting up the
first parameter for the max function */
        mov    r1, #2       /* setting up local variables (b=2). This also serves as setting up the
second parameter for the max function */
        bl     max          /* Calling/branching to function max */
        sub    sp, r11, #0  /* Start of the epilogue. Readjusting the Stack Pointer */
        pop    {r11, pc}    /* End of the epilogue. Restoring Frame pointer from the stack,
jumping to previously saved LR via direct load into PC */
```

```
int max(int a,int b)
{
 if(a<b)
 {
 return b;
 }
 else
 {
 return a;
 }
}
```

```
max:
push   {r11}        /* Start of the prologue. Saving Frame Pointer onto the stack */
add    r11, sp, #0  /* Setting up the bottom of the stack frame */
sub    sp, sp, #12  /* End of the prologue. Allocating some buffer on the stack */
cmp    r0, r1       /* Implementation of if(a<b) */
movlt  r0, r1       /* if r0 was lower than r1, store r1 into r0 */
add    sp, r11, #0  /* Start of the epilogue. Readjusting the Stack Pointer */
pop    {r11}        /* restoring frame pointer */
bx     lr           /* End of the epilogue. Jumping back to main via LR register */
```

```
int main() {
   int max = 0;
   int a = 2;
   int b = 3;
   if(a < b) {
    max = b;
   }
   else {
    max = a;
   }
   return max;
}
```

```
.global main

main:
        mov     r1, #2     /* setting up initial variable a */
        mov     r2, #3     /* setting up initial variable b */
        cmp     r1, r2     /* comparing variables to determine
which is bigger */
        blt     r1_lower   /* jump to r1_lower in case r2 is >*/
        mov     r0, r1     /* if branching/jumping did not occur, r1
is bigger (or the same) so store r1 into r0 */
        b       end        /* proceed to the end */
r1_lower:
        mov r0, r2         /* We ended up here because r1 was
smaller than r2, so move r2 into r0 */
        b end              /* proceed to the end */
end:
        bx lr              /* THE END */
```

```
int main() {
   int a = 0;
   while(a < 4) {
   a= a+1;
   }
   return a;
}
```

```
.global main

main:
        mov     r0, #0     /* setting up initial variable a */
loop:
        cmp     r0, #4     /* checking if a==4 */
        beq     end        /* proceeding to the end if a==4 */
        add     r0, r0, #1 /* increasing a by 1 if the jump to the
end did not occur */
        b loop             /* repeating the loop */
end:
        bx lr              /* THE END */
```

Software Re-Engineering

Lecture 15

Reverse engineering III – Starting with IDA, IDA Displays I

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

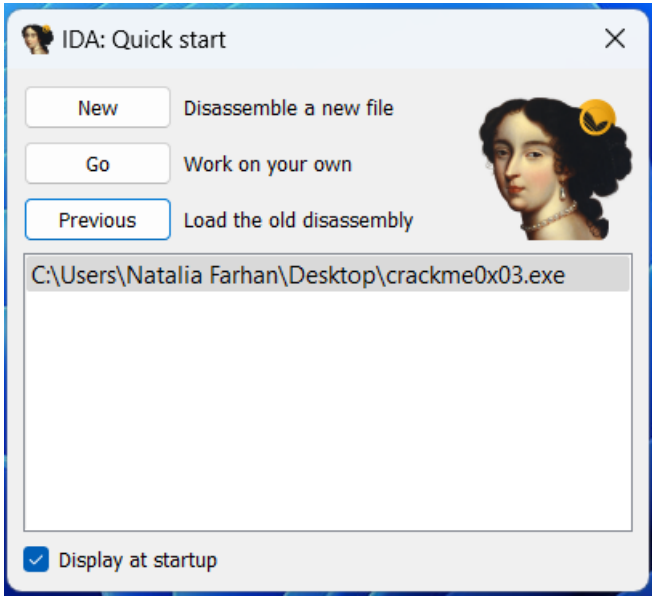
IDA Disassembler

Download IDA Free
https://hex-rays.com/ida-free/#download

Crack
https://github.com/Maijin/radare2-workshop-2015/blob/master/IOLI-crackme/bin-win32/crackme0x03.exe

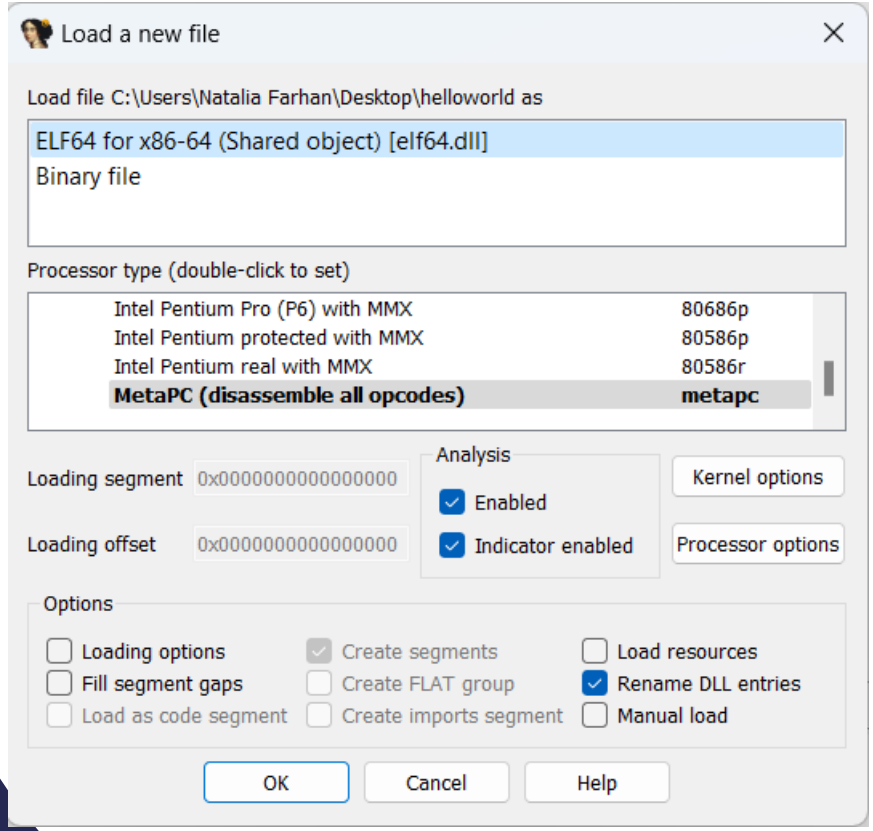
Readings

Chapter 4: GETTING STARTED WITH IDA, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle



Get the binary and add it through **new** file

https://github.com/BIackMage/ReverseEngineeringPractice/blob/main/binaries/helloworld



ELF extensible and linkable format is a standrad binary format

Database files

Upon opening a new file to analyze with IDA Pro, it analyzes the whole executable file and creates an.idb database archive. The .idb archive contains four files:

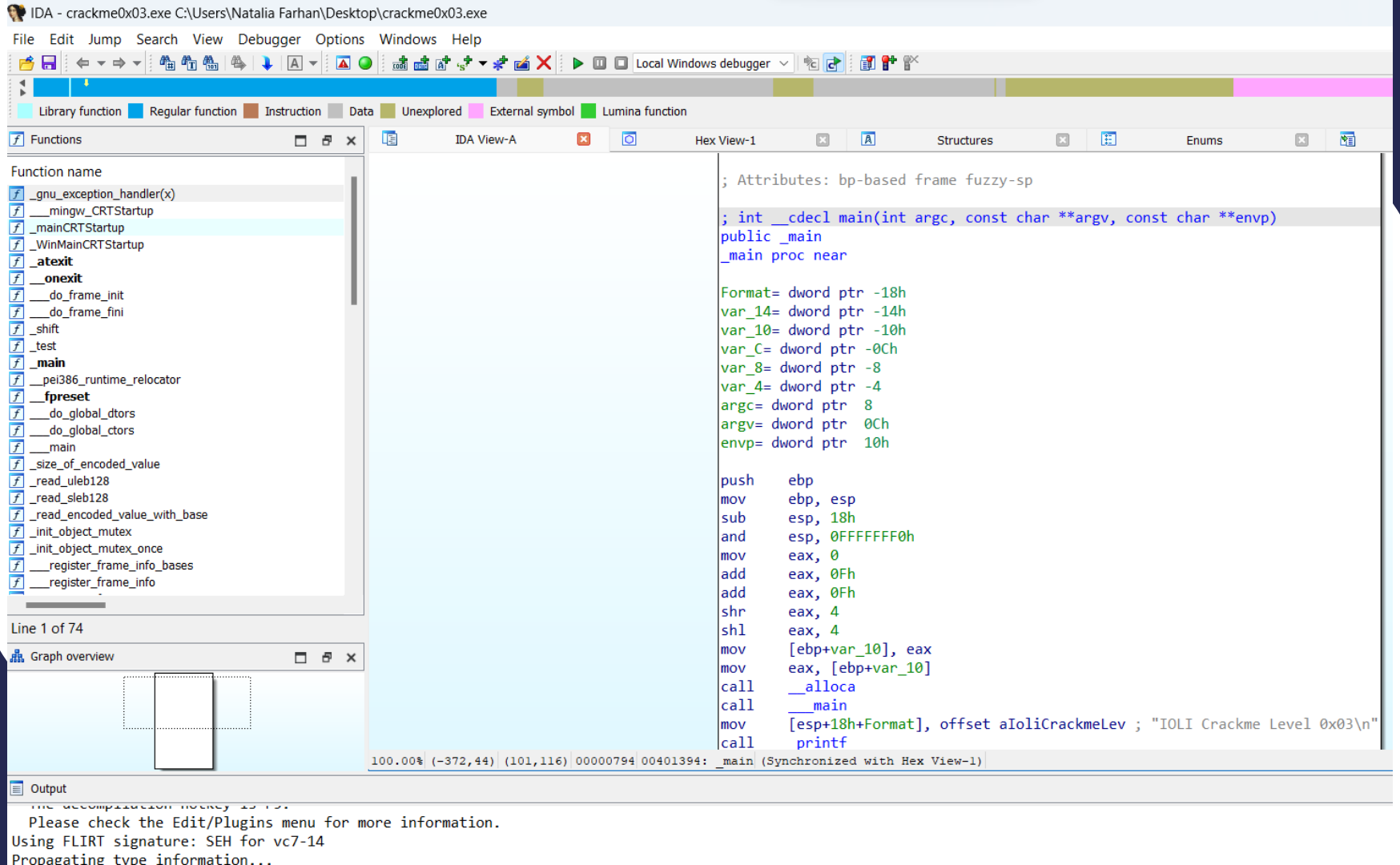
name.id0 - contains contents of B-tree style database,
name.id1 - contains flags that describe each program byte,
name.nam - contains index information related to named program locations,
name.til - contains information about local type definitions

- Once the .idb database has been created for a specific executable, IDA won't need to analyze the program again when we load it later.
- Moreover, IDA doesn't even require the executable anymore; we can now work with just the .idb file.
- This is a useful feature that can be used to pass around .idb files to other researchers without the malicious executable.
- Therefore, IDA can analyze the executable without the actual executable, and with only the database archive file.

Anytime we're trying to close the currently open.idb database (the currently analyzed executable), IDA asks us if we would like to save changes to the disk. We can choose from the following options:

- Don't pack database: flush changes to .id0, id1, nam and til databases and don't create .idb file.
- Pack database (Store): archives the .id0, id1, nam and til into the .idb archive. Note that the .idb of the previous session is overwritten.
- Pack database (Deflate): the same as the previous option, except the database files are compressed in the .idb archive.
- Collect garbage: deletes any unused memory pages from the database. This can be useful if we want to create a smaller database .idb file.
- Don't save the database: we can pick this option if we don't want to save the changes that we have made.

- During that time, IDA will also load and parse the actual code instructions of the executable file into the assembly instructions of the selected processor module.
- Those assembly instructions are then also showed to the user for analysis.
- But IDA doesn't stop there; it can also scan the generated assembly instructions to figure out additional information about the executable, like the compiler which was used to compile the executable, the function's arguments, the function's local variables, etc.

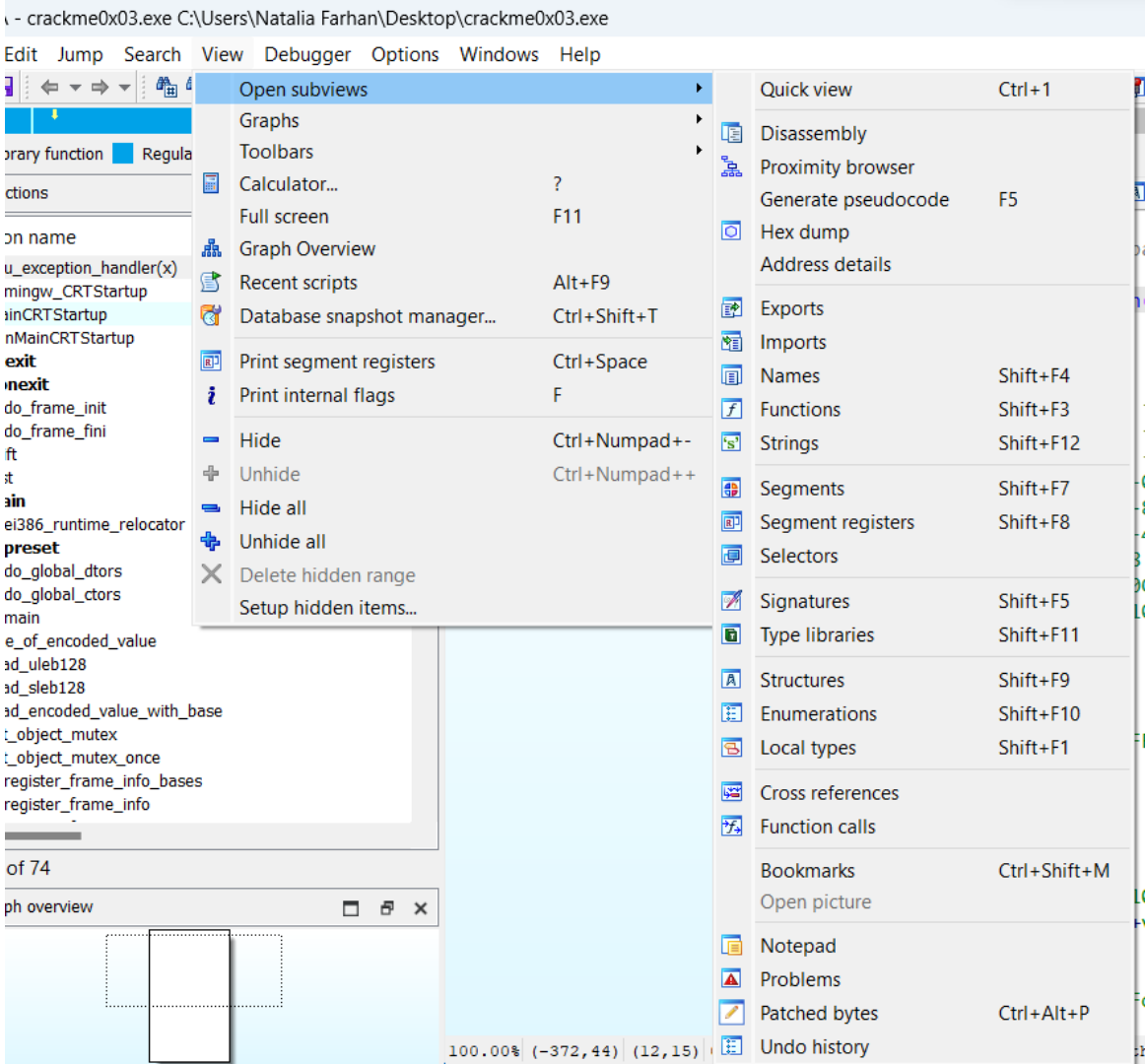

IDA - crackme0x03.exe C:\Users\Natalia Farhan\Desktop\crackme0x03.exe
File Edit Jump Search View Debugger Options Windows Help
Local Windows debugger
Library function Regular function Instruction Data Unexplored External symbol Lumina function
Functions
Function name
_gnu_exception_handler(x)
___mingw_CRTStartup
_mainCRTStartup
_WinMainCRTStartup
_atexit
__onexit
___do_frame_init
___do_frame_fini
_shift
_test
_main
__pei386_runtime_relocator
__fpreset
___do_global_dtors
___do_global_ctors
___main
_size_of_encoded_value
_read_uleb128
_read_sleb128
_read_encoded_value_with_base
_init_object_mutex
_init_object_mutex_once
___register_frame_info_bases
___register_frame_info
Line 1 of 74
Graph overview
IDA View-A
Hex View-1
Structures
Enums
; Attributes: bp-based frame fuzzy-sp
; int __cdecl main(int argc, const char **argv, const char **envp)
public _main
_main proc near
Format= dword ptr -18h
var_14= dword ptr -14h
var_10= dword ptr -10h
var_C= dword ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4
argc= dword ptr 8
argv= dword ptr 0Ch
envp= dword ptr 10h
push ebp
mov ebp, esp
sub esp, 18h
and esp, 0FFFFFFF0h
mov eax, 0
add eax, 0Fh
add eax, 0Fh
shr eax, 4
shl eax, 4
mov [ebp+var_10], eax
mov eax, [ebp+var_10]
call __alloca
call ___main
mov [esp+18h+Format], offset aIoliCrackmeLev ; "IOLI Crackme Level 0x03\n"
call _printf
100.00% (-372,44) (101,116) 00000794 00401394: _main (Synchronized with Hex View-1)
Output
Please check the Edit/Plugins menu for more information.
Using FLIRT signature: SEH for vc7-14
Propagating type information...

Readings

Chapter 5: IDA DATA DISPLAYS, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle

The Principal IDA Displays

- To open a specific data view, we can go to View - Open Subviews and choose the appropriate view we would like to show.
- We can also switch back to the default view by clicking on Windows - Reset desktop.



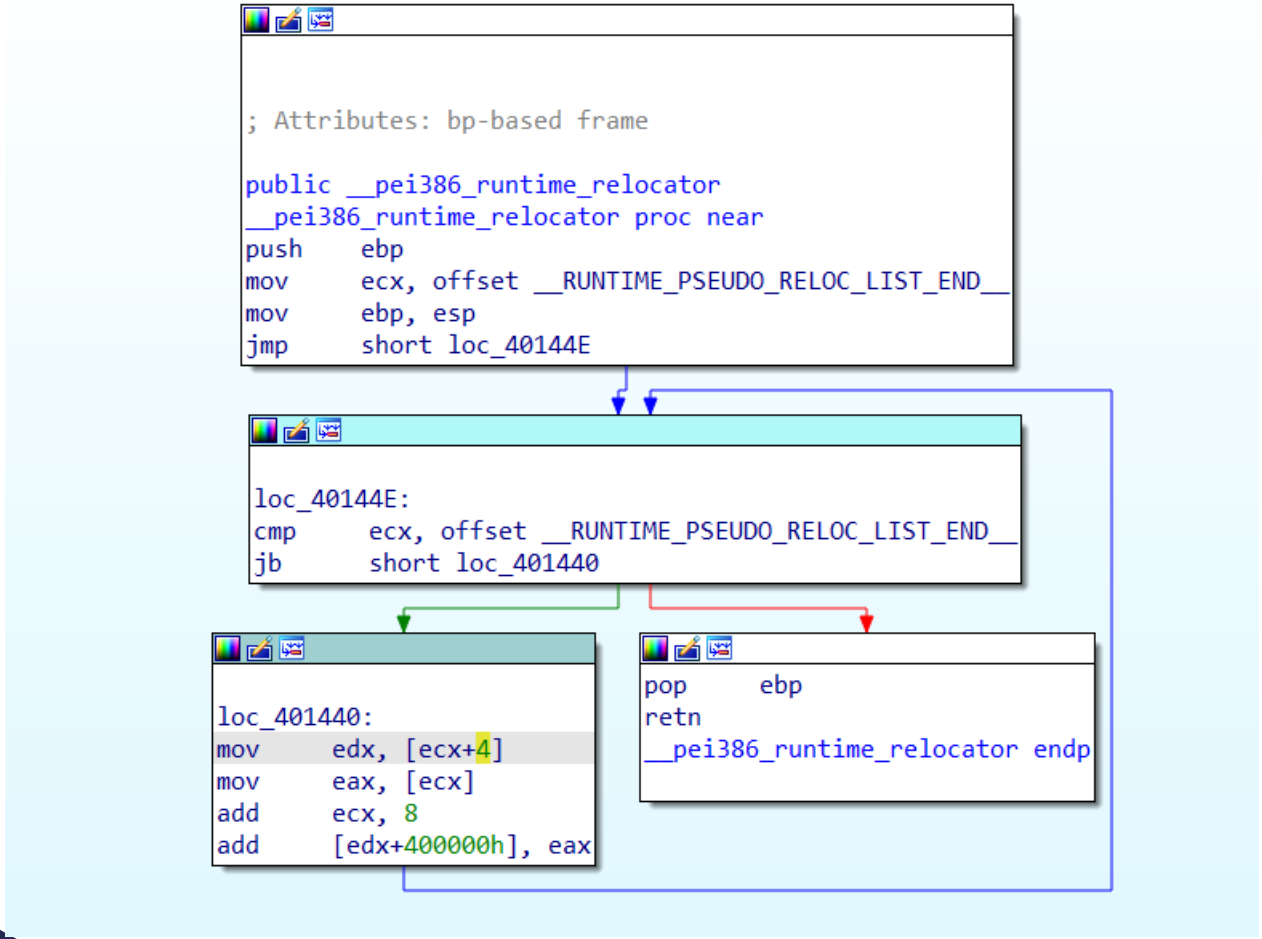
The graph view can be used if we want to quickly figure out the execution flow of the current function and the listing view can be used when we want to see the actual assembly instructions.

On the graph overview, the program is broken down into logical blocks, where each block is presenting a jump target (as defined in the assembly code). From the graph overview we can also see the logic the program uses while executing.

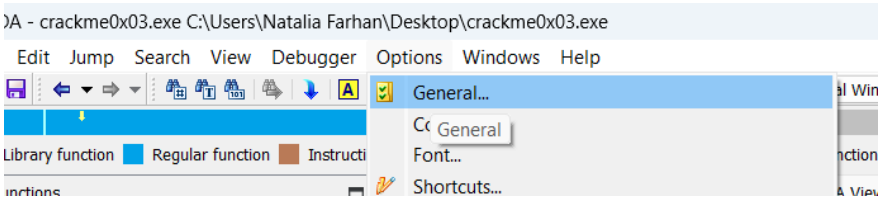
- A basic block is a maximal sequence of instructions that executes, without branching, from beginning to end.
- Each basic block therefore has a single entry point (the first instruction in the block) and a single exit point (the last instruction in the block).
- The first instruction in a basic block is often the target of a branching instruction, while the last instruction in a basic block is often a branch instruction

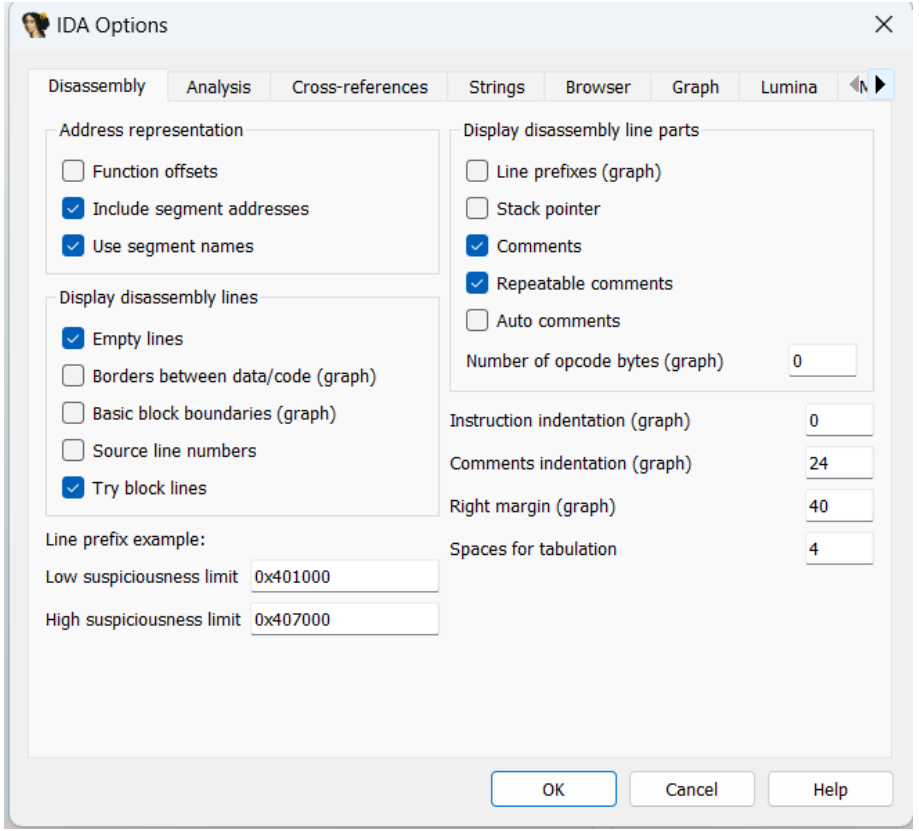
Graph views are somewhat reminiscent of program flowcharts in that a function is broken up into **basic blocks** so you can visualize the function's control flow from one block to another.

- Onscreen, you’ll notice IDA uses different colored arrows to distinguish various types of flows2 between the blocks of a function.
- `Basic blocks that terminate with a conditional jump generate two possible flows depending on the condition being tested: the Yes edge arrow (yes, the branch is taken) is green by default, and the No edge arrow (no, the branch is not taken) is red by default.
- Basic blocks that terminate with only one potential successor block utilize a Normal edge (blue by default) to point to the next block to be executed.


; Attributes: bp-based frame
public __pei386_runtime_relocator
__pei386_runtime_relocator proc near
push ebp
mov ecx, offset __RUNTIME_PSEUDO_RELOC_LIST_END__
mov ebp, esp
jmp short loc_40144E
loc_40144E:
cmp ecx, offset __RUNTIME_PSEUDO_RELOC_LIST_END__
jb short loc_401440
loc_401440:
mov edx, [ecx+4]
mov eax, [ecx]
add ecx, 8
add [edx+400000h], eax
pop ebp
retn
__pei386_runtime_relocator endp

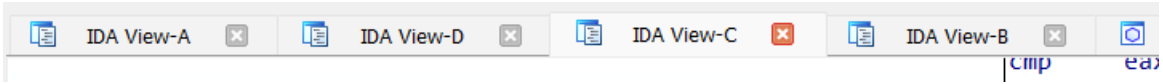
- When using graph view, it may seem as if less information is available to you about each line of the disassembly.
- The reason for this is that IDA chooses to hide many of the more traditional pieces of information about each disassembled line (such as virtual address information) in order to minimize the amount of space required to display each basic block.
- You can choose to display additional information with each disassembly line by choosing among the available disassembly line parts accessible via the Disassembly tab from Options->General. For example, to add virtual addresses to each disassembly line, we enable line prefixes


)A - crackme0x03.exe C:\Users\Natalia Farhan\Desktop\crackme0x03.exe
Edit Jump Search View Debugger Options Windows Help
General...
General
Font...
Shortcuts...
Library function
Regular function
Instructi


IDA Options
Disassembly
Analysis
Cross-references
Strings
Browser
Graph
Lumina
Address representation
Function offsets
Include segment addresses
Use segment names
Display disassembly lines
Empty lines
Borders between data/code (graph)
Basic block boundaries (graph)
Source line numbers
Try block lines
Line prefix example:
Low suspiciousness limit 0x401000
High suspiciousness limit 0x407000
Display disassembly line parts
Line prefixes (graph)
Stack pointer
Comments
Repeatable comments
Auto comments
Number of opcode bytes (graph) 0
Instruction indentation (graph) 0
Comments indentation (graph) 24
Right margin (graph) 40
Spaces for tabulation 4
OK
Cancel
Help

Creating additional disassembly windows

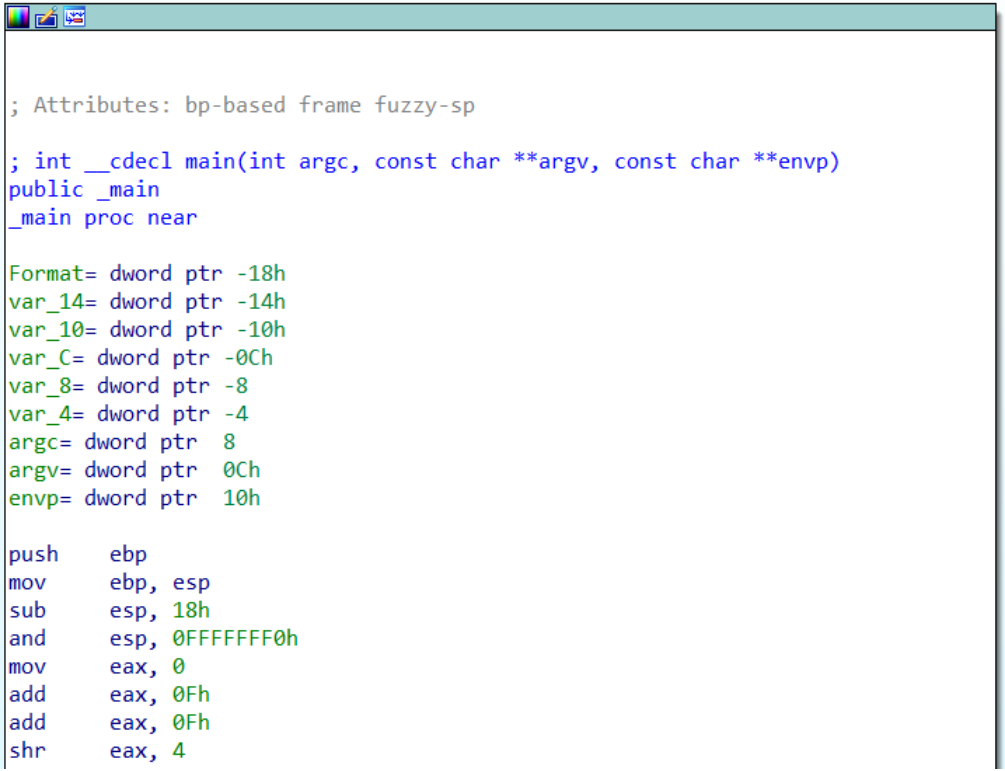
- If you ever find yourself wanting to view graphs of two functions simultaneously, all you need to do is open another disassembly window using Views->Open Subviews->Disassembly.
- The first disassembly window opened is titled IDA View-A. Subsequent disassembly windows are titled IDA View-B, IDA View-C, and so on.
- Each disassembly is independent of the other, and it is perfectly acceptable to view a graph in one window while viewing a text listing in another or to view three different graphs in three different windows



IDA View-A | IDA View-D | IDA View-C | IDA View-B | Hex View-1 | Structures

IDA Text View

- The text-oriented disassembly window is the traditional display used for viewing and manipulating IDA-generated disassemblies.
- The text display presents the entire disassembly listing of a program (as opposed to a single function at a time in graph mode) and provides the only means for viewing the data regions of a binary.



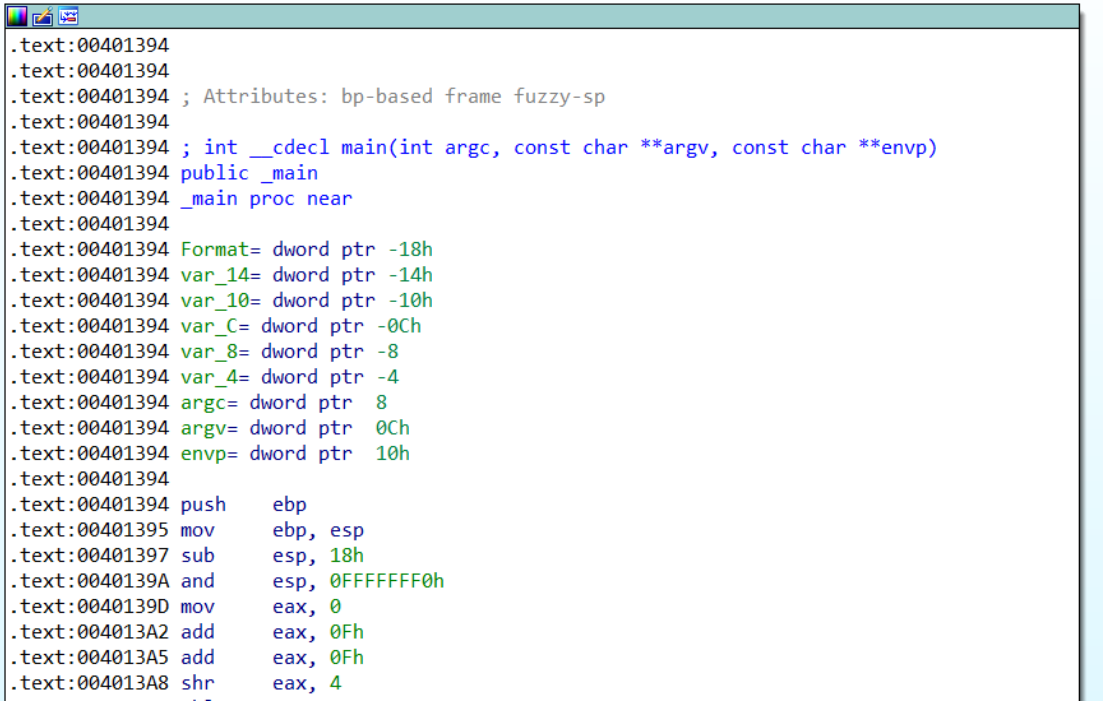
```
; Attributes: bp-based frame fuzzy-sp

; int __cdecl main(int argc, const char **argv, const char **envp)
public _main
_main proc near

Format= dword ptr -18h
var_14= dword ptr -14h
var_10= dword ptr -10h
var_C= dword ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4
argc= dword ptr  8
argv= dword ptr  0Ch
envp= dword ptr  10h

push    ebp
mov     ebp, esp
sub     esp, 18h
and     esp, 0FFFFFFF0h
mov     eax, 0
add     eax, 0Fh
add     eax, 0Fh
shr     eax, 4
```

- The disassembly is presented in linear fashion, with virtual addresses displayed by default.
- Virtual addresses are typically displayed in a [SECTION NAME]:[VIRTUAL ADDRESS] format such as .text:004011C1.



```
.text:00401394
.text:00401394
.text:00401394 ; Attributes: bp-based frame fuzzy-sp
.text:00401394
.text:00401394 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:00401394 public _main
.text:00401394 _main proc near
.text:00401394
.text:00401394 Format= dword ptr -18h
.text:00401394 var_14= dword ptr -14h
.text:00401394 var_10= dword ptr -10h
.text:00401394 var_C= dword ptr -0Ch
.text:00401394 var_8= dword ptr -8
.text:00401394 var_4= dword ptr -4
.text:00401394 argc= dword ptr  8
.text:00401394 argv= dword ptr  0Ch
.text:00401394 envp= dword ptr  10h
.text:00401394
.text:00401394 push    ebp
.text:00401395 mov     ebp, esp
.text:00401397 sub     esp, 18h
.text:0040139A and     esp, 0FFFFFFF0h
.text:0040139D mov     eax, 0
.text:004013A2 add     eax, 0Fh
.text:004013A5 add     eax, 0Fh
.text:004013A8 shr     eax, 4
```

The declarations (also present in graph view) represent IDA's best estimate concerning the layout of the function's stack frame

```
.text:00401394 Format= dword ptr -18h
.text:00401394 var_14= dword ptr -14h
.text:00401394 var_10= dword ptr -10h
.text:00401394 var_C= dword ptr -0Ch
.text:00401394 var_8= dword ptr -8
.text:00401394 var_4= dword ptr -4
.text:00401394 argc= dword ptr  8
.text:00401394 argv= dword ptr  0Ch
.text:00401394 envp= dword ptr  10h
```

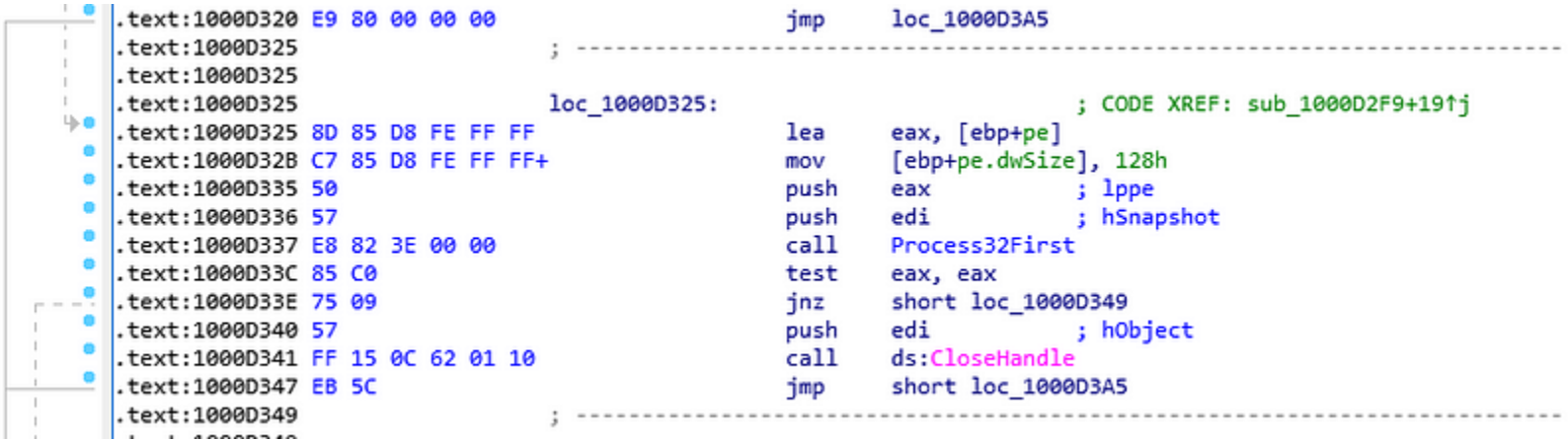
Cross-references, or more commonly referred to as xrefs, are used to identify references (usage/call or declaration) of a particular function, string, variable, etc. Example — if you've identified a particular function being called inside another subroutine, you might be interested in knowing other calls to the same function.

Cross-references can be categorized as follows:

- Code cross-references
- Data cross-references

Code Cross-references

Code cross-references are used to identify relationships between function calls, declarations, or jumps.



```
.text:1000D320 E9 80 00 00 00                  jmp     loc_1000D3A5
.text:1000D325                 ; ---------------------------------------------------------------------------
.text:1000D325
.text:1000D325                 loc_1000D325:                           ; CODE XREF: sub_1000D2F9+19↑j
.text:1000D325 8D 85 D8 FE FF FF               lea     eax, [ebp+pe]
.text:1000D32B C7 85 D8 FE FF FF+              mov     [ebp+pe.dwSize], 128h
.text:1000D335 50                              push    eax             ; lppe
.text:1000D336 57                              push    edi             ; hSnapshot
.text:1000D337 E8 82 3E 00 00                  call    Process32First
.text:1000D33C 85 C0                           test    eax, eax
.text:1000D33E 75 09                           jnz     short loc_1000D349
.text:1000D340 57                              push    edi             ; hObject
.text:1000D341 FF 15 0C 62 01 10               call    ds:CloseHandle
.text:1000D347 EB 5C                           jmp     short loc_1000D3A5
.text:1000D349                 ; ---------------------------------------------------------------------------
```

Here, we can see that **loc_1000D325** has a code cross-reference to **sub_1000D2F9+19**. It suggests that this location is accessed by the **function (sub_*)** at the address **1000D2F9** and **an offset of 0x19** into the function. Next, the arrow shows us that the code is at a lower address than the reference and the ‘j’ suggests it’s a jump.

Data Cross-references

Much like code cross-references, data cross-references can help us keep track of data being accessed or written throughout the binary. For example, you might find an IP address being referenced in a few functions; using xrefs, you can identify all those functions easily.

```
xdoors_d:10094E64 43 6F 70 79 20 27+aCopySToSFailed   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Failed',0
xdoors_d:10094E64 25 73 27 20 54 6F+                                       ; DATA XREF: sub_1000D3D0:loc_1000D501↑o
xdoors_d:10094E64 20 27 25 73 27 20+                                       ; sub_1000DF22:loc_1000E59D↑o ...
xdoors_d:10094E7D 00 00 00                         align 10h
xdoors_d:10094E80                  ; char aCopySToSSucces[]
xdoors_d:10094E80 43 6F 70 79 20 27+aCopySToSSucces   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Successfully',0
xdoors_d:10094E80 25 73 27 20 54 6F+                                       ; DATA XREF: sub_1000D3D0+12A↑o
xdoors_d:10094E80 20 27 25 73 27 20+                                       ; sub_1000DF22+52F↑o ...
xdoors_d:10094E9F 00                               align 10h
xdoors_d:10094EA0                  ; char aThePidOfProces_1[]
xdoors_d:10094EA0 0D 0A 54 68 65 20+aThePidOfProces_1 db 0Dh,0Ah           ; DATA XREF: sub_1000D5B0+264↑o
xdoors_d:10094EA0 50 49 44 20 4F 66+                  db 'The PID Of Process ',27h,'%s',27h,' Not Found',0Dh,0Ah,0
xdoors_d:10094EC6 00 00                            align 4
xdoors_d:10094EC8                  ; char aInjectSToProce_2[]
```

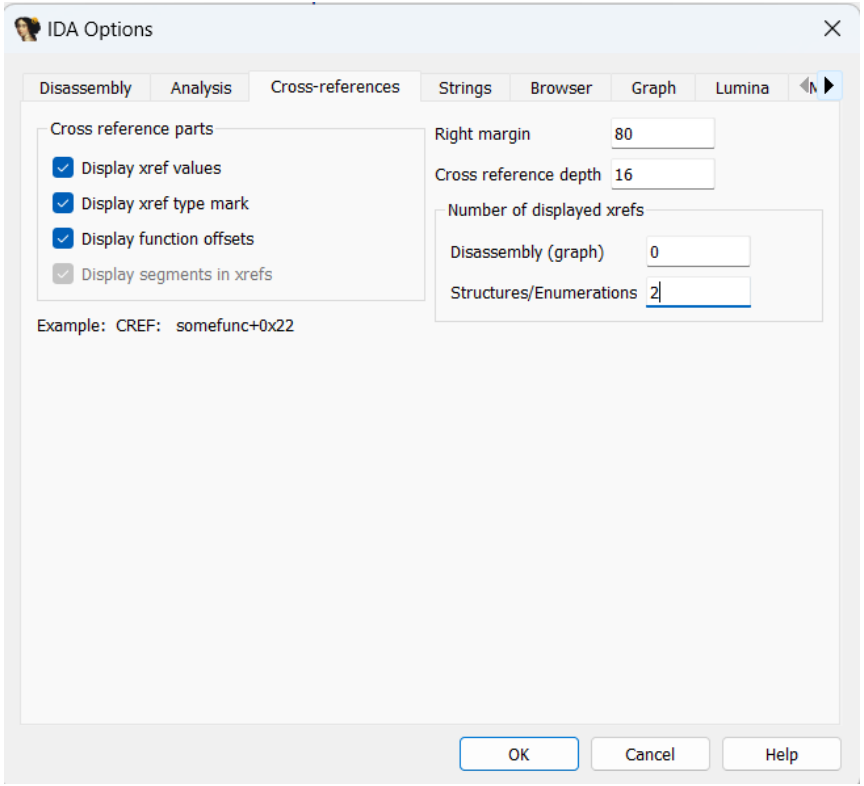
Here, we can see several data strings being referenced in functions. For example, the **aThePidofProces_1** is being referenced in the function, **sub_1000D5B0** and at on offset of 0x264. The direction is upwards again suggesting the function is at a lower address than string's address.

```
xdoors_d:10094E64 43 6F 70 79 20 27+aCopySToSFailed   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Failed',0
xdoors_d:10094E64 25 73 27 20 54 6F+                                       ; DATA XREF: sub_1000D3D0:loc_1000D501↑o
xdoors_d:10094E64 20 27 25 73 27 20+                                       ; sub_1000DF22:loc_1000E59D↑o ...
xdoors_d:10094E7D 00 00 00                         align 10h
xdoors_d:10094E80                  ; char aCopySToSSucces[]
xdoors_d:10094E80 43 6F 70 79 20 27+aCopySToSSucces   db 'Copy ',27h,'%s',27h,' To ',27h,'%s',27h,' Successfully',0
xdoors_d:10094E80 25 73 27 20 54 6F+                                       ; DATA XREF: sub_1000D3D0+12A↑o
xdoors_d:10094E80 20 27 25 73 27 20+                                       ; sub_1000DF22+52F↑o ...
xdoors_d:10094E9F 00                               align 10h
xdoors_d:10094EA0                  ; char aThePidOfProces_1[]
xdoors_d:10094EA0 0D 0A 54 68 65 20+aThePidOfProces_1 db 0Dh,0Ah           ; DATA XREF: sub_1000D5B0+264↑o
xdoors_d:10094EA0 50 49 44 20 4F 66+                  db 'The PID Of Process ',27h,'%s',27h,' Not Found',0Dh,0Ah,0
xdoors_d:10094EC6 00 00                            align 4
xdoors_d:10094EC8                  ; char aInjectSToProce_2[]
```

- For the DATA XREF to aCopySToSSucces you can see two cross-references being listed.

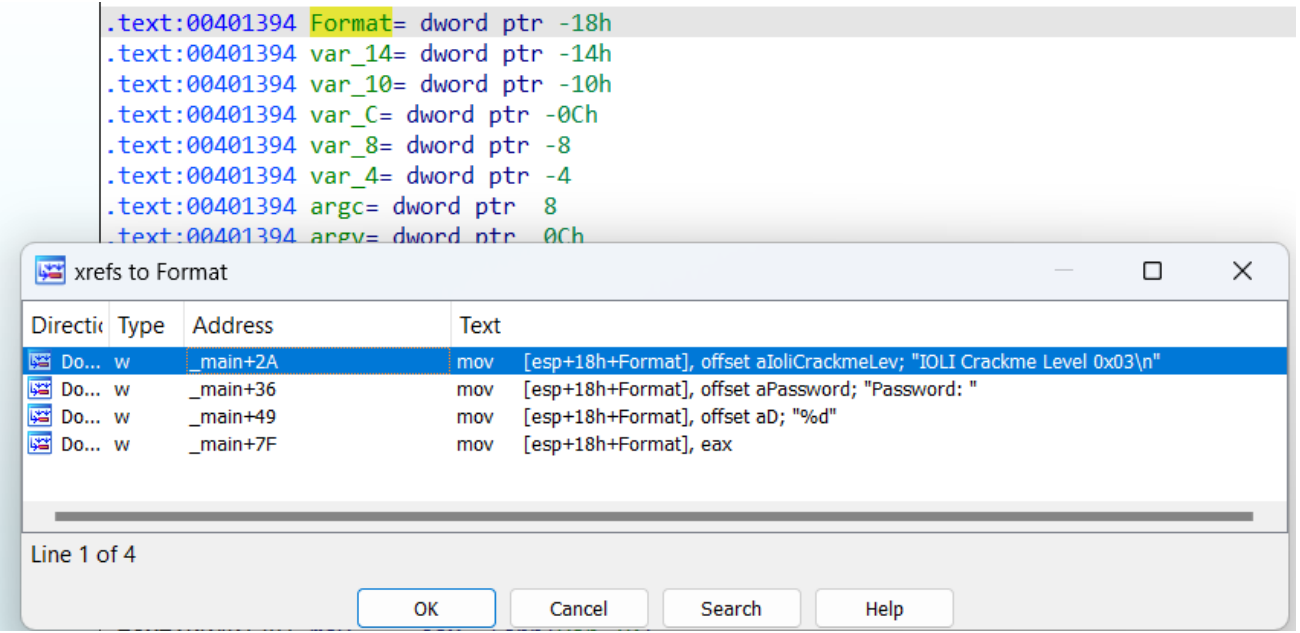
- IDA restricts listing multiple references by default. You can change this setting and show as many references as you want. It is under ‘Options’ — ‘General’ — ‘Cross-references’.

- IDA restricts listing multiple references by default. You can change this setting and show as many references as you want. It is under ‘Options’ — ‘General’ — ‘Cross-references’.



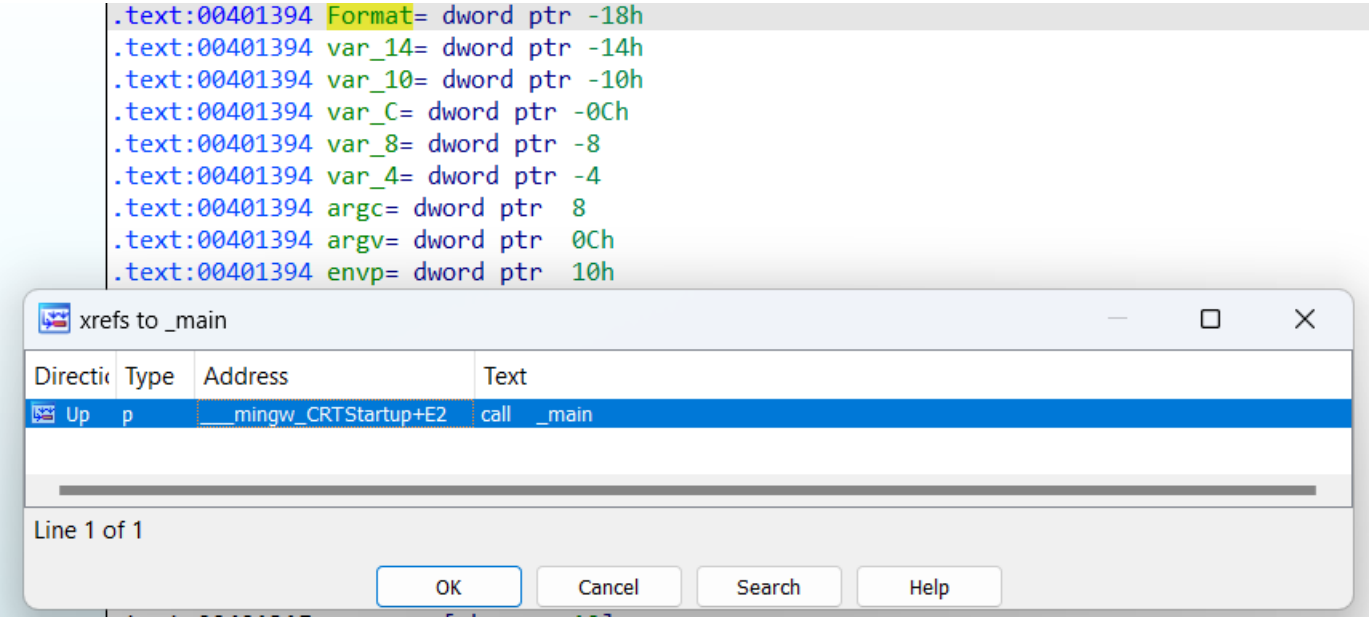
Listing Cross-references

- you can simply do so by selecting the label and pressing ‘X’ to list all cross-references to that particular label (or identifier).



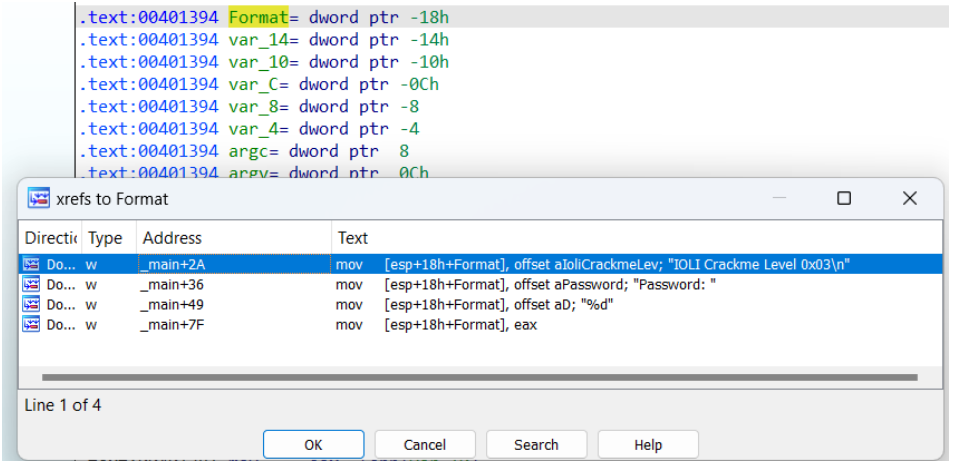
Listing Cross-references

- ‘CTRL+X’ shows us cross-references to the address (cursor’s selection does not matter as it is worked with addresses not identifiers).



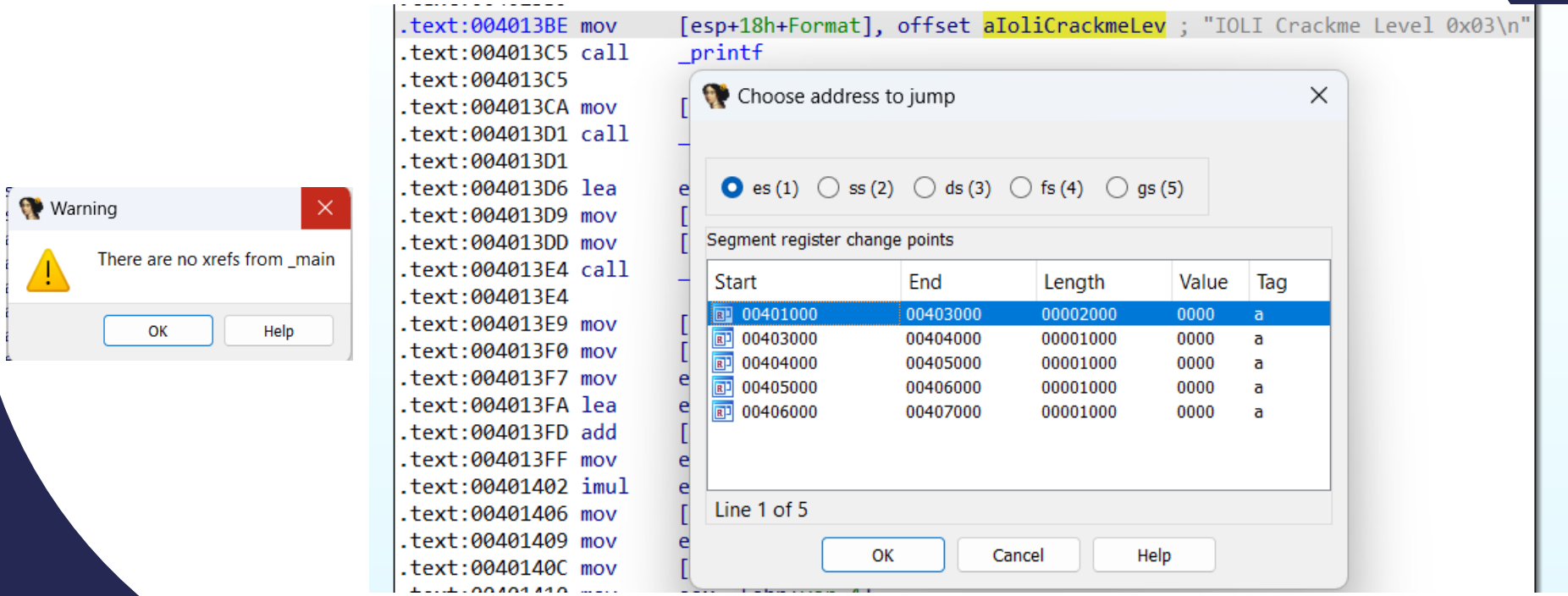
Here, we can find the following information:

- **Direction:** Whether the target reference is at a higher (Down) or lower (Up) address
- **Type:** Type of the cross-reference
- **Address/Text:** Address of the target reference and the actual code/text at that particular address

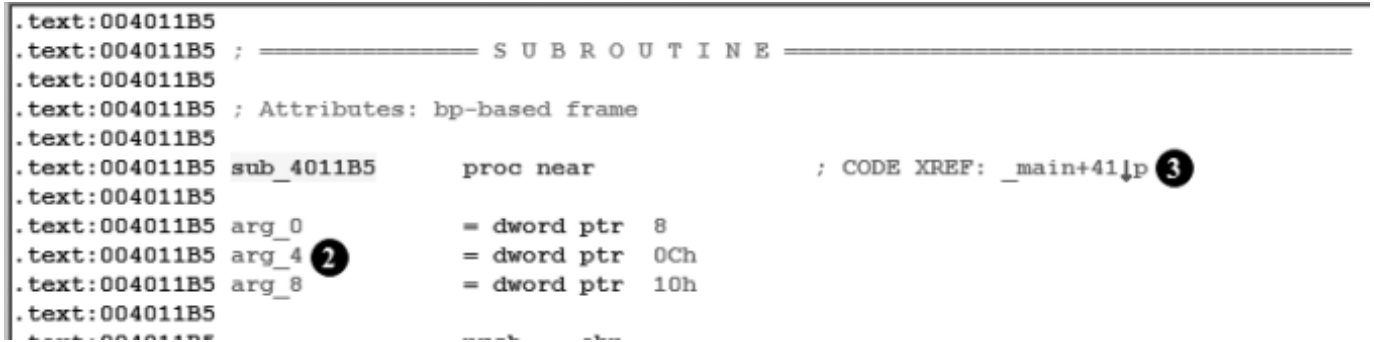


Listing Cross-references

- ‘CTRL+J’ shows us cross-references from the address

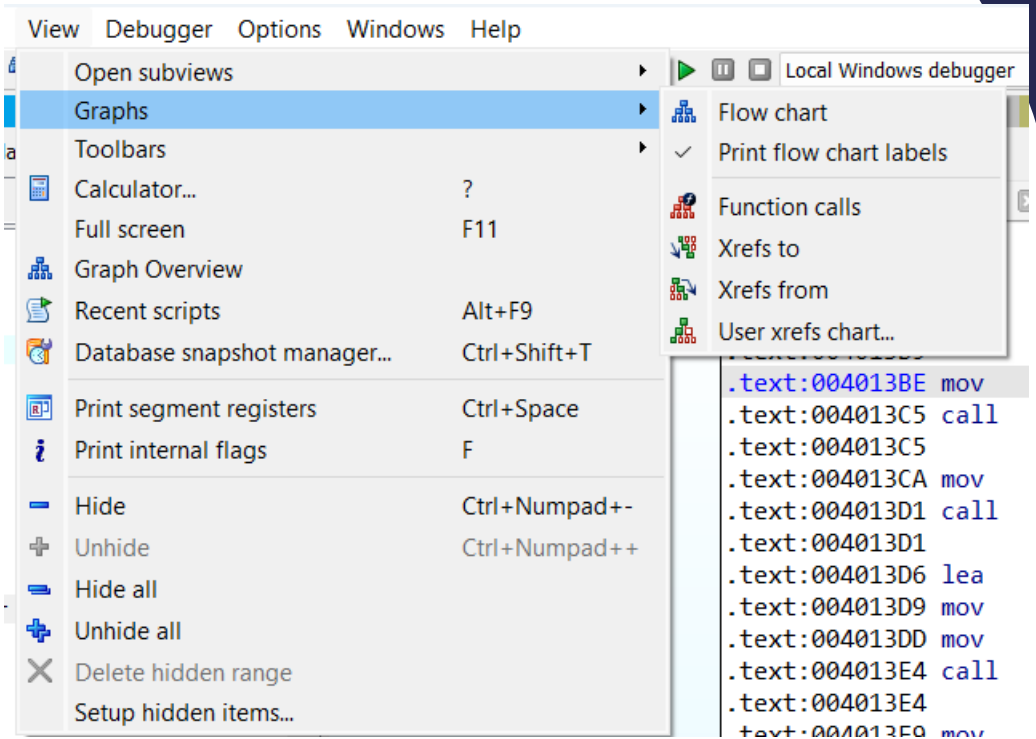


- The comments (a semicolon introduces a comment) are crossreferences.
- In this case we see code cross-references (as opposed to data crossreferences), which indicate that another program instruction transfers control to the location containing the cross-reference comment.



We can also view the information in a graphical view.
Select ‘View’, Go into ‘Graphs’ and select either of the three xref charts available.
You can:

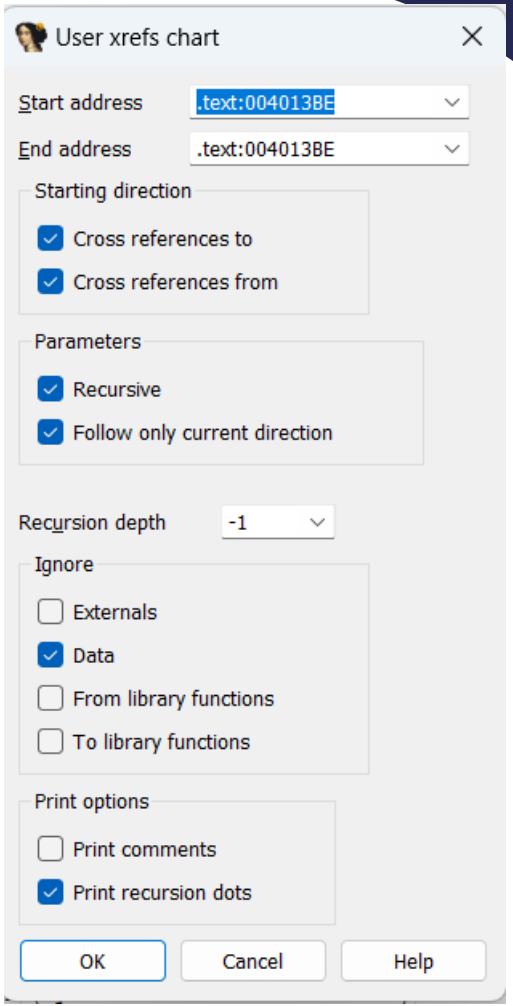
- View xrefs from the identifier (where the reference ‘goes to’)
- View xrefs to the identifier (where the reference ‘comes from’)
- Use a custom xref chart with several customization options (this is particularly helpful in larger binaries and to filter out unhelpful identifiers)



This command displays a user-defined graph of xrefs from/to the current address/range of selected addresses in the program.
The direction of the xrefs to analyze can be chosen.

If the Recursive flag is checked, all found xrefs are themselves analyzed to find new xrefs.

You can choose to search for xrefs to new referenced addresses only in the current direction.

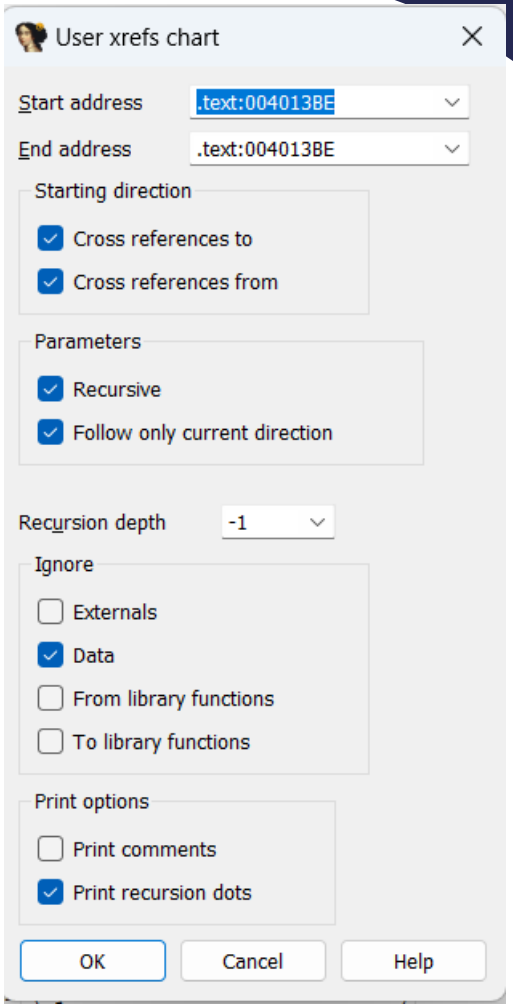


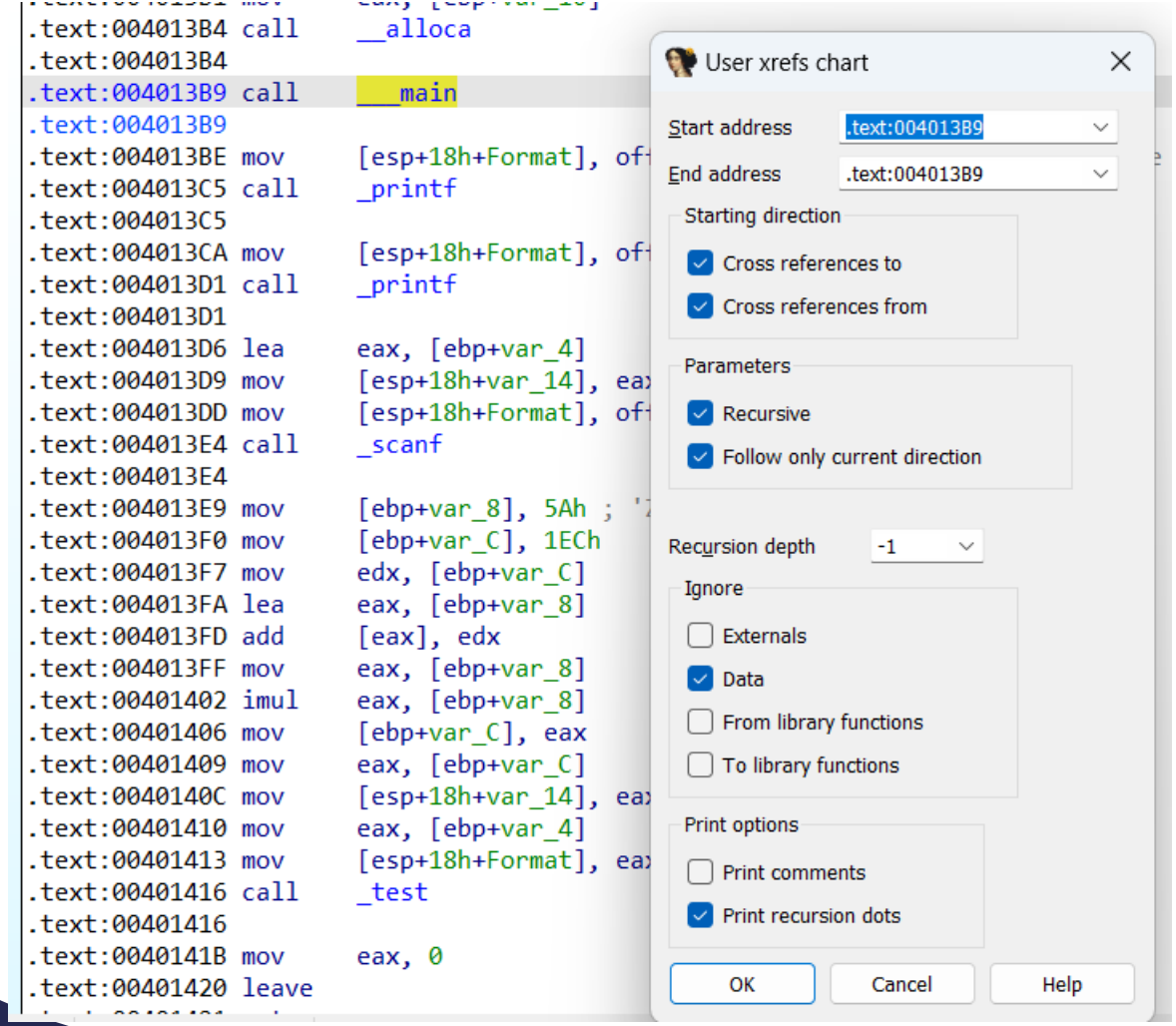
Only External, data xrefs, xrefs from library functions and to library functions can possibly be ignored.

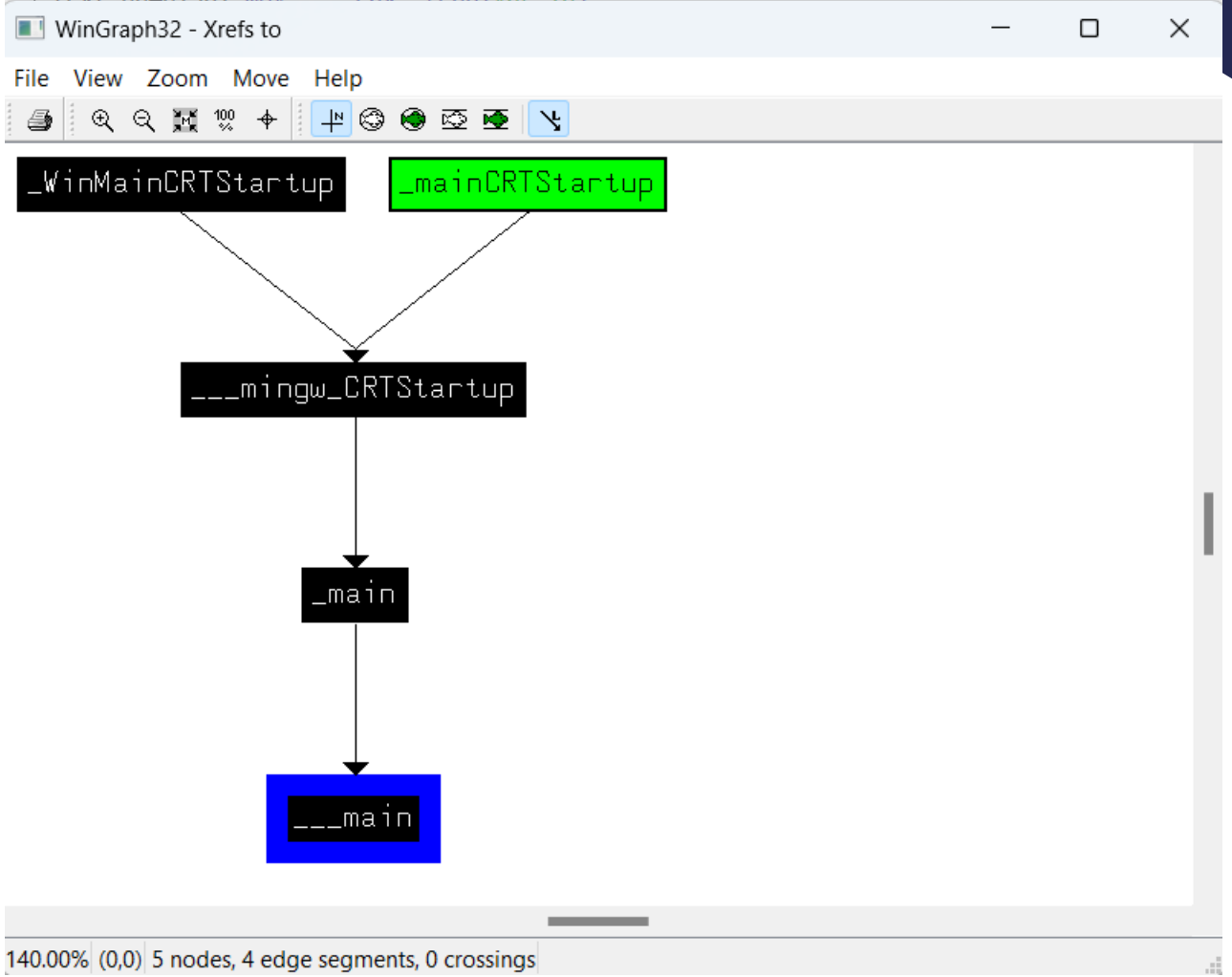
A recursion depth can be specified. If 'Print recursion dots' is checked, and a function has others xrefs outside of the range defined by the 'recursion depth' setting, small nodes containing dots are printed.

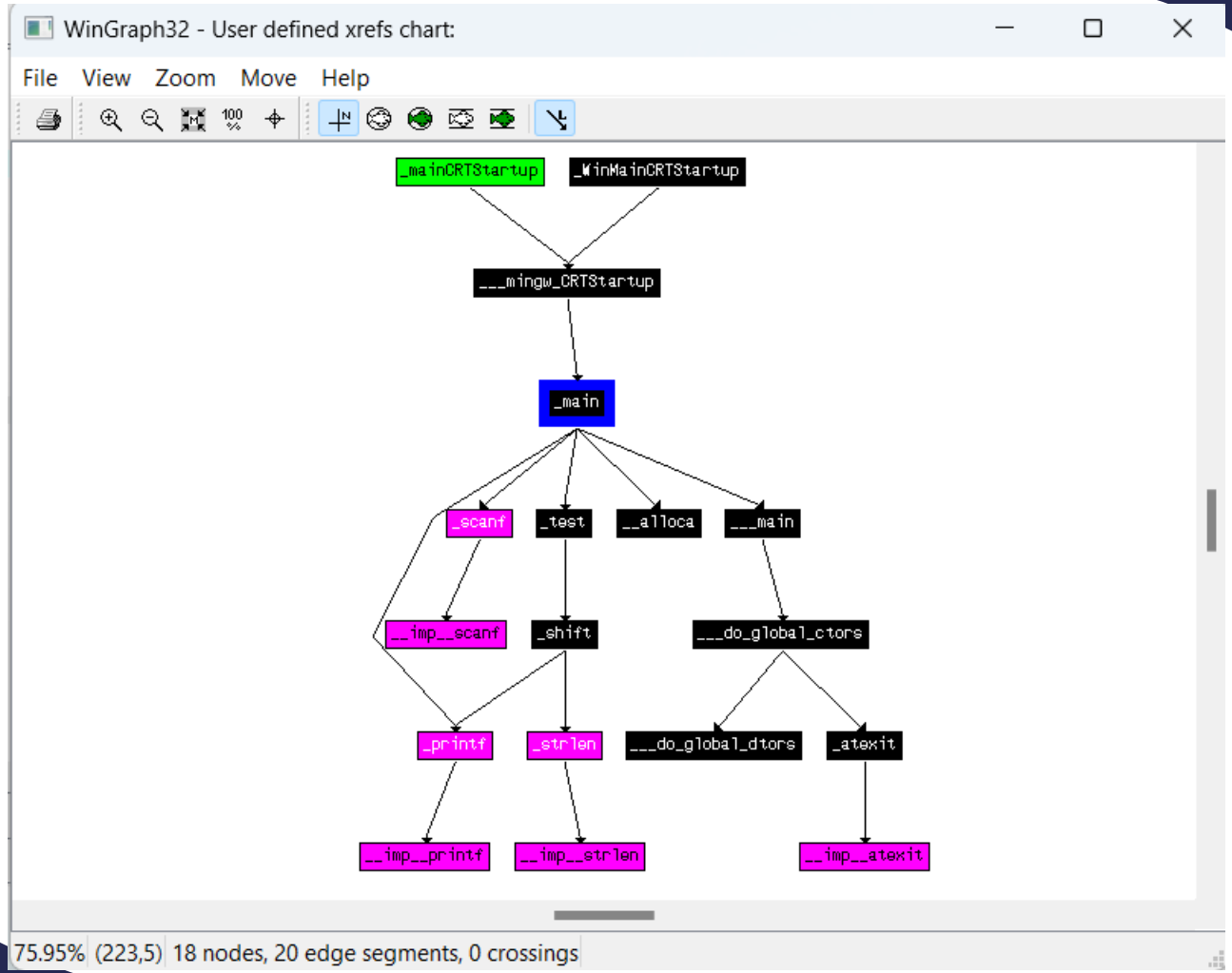
The 'Print comments' flag causes the generated function node to also contain the function comment.

The addresses are represented as nodes. The xrefs are represented as edges between nodes. The used colors are the same as in IDA.

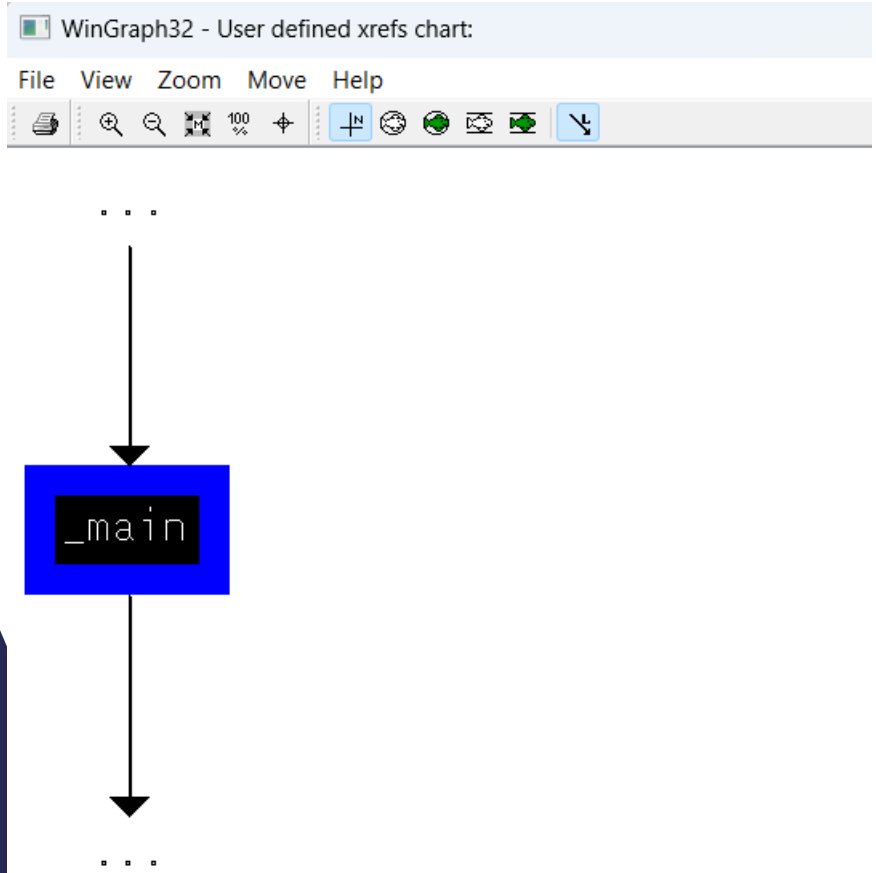



.text:004013B4 call __alloca
.text:004013B4
.text:004013B9 call __main
.text:004013B9
.text:004013BE mov [esp+18h+Format], of
.text:004013C5 call _printf
.text:004013C5
.text:004013CA mov [esp+18h+Format], of
.text:004013D1 call _printf
.text:004013D1
.text:004013D6 lea eax, [ebp+var_4]
.text:004013D9 mov [esp+18h+var_14], ea
.text:004013DD mov [esp+18h+Format], of
.text:004013E4 call _scanf
.text:004013E4
.text:004013E9 mov [ebp+var_8], 5Ah ; '
.text:004013F0 mov [ebp+var_C], 1ECh
.text:004013F7 mov edx, [ebp+var_C]
.text:004013FA lea eax, [ebp+var_8]
.text:004013FD add [eax], edx
.text:004013FF mov eax, [ebp+var_8]
.text:00401402 imul eax, [ebp+var_8]
.text:00401406 mov [ebp+var_C], eax
.text:00401409 mov eax, [ebp+var_C]
.text:0040140C mov [esp+18h+var_14], ea
.text:00401410 mov eax, [ebp+var_4]
.text:00401413 mov [esp+18h+Format], ea
.text:00401416 call _test
.text:00401416
.text:0040141B mov eax, 0
.text:00401420 leave
User xrefs chart
Start address .text:004013B9
End address .text:004013B9
Starting direction
Cross references to
Cross references from
Parameters
Recursive
Follow only current direction
Recursion depth -1
Ignore
Externals
Data
From library functions
To library functions
Print options
Print comments
Print recursion dots
OK
Cancel
Help

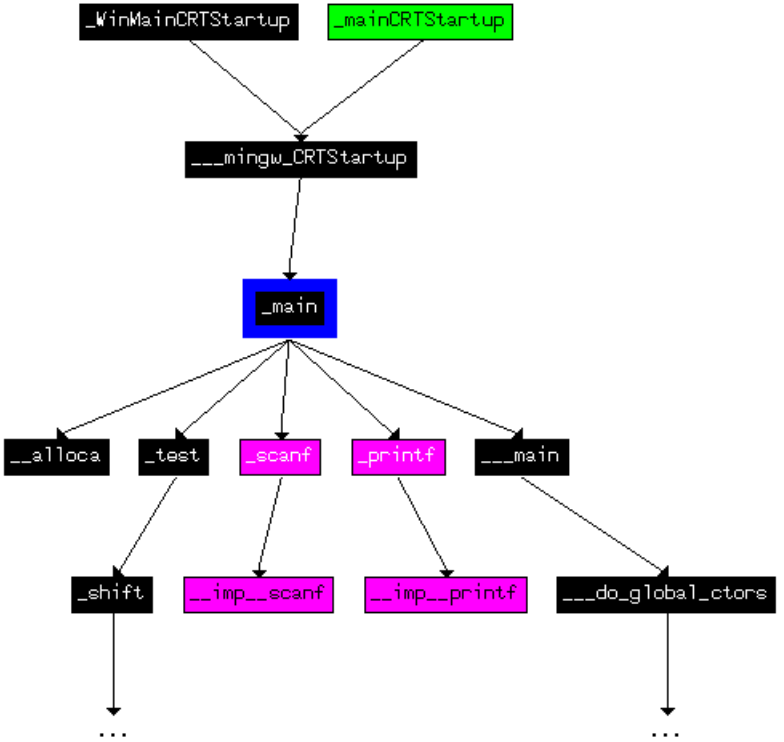

WinGraph32 - Xrefs to
File
View
Zoom
Move
Help
_WinMainCRTStartup
_mainCRTStartup
___mingw_CRTStartup
_main
___main
140.00%
(0,0)
5 nodes, 4 edge segments, 0 crossings


WinGraph32 - User defined xrefs chart:
File View Zoom Move Help
_mainCRTStartup
_WinMainCRTStartup
___mingw_CRTStartup
_main
_scanf
_test
__alloca
___main
__imp__scanf
_shift
___do_global_ctors
_printf
_strlen
___do_global_dtors
_atexit
__imp__printf
__imp__strlen
__imp__atexit
75.95% (223,5) 18 nodes, 20 edge segments, 0 crossings

Recursion depth : 0



Recursion depth : 2



Types of Cross-references

13 types of cross-references. A few of these cross-reference types are listed below:

O: Offset — Address is taken by the identifier selected by the cursor (there's no read or write operation here)

R: Read access — Data is being read from the address

W: Write access — Data is being written at the address

J: Far (Inter-segment) jump — Code being jumped to is in a different code segment than the current segment

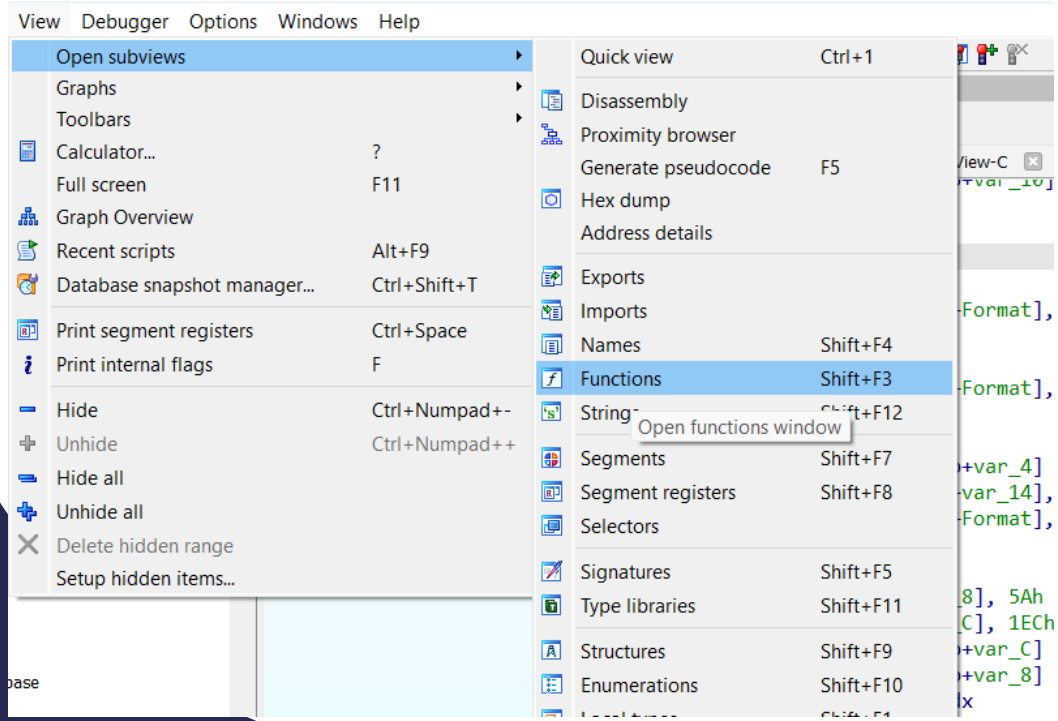
j**: Near (Intra-segment) jump** — Code being jumped to is in the same code segment as the identifier

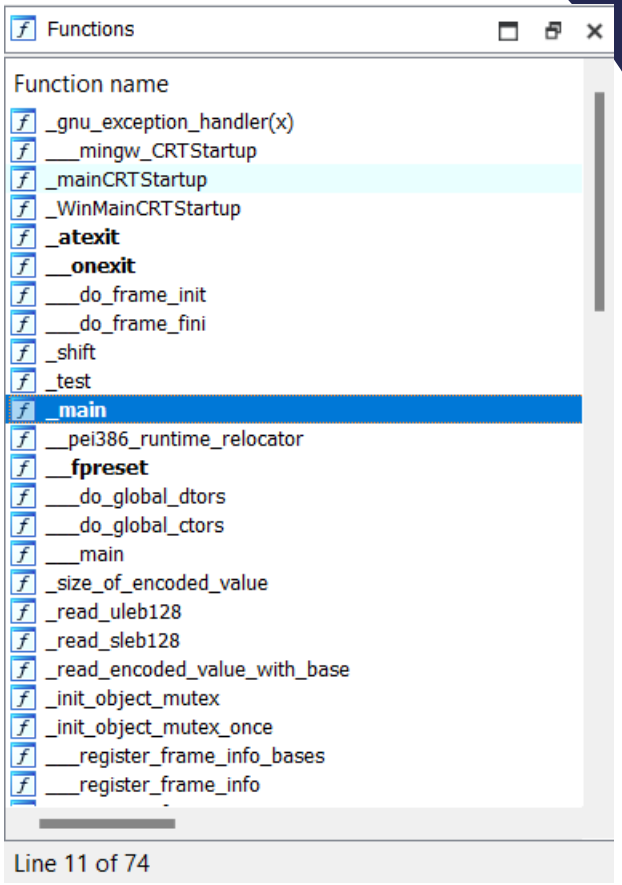
P: Far (Inter-segment) call — Call is in a different code segment

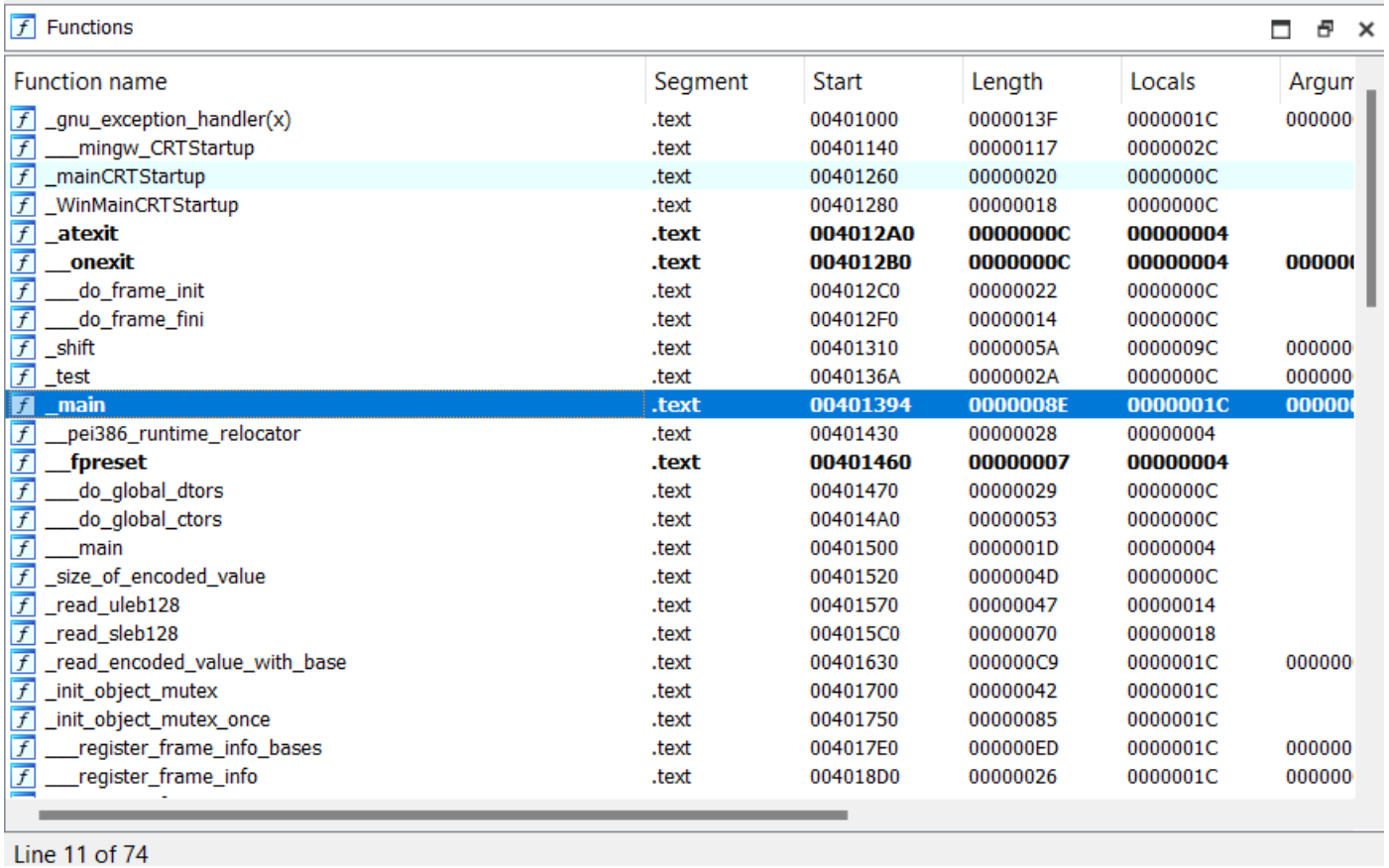
p: Near (Intra-segment) call — Call is in the same code segment

The Functions Window

The Functions window is used to list every function that IDA has recognized in the database





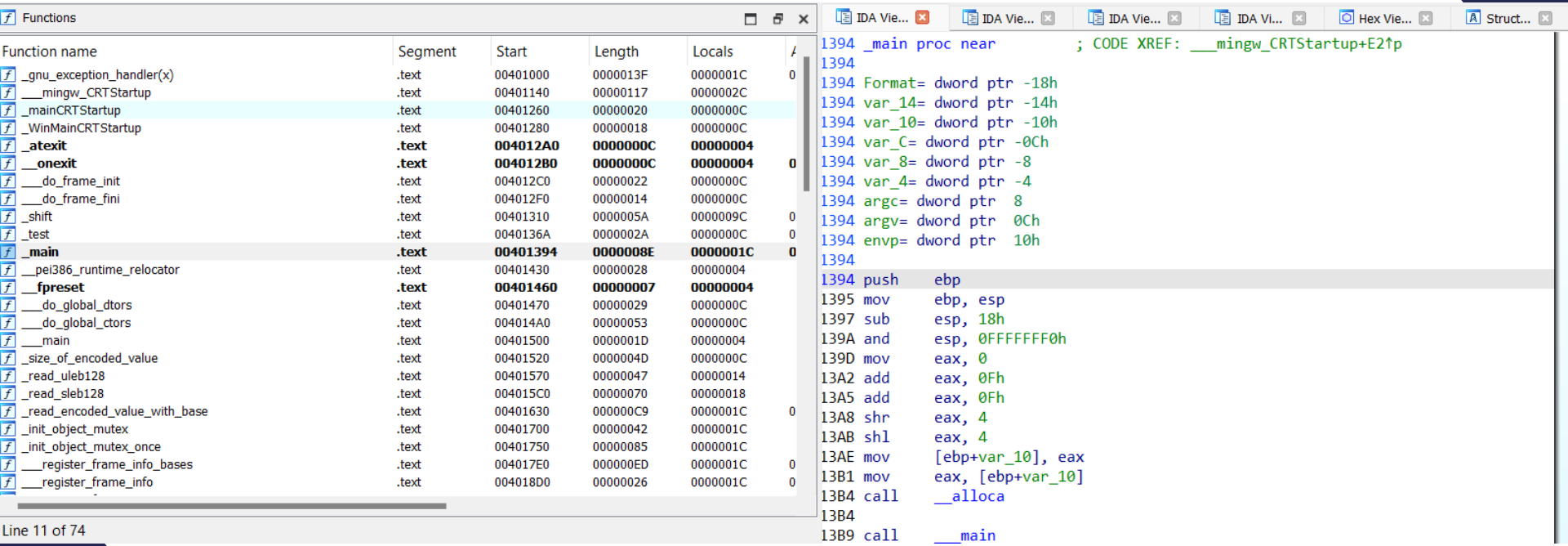

Functions
Function name
Segment
Start
Length
Locals
Argun
_gnu_exception_handler(x) .text 00401000 0000013F 0000001C 000000
___mingw_CRTStartup .text 00401140 00000117 0000002C
_mainCRTStartup .text 00401260 00000020 0000000C
_WinMainCRTStartup .text 00401280 00000018 0000000C
_atexit .text 004012A0 0000000C 00000004
__onexit .text 004012B0 0000000C 00000004 000000
___do_frame_init .text 004012C0 00000022 0000000C
___do_frame_fini .text 004012F0 00000014 0000000C
_shift .text 00401310 0000005A 0000009C 000000
_test .text 0040136A 0000002A 0000000C 000000
_main .text 00401394 0000008E 0000001C 000000
__pei386_runtime_relocator .text 00401430 00000028 00000004
__fpreset .text 00401460 00000007 00000004
___do_global_dtors .text 00401470 00000029 0000000C
___do_global_ctors .text 004014A0 00000053 0000000C
___main .text 00401500 0000001D 00000004
_size_of_encoded_value .text 00401520 0000004D 0000000C
_read_uleb128 .text 00401570 00000047 00000014
_read_sleb128 .text 004015C0 00000070 00000018
_read_encoded_value_with_base .text 00401630 000000C9 0000001C 000000
_init_object_mutex .text 00401700 00000042 0000001C
_init_object_mutex_once .text 00401750 00000085 0000001C
___register_frame_info_bases .text 004017E0 000000ED 0000001C 000000
___register_frame_info .text 004018D0 00000026 0000001C 000000
Line 11 of 74

This particular line indicates that the _main function can be found in the .text section of the binary at virtual address 00401394, is (hex 08E) long, returns to the caller (R), and uses the EBP register (B) to reference its local variables.

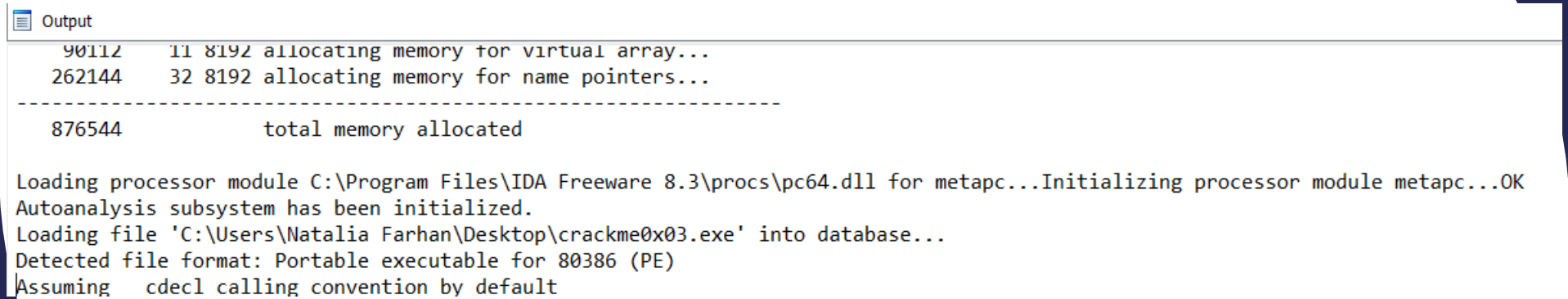
Functions

Function name	Segment	Start	Length	Locals	Arguments	R	F	L	M	O	S	B	T	=
_gnu_exception_handler(x)	.text	00401000	0000013F	0000001C	00000004	R	.	.	.	.	.	B	T	.
___mingw_CRTStartup	.text	00401140	00000117	0000002C		.	.	.	.	.	.	B	.	.
_mainCRTStartup	.text	00401260	00000020	0000000C		.	.	L	.	.	.	B	.	.
_WinMainCRTStartup	.text	00401280	00000018	0000000C		.	.	.	.	.	.	B	.	.
_atexit	**.text**	**004012A0**	**0000000C**	**00000004**		**R**	**.**	**.**	**.**	**.**	**.**	**B**	**T**	**.**
__onexit	**.text**	**004012B0**	**0000000C**	**00000004**	**00000004**	**R**	**.**	**.**	**.**	**.**	**.**	**B**	**T**	**.**
___do_frame_init	.text	004012C0	00000022	0000000C		R	.	.	.	.	.	B	.	.
___do_frame_fini	.text	004012F0	00000014	0000000C		R	.	.	.	.	.	B	.	.
_shift	.text	00401310	0000005A	0000009C	00000004	R	.	.	.	.	.	B	T	.
_test	.text	0040136A	0000002A	0000000C	00000008	R	.	.	.	.	.	B	.	.
_main	**.text**	**00401394**	**0000008E**	**0000001C**	**0000000C**	**R**	**.**	**.**	**.**	**.**	**.**	**B**	**T**	**.**
__pei386_runtime_relocator	.text	00401430	00000028	00000004		R	.	.	.	.	.	B	.	.

As with other display windows, double-clicking an entry in the Functions window causes the disassembly window to jump to the location of the selected function.



The **Output window** serves as IDA's output console and is the place to look for information on tasks IDA is performing.



Software Re-Engineering

Lecture 16

Reverse engineering III – IDA Displays II

Instructor:

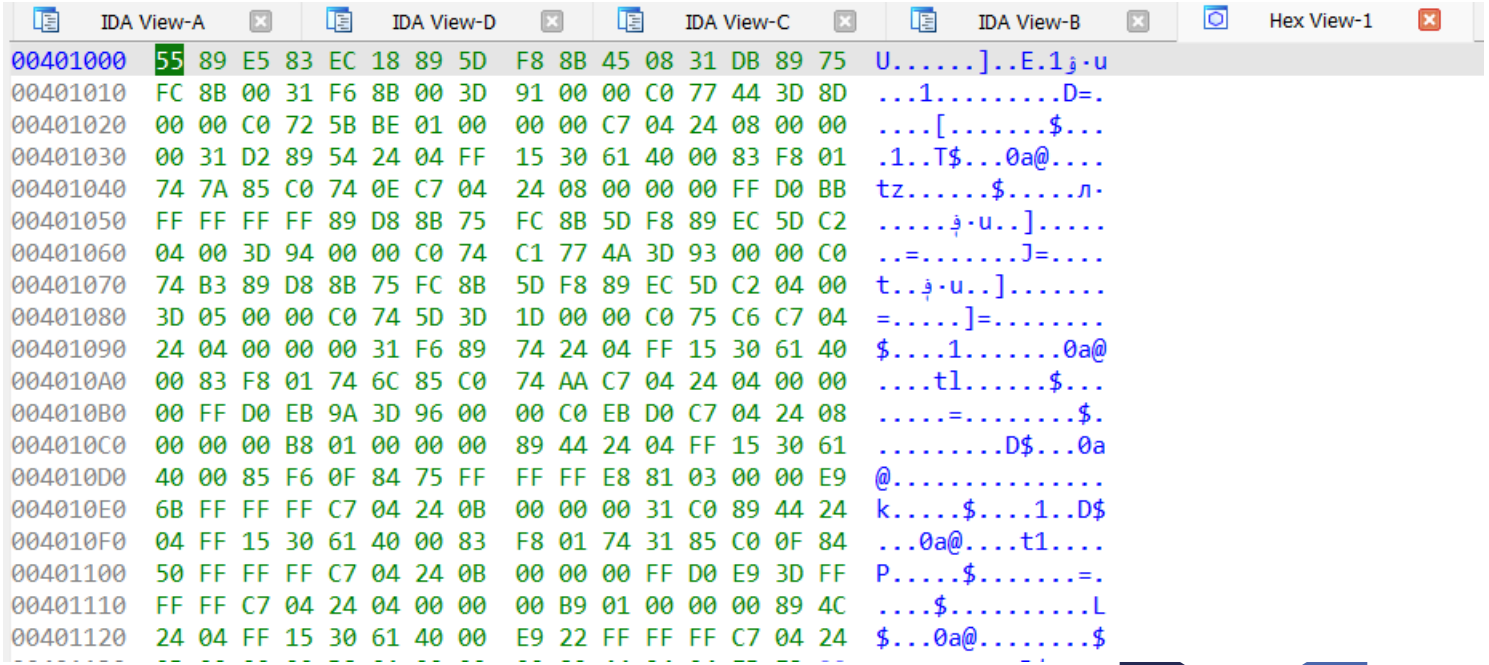
Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

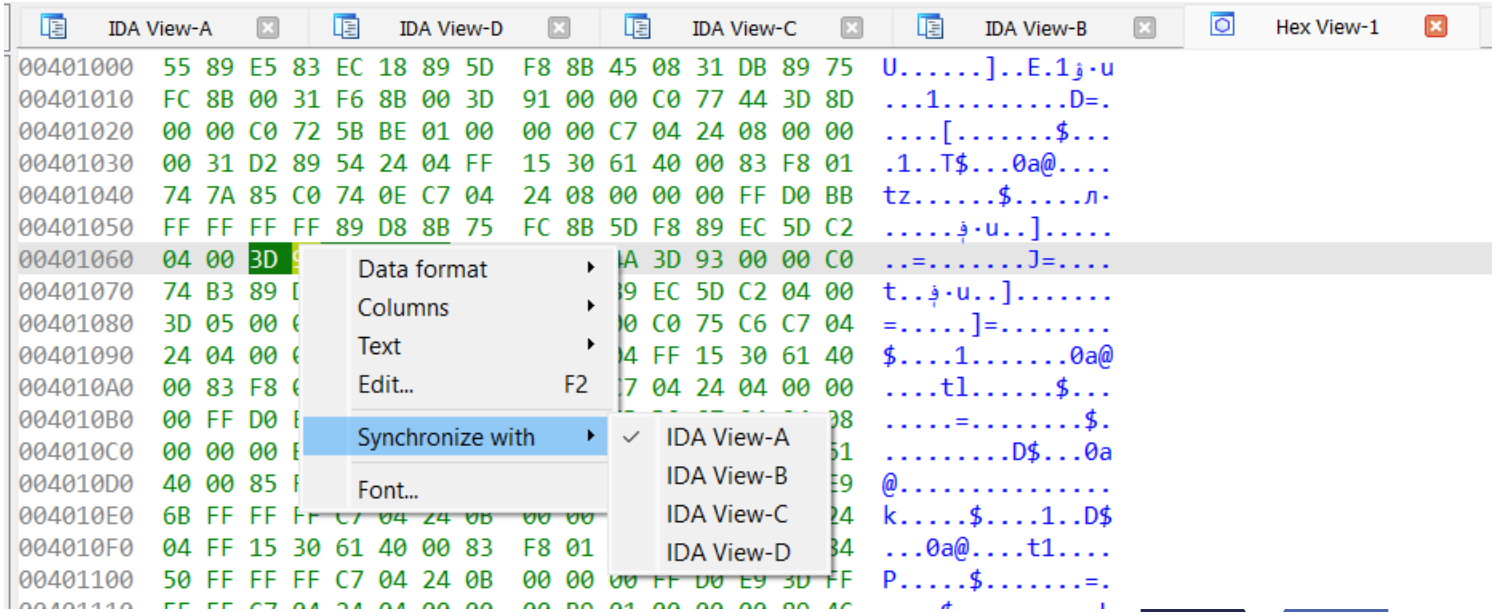
SE Fall 2020

The Secondary IDA Displays

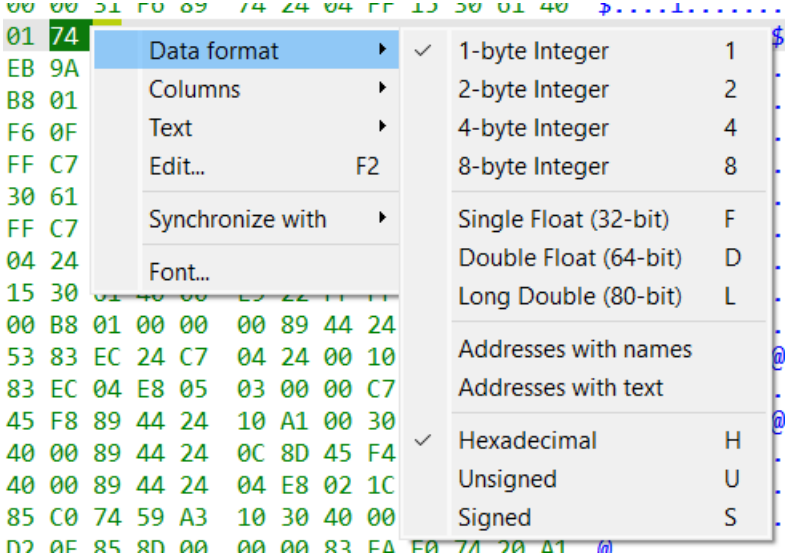
By default, the **Hex View window** provides a standard hex dump of the program content with 16 bytes per line and ASCII equivalents displayed alongside.



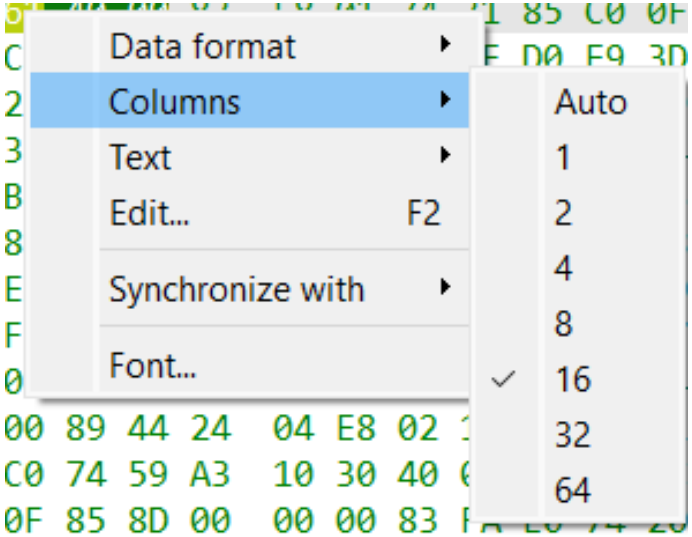
By default, the first Hex window is synchronized with the first disassembly window. When a disassembly view is synchronized with a hex view, scrolling in one window causes the other window to scroll to the same location (same virtual address).



- The Data Format menu item allows you to select from a variety of display formats such as 1-, 2-, 4-, or 8-byte hexadecimal; signed decimal; or unsigned decimal integers and various floating point formats.

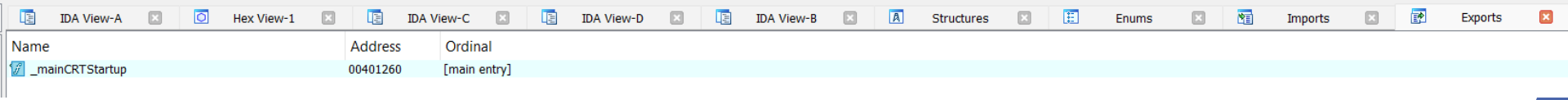


- The Columns menu option allows you to change the number of columns used in the display



Exports Window

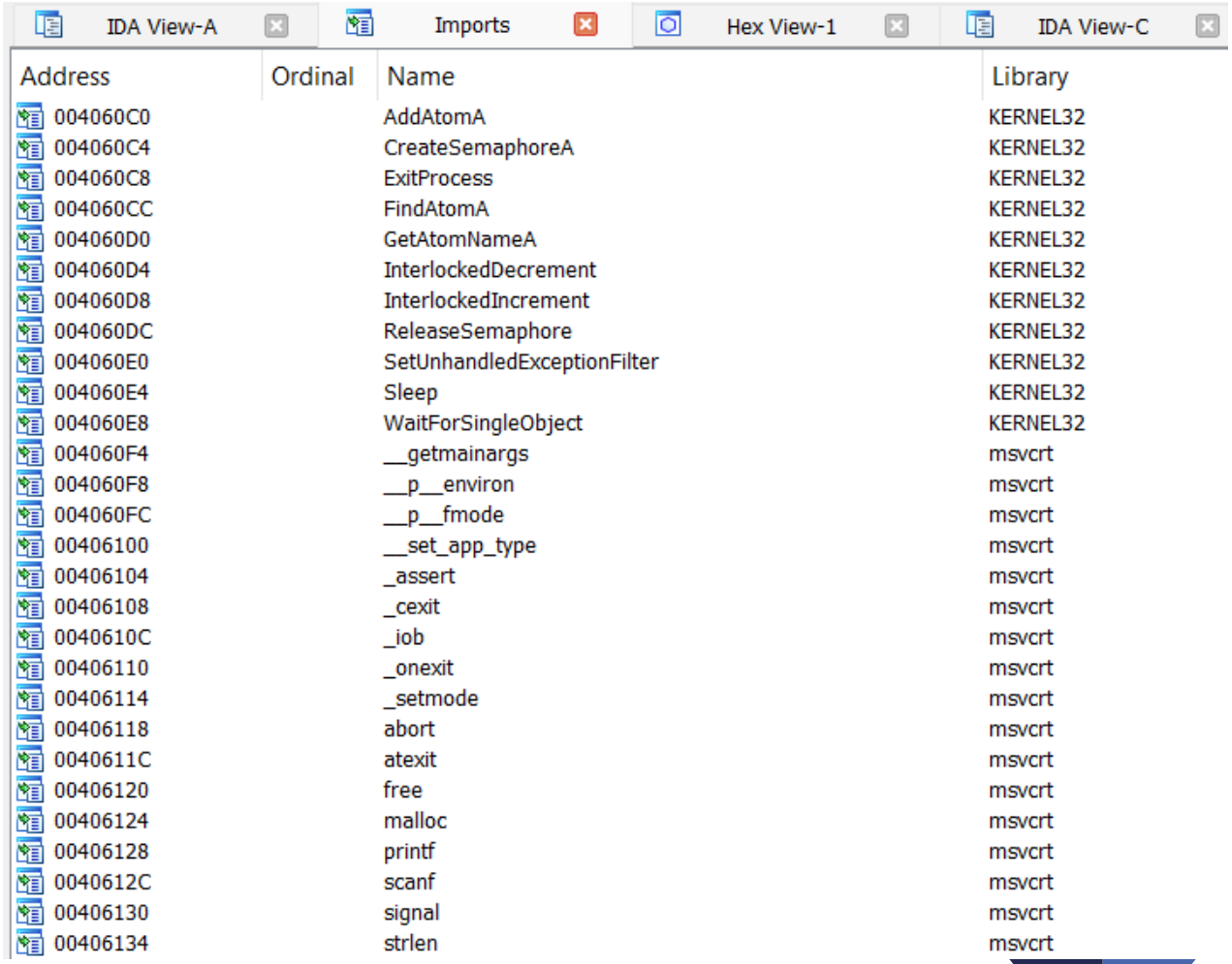
- The Exports window lists the entry points into a file.
- These include the program's execution entry point, as specified in its header section, along with any functions and variables that the file exports for use by other files.
- Exported functions are commonly found in shared libraries such as Windows DLL files.



- Exports window always contains at least one entry: the program's execution entry point. IDA names this entry point start

The Imports Window

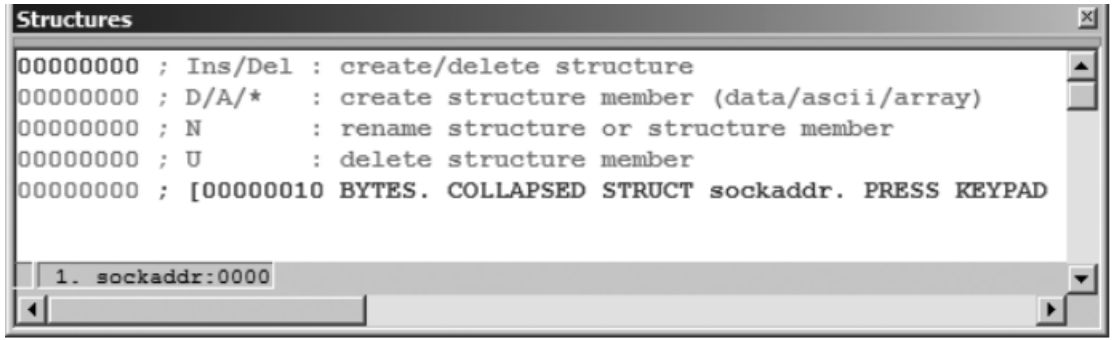
- The Imports window is a counterpart to the Exports window. It lists all functions that are imported by the binary being analyzed.
- The Imports window is relevant only when a binary makes use of shared libraries.
- Statically linked binaries have no external dependencies and therefore no imports.
- Each entry in the Imports window lists the name of an imported item (function or data) and the name of the library that contains that item.



Address	Ordinal	Name	Library
004060C0		AddAtomA	KERNEL32
004060C4		CreateSemaphoreA	KERNEL32
004060C8		ExitProcess	KERNEL32
004060CC		FindAtomA	KERNEL32
004060D0		GetAtomNameA	KERNEL32
004060D4		InterlockedDecrement	KERNEL32
004060D8		InterlockedIncrement	KERNEL32
004060DC		ReleaseSemaphore	KERNEL32
004060E0		SetUnhandledExceptionFilter	KERNEL32
004060E4		Sleep	KERNEL32
004060E8		WaitForSingleObject	KERNEL32
004060F4		__getmainargs	msvcrt
004060F8		__p__environ	msvcrt
004060FC		__p__fmode	msvcrt
00406100		__set_app_type	msvcrt
00406104		_assert	msvcrt
00406108		_cexit	msvcrt
0040610C		_iob	msvcrt
00406110		_onexit	msvcrt
00406114		_setmode	msvcrt
00406118		abort	msvcrt
0040611C		atexit	msvcrt
00406120		free	msvcrt
00406124		malloc	msvcrt
00406128		printf	msvcrt
0040612C		scanf	msvcrt
00406130		signal	msvcrt
00406134		strlen	msvcrt

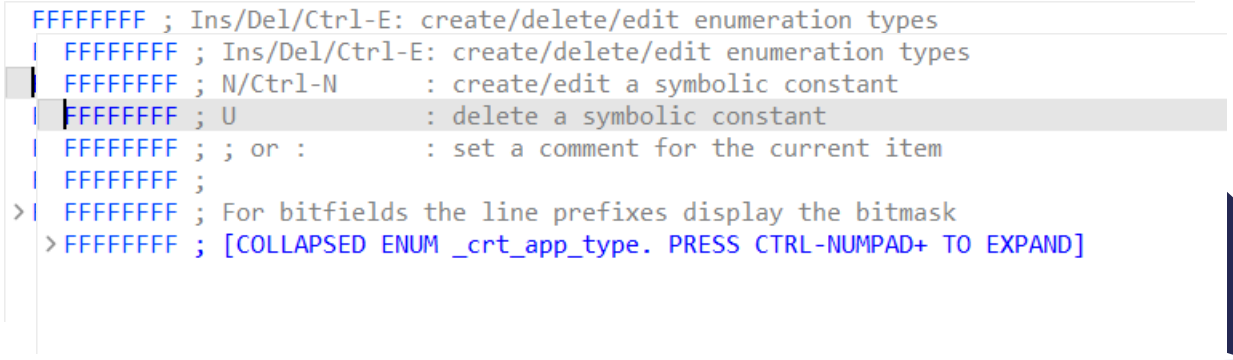
The Structures Window

- The Structures window is used to display the layout of any complex data structures, such as C structs or unions, that IDA determines are in use within a binary.
- During the analysis phase, IDA consults its extensive library of function type signatures in an attempt to match function parameter types to memory used within the program.

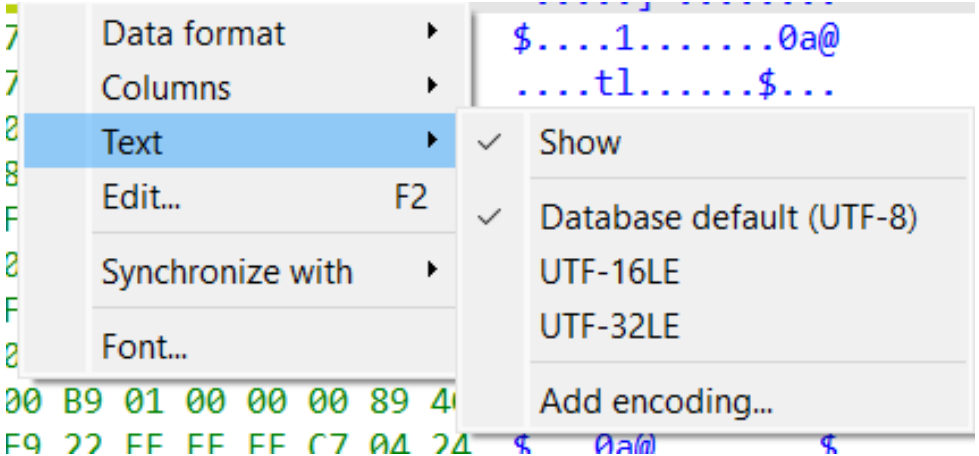


The Enums Window

- The Enums window is somewhat similar to the Structures window. When IDA detects the use of a standard enumerated datatype (C enum), that datatype will be listed in the Enums window.
- You can make your disassemblies far more readable by using enums in place of integer constants.
- Like the Structures window, the Enums window offers facilities for defining your own enumerated types that you can use with your disassembled binaries



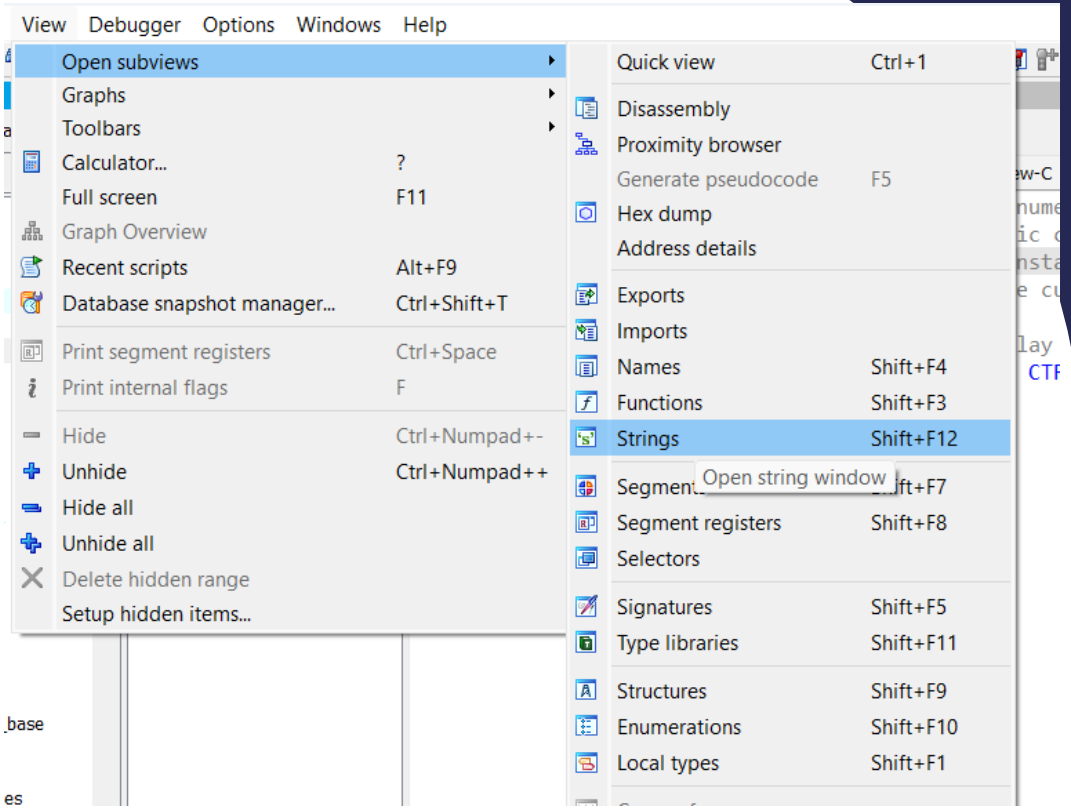
- Text option allows you to turn the text dump on and off.



The Tertiary IDA Displays

- The last windows that we will discuss are those that IDA does not open by default.
- Each of these windows is available via View->Open Subviews, but they tend to provide information to which you may not require immediate access and are thus initially kept out of the way

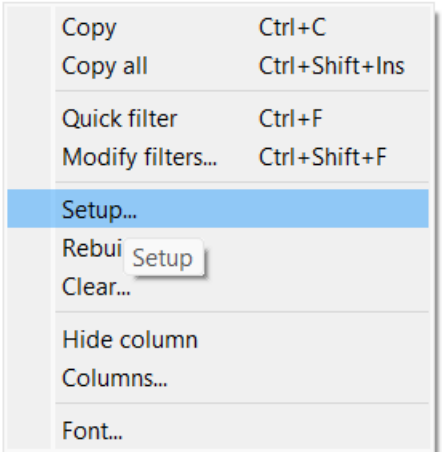
- The purpose of the Strings window is to display a list of strings extracted from a binary along with the address at which each string resides.
- Like doubleclicking names in the Names window, double-clicking any string listed in the Strings window causes the disassembly window to jump to the address of the selected string.

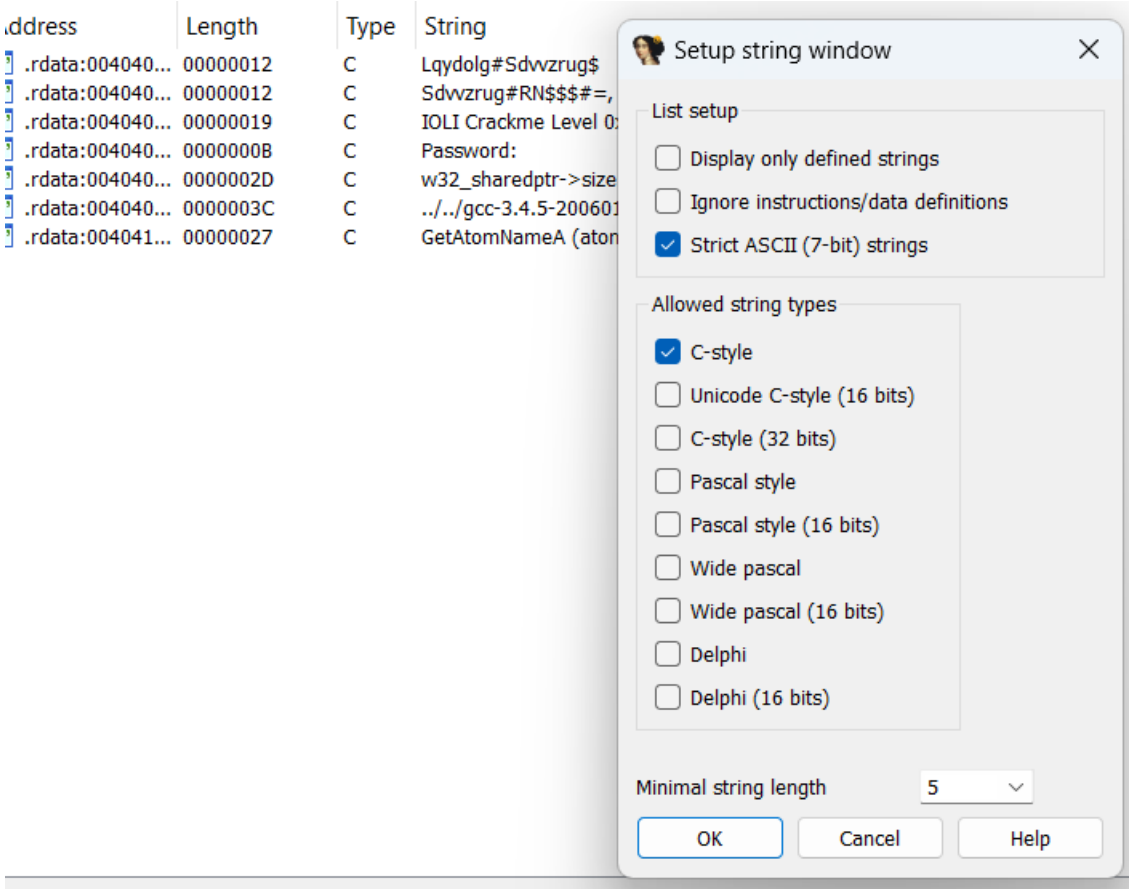


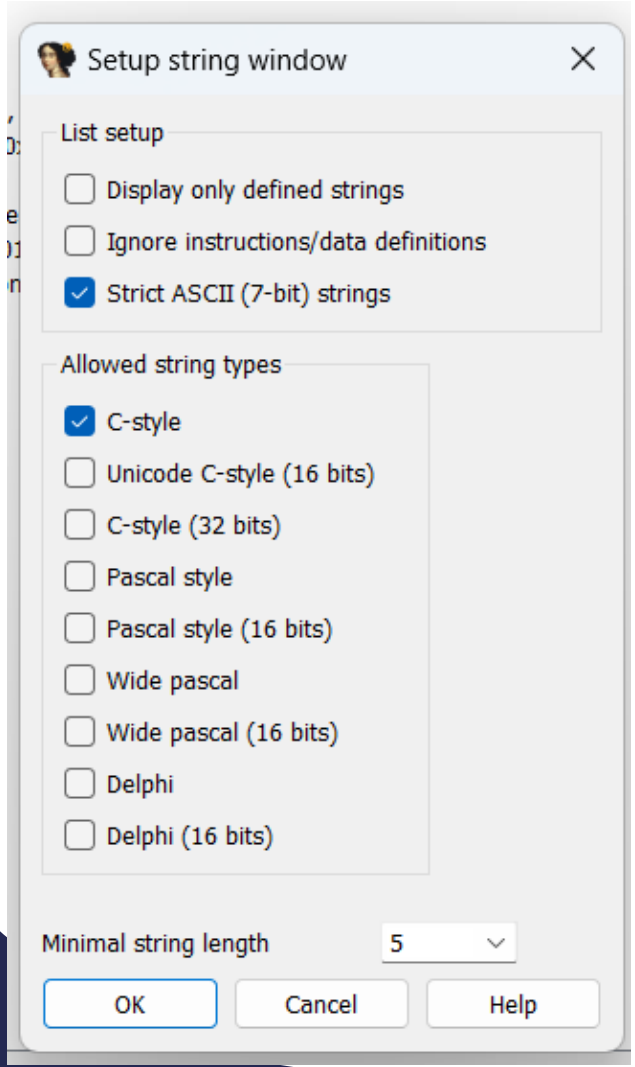
Address	Length	Type	String
.rdata:004040...	00000012	C	Lqydolg#Sdvvzrug$
.rdata:004040...	00000012	C	Sdvvzrug#RN$$$#=,
.rdata:004040...	00000019	C	IOLI Crackme Level 0x03\n
.rdata:004040...	0000000B	C	Password:
.rdata:004040...	0000002D	C	w32_sharedptr->size == sizeof(W32_EH_SHARED)
.rdata:004040...	0000003C	C	../../gcc-3.4.5-20060117-1/gcc/config/i386/w32-shared-ptr.c
.rdata:004041...	00000027	C	GetAtomNameA (atom, s, sizeof(s)) != 0

String setup window

Address	Length	Type	String
.rdata:004040...	00000012	C	Lqydolg#Sdvvzrug$
.rdata:004040...	00000012	C	Sdvvzrug#RN$$$#=,
.rdata:004040...	00000019	C	IOLI Crackme Level 0x03\n
.rdata:004040...	0000000B	C	Password:
.rdata:004040...	0000002D	C	w32_sharedptr->size == sizeof(W32_EH_SHARED)
.rdata:004040...	0000003C	C	../../gcc-3.4.5-20060117-1/gcc/config/i386/w32-shared-ptr.c
.rdata:004041...	00000027	C	GetAtomNameA (atom, s, sizeof(s)) != 0

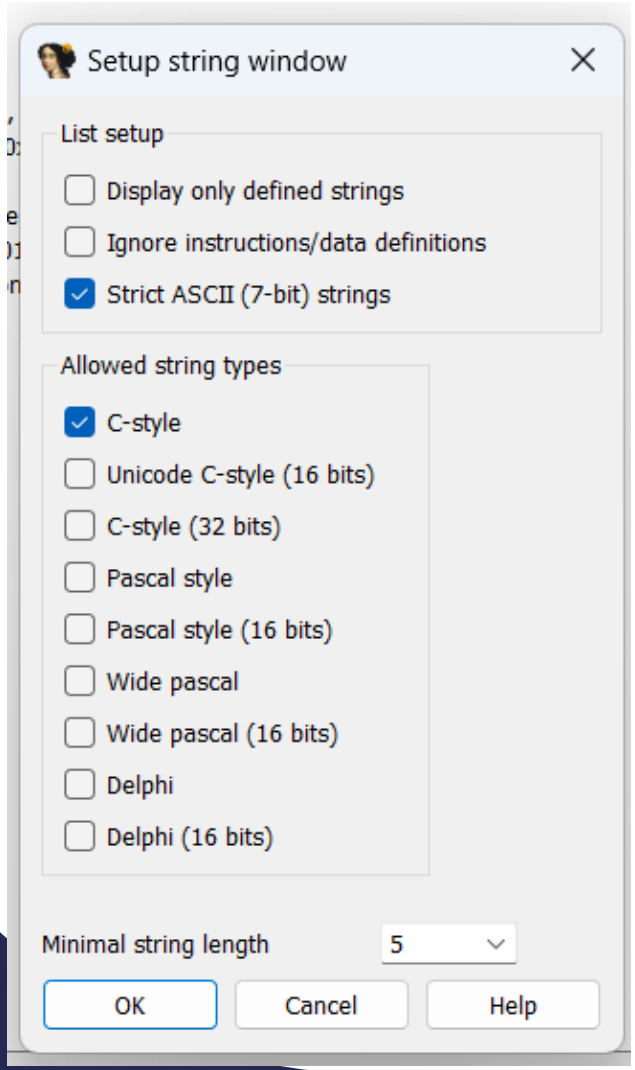



ddress
Length
Type
String
.rdata:004040... 00000012 C Lqydolg#Sdvvzrug$
.rdata:004040... 00000012 C Sdvvzrug#RN$$$#=,
.rdata:004040... 00000019 C IOLI Crackme Level 0
.rdata:004040... 0000000B C Password:
.rdata:004040... 0000002D C w32_sharedptr->size
.rdata:004040... 0000003C C ../../gcc-3.4.5-20060
.rdata:004041... 00000027 C GetAtomNameA (aton
Setup string window
List setup
Display only defined strings
Ignore instructions/data definitions
Strict ASCII (7-bit) strings
Allowed string types
C-style
Unicode C-style (16 bits)
C-style (32 bits)
Pascal style
Pascal style (16 bits)
Wide pascal
Wide pascal (16 bits)
Delphi
Delphi (16 bits)
Minimal string length
5
OK
Cancel
Help



Display only defined strings

- This option restricts the Strings window to displaying only named string data items that have been automatically created by IDA or manually created by the user.
- With this option selected, all other options are disabled, and IDA will not automatically scan for additional string content



Ignore instructions/data definitions

If checked, IDA will ignore instruction/data definitions and will try to treat them as strings.

Using this option allows IDA to

(1) see strings that may be embedded in the code portion of a binary and have been mistakenly converted into instructions or

(2) to see strings within data that may be formatted as something other than a string (such as an array of bytes or integers).

This option will also lead to the generation of many junk strings

The Names window provides a summary listing of all of the global names within a binary.

- A name is nothing more than a symbolic description given to a program virtual address.
- IDA initially derives the list of names from symbol-table and signature analysis during the initial loading of a file.
- Names can be sorted alphabetically or in virtual address order (either ascending or descending).
- The Names window is useful for rapidly navigating to known locations within a program listing.

Name	Address	Public
_abort	**00402DF0**	**P**
__assert	00402E00	P
ExitProcess(x)	**00402E10**	**P**
SetUnhandledExceptionFilter(x)	**00402E20**	**P**
ReleaseSemaphore(x,x,x)	**00402E30**	**P**
InterlockedDecrement(x)	**00402E40**	**P**
WaitForSingleObject(x,x)	**00402E50**	**P**
CreateSemaphoreA(x,x,x,x)	**00402E60**	**P**
Sleep(x)	**00402E70**	**P**
InterlockedIncrement(x)	**00402E80**	**P**
GetAtomNameA(x,x,x)	**00402E90**	**P**
AddAtomA(x)	**00402EA0**	**P**
FindAtomA(x)	**00402EB0**	**P**
___reg_frame_ctor	00402EC0	
___dereg_frame_dtor	00402ED0	
___CTOR_LIST__	00402EE0	P
__DTOR_LIST__	00402EEC	P
__CRT_glob	00403000	P
__fmode	00403010	P
p.0	00403020	
dw2_once.1	00403030	
eh_globals_once.4	00403038	
__data_end__	00403040	P
Format	00404000	
aLqydolgSdvvzru	00404004	
aSdvvzrugRn	00404016	
aIoliCrackmeLev	00404028	
aPassword	00404041	

The coding scheme is summarized below:

F A regular function. These are functions that IDA does not recognize as library functions.

L A library function. IDA recognizes library functions through the use of signature-matching algorithms. If a signature does not exist for a given library function, the function will be labeled as a regular function instead.

I An imported name, most commonly a function name imported from a shared library. The difference between this and a library function is that no code is present for an imported name, while

the body of a library function will be present in the disassembly.

C Named code. These are named program instruction locations that IDA does not consider to be part of any function. This is possible when IDA finds a name in a program’s symbol table but never sees a call to the corresponding program location.

D Data. Named data locations typically represent global variables.

A String data. This is a referenced data location containing a sequence of characters that conform to one of IDA’s known string data types, such as a null-terminated ASCII C string

- If a location is named in the program’s symbol table, IDA adopts the name from the symbol table. If no symbol table entry is available for a given program location, IDA generates a default name for use in the disassembly.
- When IDA chooses to name a location, the virtual address of the location is combined with a prefix that indicates what type of
- location is being named.
- Incorporating the virtual address into a generated name ensures that all generated names will be unique, as no two locations can share the same virtual address.
- Autogenerated names of this type are not displayed in the Names window.

Some of the more common prefixes used for autogenerated names include these:

- **sub_xxxxxx** A subroutine at address
- **xxxxxx loc_xxxxxx** An instruction location at address
- **xxxxxx byte_xxxxxx** 8-bit data at location
- **xxxxxx word_xxxxxx** 16-bit data at location
- **xxxxxx dword_xxxxxx** 32-bit data at location
- **xxxxxx unk_xxxxxx** Data of unknown size at location xxxxxx

The Segments window displays a summary listing of the segments present in the binary file.

- Note that what IDA terms segments are most often called sections when discussing the structure of binary files.
- Do not confuse the use of the term segments in this manner with the memory segments associated with CPUs that implement a segmented memory architecture.
- Information presented in the window includes the segment name, start and end addresses, and permission flags. The start and end addresses represent the virtual address range to which the program sections will be mapped at runtime. T

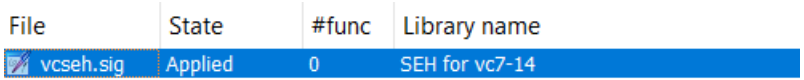
Name	Start	End	R	W	X	D	L	Align	Base	Type	Class	AD	es	ss	ds	fs	gs
.text	00401000	00403000	R	.	X	.	L	para	0001	public	CODE	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.data	00403000	00404000	R	W	.	.	L	para	0002	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.rdata	00404000	00405000	R	.	.	.	L	para	0003	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.bss	00405000	00406000	R	W	.	.	L	para	0004	public	BSS	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.idata	00406000	00407000	R	W	.	.	L	para	0005	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF

Name	Start	End	R	W	X	D	L
.text	00401000	00403000	R	.	X	.	L
.data	00403000	00404000	R	W	.	.	L
.rdata	00404000	00405000	R	.	.	.	L
.bss	00405000	00406000	R	W	.	.	L
.idata	00406000	00407000	R	W	.	.	L

Align	Base	Type	Class	AD	es	ss	ds	fs	gs
para	0001	public	CODE	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0002	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0003	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0004	public	BSS	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0005	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF

The Signatures Window IDA makes use of an extensive library of signatures for identifying known blocks of code.

- Signatures are also used to categorize functions as known library functions inserted by a compiler or as functions added to the binary as a result of static linking.
- When IDA identifies library functions for you, you can focus more of your effort on the code that IDA did not recognize (which is probably far more interesting to you than reverse engineering the inner workings of printf).
- The Signatures window is used to list the signatures that IDA has already matched against the open binary file.

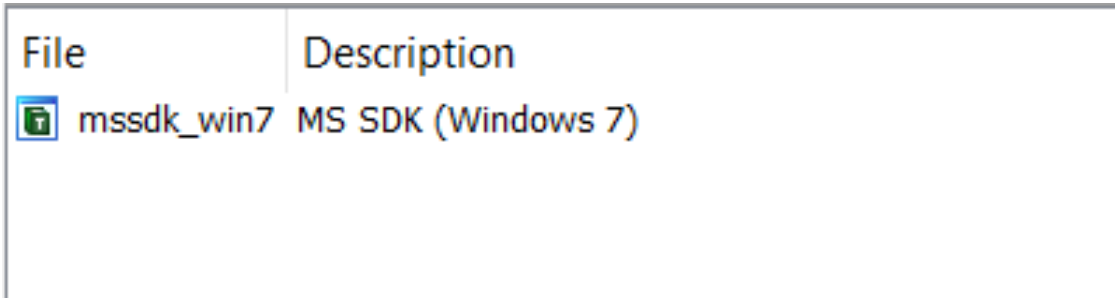


This example indicates that IDA has applied the vc32rtf signatures (from /sigs) against the binary and, in doing so, has been able to recognize 501 functions as library functions. That's 501 functions that you will not need to reverse engineer!

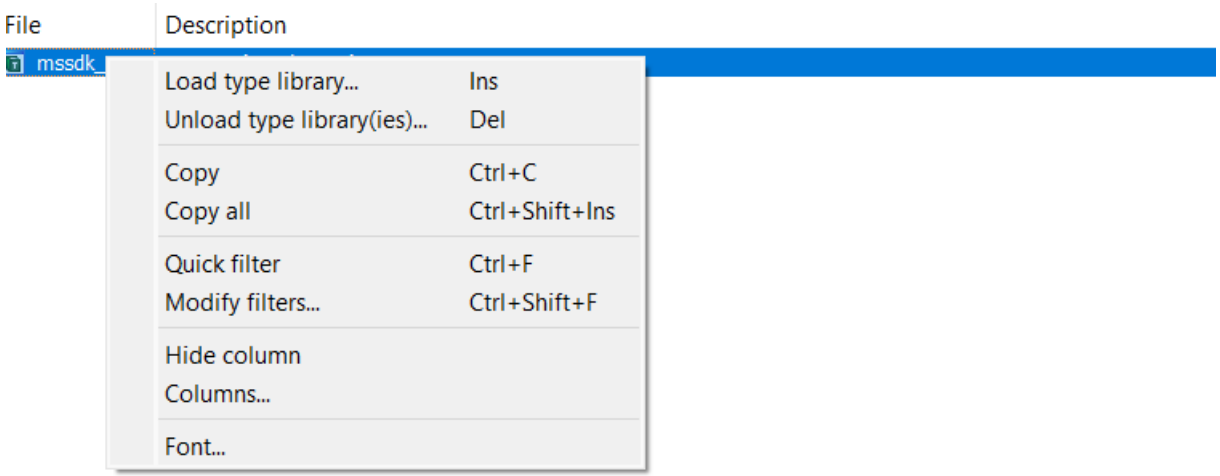
File	State	#func	Library name
vc32rtf	Applied	501	Microsoft VisualC 2-8/net runtime

The Type Libraries Window

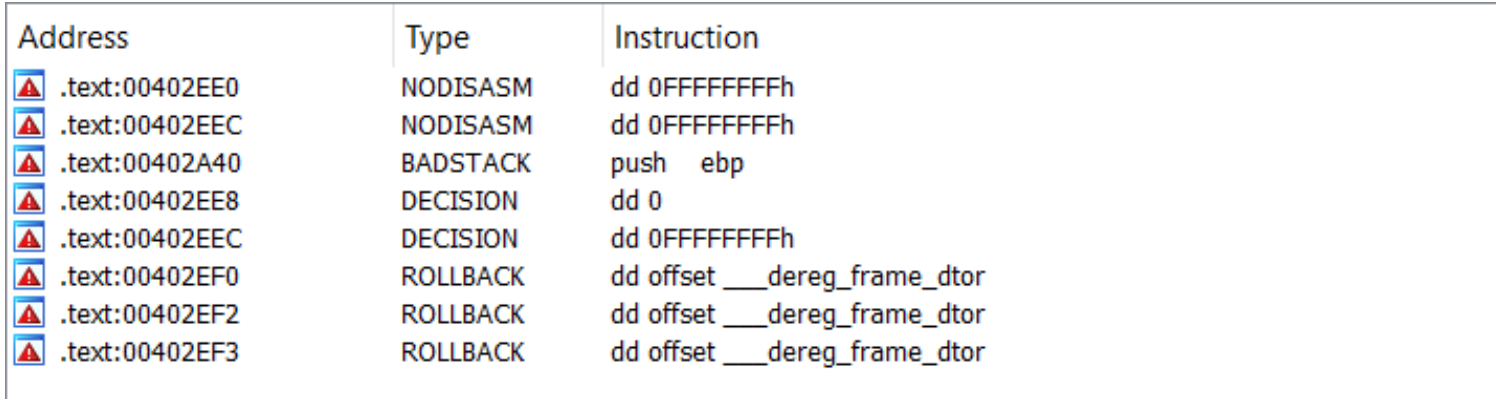
- Type libraries represent IDA’s accumulated knowledge of predefined datatypes and function prototypes gleaned from header files included with most popular compilers.
- By processing header files, IDA understands the datatypes that are expected by common library functions and can annotate your disassemblies accordingly



- Here the user can load and unload standard type libraries.
- The standard type libraries contain type definitions from the standard C header supplied with compilers.



The Problems window is IDA's way of informing you of any difficulties that it has encountered in disassembling a binary and how it has chosen to deal with those difficulties.



Address	Type	Instruction
.text:00402EE0	NODISASM	dd 0FFFFFFFFh
.text:00402EEC	NODISASM	dd 0FFFFFFFFh
.text:00402A40	BADSTACK	push ebp
.text:00402EE8	DECISION	dd 0
.text:00402EEC	DECISION	dd 0FFFFFFFFh
.text:00402EF0	ROLLBACK	dd offset ___dereg_frame_dtor
.text:00402EF2	ROLLBACK	dd offset ___dereg_frame_dtor
.text:00402EF3	ROLLBACK	dd offset ___dereg_frame_dtor

Each problem is characterized by

(1) the address at which the problem occurs,

(2) the type of problem encountered, and

(3) the instruction present at the problem location.

In this example, we see a BOUNDS problem. A BOUNDS problem occurs when the destination of a call or jump either can't be determined (as in this example, since the value of eax is unknown to IDA) or appears to lie outside the range of virtual addresses in a program.

```
Address          Type       Instruction
.text:0040104C   BOUNDS     call    eax
.text:004010B0   BOUNDS     call    eax
.text:00401108   BOUNDS     call    eax
.text:00401350   BOUNDS     call    dword ptr [eax]
.text:004012A0   DECISION   push    ebp
.text:004012D0   DECISION   push    ebp
.text:00401560   DECISION   jmp     ds:__set_app_type
.text:004015F8   DECISION   dd 0FFFFFFFFh
.text:004015FC   DECISION   dd 0
```

The following problems may occur:

- NOOFFSET Cannot find offset base
- NONAME Cannot find name
- NOFORCED Cannot find alternative string for an operand
- NOCMT Cannot find comment
- NOREF Cannot find references
- INDIRJMP Indirect execution flow
- NODISASM Cannot disassemble
- ALREADY Already data or code
- BOUNDS Execution flows beyond limits
- OVERFLOW Too many lines
- BADSTACK Failed to trace the value of the stack pointer
- LOOKHERE Attention! Probably erroneous situation
- DECISION Decision to convert to instruction/data is made by IDA
- ROLLBACK The decision made by IDA was wrong and rolled back
- COLISION FLAIR collision: the function with the given name already exists
- SIGFNREF FLAIR match indecision: reference to function expected

Software Re-Engineering

Lecture 17

Reverse engineering III – IDA Navigation

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

IDA Disassembler

Download IDA Free
https://hex-rays.com/ida-free/#download

Crack
https://github.com/Maijin/radare2-workshop-2015/blob/master/IOLI-crackme/bin-win32/crackme0x03.exe

Binaries
https://github.com/BIackMage/ReverseEngineeringPractice

https://www.youtube.com/watch?v=tWSa1L5L394
https://www.youtube.com/watch?v=Bv-7SszPuvI

https://www.youtube.com/watch?v=d4Pgi5XML8E

Readings

Chapter 6: DISASSEMBLY NAVIGATION, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle

Patching demo

https://medium.com/@k_kisanak/binary-patching-with-ida-pro-part-1-c806d0f20d22

Software Re-Engineering

Lecture 20

Migration to Object oriented platforms

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Readings

Chapter 2 to 5, Object-Oriented Reengineering Patterns by Serge Demeyer

Patterns

Patterns turn out to be a compact way to communicate best practice

Reengineering patterns codify and record knowledge about modifying legacy software: they help in diagnosing problems and identifying weak nesses which may hinder further development of the system, and they aid in finding solutions which are more appropriate to the new requirements.

Design patterns vs reengineering patterns

Design patterns have to do with choosing a particular solution to a design problem, whereas reengineering patterns have to do with discovering an existing design, determining what problems it has, and repairing these problems.

As a consequence, reengineering patterns have more to do with the process of discovery and transformation

Whereas a design pattern presents a solution for a recurring design problem, a reengineering pattern presents a solution for a recurring reengineering problem. The artifacts produced by reengineering patterns are not necessarily designs. They may be as concrete as refactored code

Re engineering patterns

- Setting Direction
- First contact
- Initial understanding
- Detailed model capture

Setting Direction

- Agree on Maxims
- Appoint a Navigator
- Speak to the Round Table
- Most Valuable First
- Fix Problems, Not Symptoms
- If It Ain’t Broke, Don’t Fix It
- Keep It Simple

Initial Understanding

- Analyze the Persistent Data
- Speculate about Design
- Study the Exceptional Entities

First Contact

- Chat with the Maintainers
- Read all the Code in One Hour
- Skim the Documentation
- Interview During Demo
- Do a Mock Installation

Detailed Model Capture

- Tie Code and Questions Refactor to Understand
- Step Through the Execution
- Look for the Contracts
- Learn from the Past

Setting Direction

- Many different factors might affect a re-engineering project
- Technical, ergonomic, economic, and political considerations make it hard to establish and maintain focus
- Hard to set priorities between the many problems of the legacy software
- Danger of focusing on interesting parts rather than what’s good for the system

Setting Direction Patterns

- **Agree on Maxims**

You should Agree on Maxims in order to establish a common understanding within the re-engineering team of what is at stake and how to achieve it.

Maxims are rules of conduct that can help steer a project that is pulled in many directions.

Goldberg and Rubin [GR95] give numerous examples of maxims, such as “Everyone is responsible for testing and debugging” and “You cannot do it right the first time.”

Setting Direction Patterns

- **Appoint a Navigator**

You should Appoint a Navigator to maintain the architectural vision.

The architecture of any system tends to degrade with time as it becomes less relevant to new, emerging requirements.

The challenge of a reengineering project is to develop a new architectural vision that will allow the legacy system to continue to live and evolve for several more years.

Setting Direction Patterns

- **Speak to the Round Table**

Everyone should Speak to the Round Table to maintain team awareness of the state of the project.

Regular, focussed, round table meetings can achieve the goal of keeping team members synchronized with the current state of affairs.

Round table meetings should be brief, but everyone must be required to contribute.

A simple approach is to have everyone say what they have done since the last meeting, what they have learned or perhaps what problems they have encountered, and what they plan to do until the next meeting.

Setting Direction Patterns

- **Most Valuable First**

A legacy system may suffer from a great number of problems, some of which are important, and others which may not be at all critical for the customer’s business.

By focussing on the most valuable parts first, you increase the chance that you will identify the right issues at stake, and that you will be able to test early in the project the most important decisions, such as which architecture to migrate to, or what kind of flexibility to build into the new system.

Setting Direction Patterns

- **Fix problems, not symptoms**

Address the source of a problem, rather than particular requests of your stakeholders.

> If the code of a legacy component is basically stable, and problems mainly occur with changes to clients, then the problem is likely to be with the interface to the legacy component, rather than its implementation, no matter how nasty the code is. In such a case, you should consider applying Present the Right Interface to just fix the interface.
>
> If the legacy component is largely defect-free, but is a major bottle neck for changes to the system, then it should probably be refactored to limit the effect of future changes. You might consider applying Split Up God Class to migrate towards a cleaner design.

Setting Direction Patterns

- **If it ain’t broke, don’t fix it**

Anytime you “fix” something, you also risk breaking something else in the system. You also risk wasting precious time and effort on marginal issues.

Setting Direction Patterns

- **Keep it simple**

An important reengineering goal is to accommodate future change. But too much flexibility will make the new system so complex that you may actually impede future change.

First contract

- All the patterns in this cluster can be applied to the very early stages of a re-engineering project: you're facing a system that is completely new for you, and within a few days you must determine whether something can be done with it and present a plan how to proceed.
- Making such an initial assessment is difficult, however, because you quickly need accurate results while considering the long-term effects of your decisions.

Time is scarce. Wasting time early on in a project has severe consequences later on. This is especially relevant during reverse engineering, because there you feel uncertain and then it is tempting to start an activity that will keep you busy for a while instead of address ing the root of the problem.

Consequently, consider time as your most precious resource. Therefore, defer all time-consuming activities until later and use the first days of the project to assess the feasibility of the project's goals.

All patterns in this cluster are meant to quickly identify the opportunities and risks for your project and as such will help you set the overall direction of the project.

First impressions are dangerous. Making important decisions based on incomplete knowledge implies that there is a chance you will make the wrong decision. There is no way to avoid that risk during your first contact with a system, however you can minimize its impact if you always double-check your sources.

Chat with the Maintainers

Learn about the historical and political context of your project through discussions with the people maintaining the system.

Documentation, if present, typically records decisions about the solution, not about the factors which have influenced that solution. Consequently, the important events in the history of the system (i.e., its historical context) are rarely documented.

Persons working with the system might mislead you. Sometimes people will deliberately deceive you, especially when they are responsible for the problematic parts of the system or when they want to protect their jobs.

Hints

Here are some questions that may help you while discussing with the maintainers.

What was the easiest bug you had to fix during the last month? And what was the most difficult one? How long did it take you to fix each of them? Why was it so easy or so difficult to fix that particular bug?

How does the maintenance team collect bug reports and feature requests? Who decides which request gets handled first? Who decides to assign a bug report or feature request to a maintainer? Are these events logged in some kind of database? Is there a version or configuration management system in place?

These questions help to understand the organization of the maintenance process and the internal working habits of the maintenance team. As far as the political context concerns, it helps to assess the relationship within the team (task assignment) and with the end users (collection of bug reports).

Read all the Code in One Hour

Assess the state of a software system by means of a brief, but intensive code review.

- Grant yourself a reasonably short amount of study time (i.e., approximately one hour) to read the source code. Make sure that you will not be disturbed (unplug the telephone and disconnect your e-mail) and take notes sparingly to maximize the contact with the code.
- After this reading session, produce a short report about your findings, including
  - ❑ a general assessment of whether re-engineering seems feasible and why (not);
  - ❑ entities which seem important (i.e., classes, packages, ⋯);
  - ❑ suspicious coding styles discovered (i.e., “code smells” [FBB+99]);
  - ❑ parts which must be investigated further (i.e., tests).
- Keep this report short, and name the entities like they are mentioned in the source code.

After you **Read all the Code in One Hour** you should Do a Mock Installation to evaluate the suitability for re-engineering. You may complement your findings if you **Skim the Documentation** and carry out an **Interview During Demo** to maximize your chances of getting a coherent view of the system. Before actually making a decision on how to proceed with the re-engineering project, it is probably worthwhile to Chat with the Maintainers once more.

Skim the Documentation

Intent Assess the relevance of the documentation by reading it in a limited amount of time.

- Prepare a list summarizing those aspects of the system that seem interesting for your reengineering project.
- Then, match this list against the documentation and meanwhile make a crude assessment of how up to date the documentation seems.

Finally, summarize your findings in a short report, including

- steer the reading process to match your main interest. For instance, if you want insight into the original system requirements then you should look inside the system specification
- knowledge about which features are actually implemented should be collected from the end-user manual or tutorial notes.
- Check whether the documentation is outdated with respect to the actual system. Always compare version dates with the date of delivery of the system and make note of those parts that you suspect are unreliable.

Interview During Demo

Obtain an initial feeling for the appreciated functionality of a software system by seeing a demo and interviewing the person giving the demo.

- do the demonstration several times with different persons giving the demo.
- This way you will see variations in what people find important and you will hear different opinions about the value of the software system.

Do a Mock Installation

Check whether you have the necessary artefacts available by installing the system and recompiling the code.

- Log all small failures you encounter during the build and installation process and the way you solved them, because this will tell you about the configuration of the system and its dependencies on libraries, frameworks and patches.

Initial Understanding

- When developing your initial understanding of a software system, incorrect information is your biggest concern. Therefore these patterns rely mainly on source-code because this is the only trustworthy information source.
- In principle, there are two approaches for studying source-code: one is top-down, the other is bottom-up. In practice, every reverse engineer ing approach must incorporate a little bit of both, still it is worthwhile to make the distinction. With the top-down approach, you start from a high level representation and verify it against the source-code (as for instance described in Speculate about Design). In the bottom-up approach, you start from the source-code, filter out what's relevant and cast the relevant entities into a higher-level representation

Analyze the Persistent Data

Learn about objects that are so valuable they must be kept inside a database system

Speculate about Design

Progressively refine a design against source code by checking hypotheses about the design against the source code.

Use your development expertise to conceive a hypothetical class diagram representing the design.

Refine that model by verifying whether the names in the class diagram occur in the source code and by adapting the model accordingly.

Repeat the process until your class diagram stabilizes.

Study the Exceptional Entities

Identify potential design problems by collecting measurements and studying the exceptional values.

Measure the structural entities forming the software system (i.e., the inheritance hierarchy, the packages, the classes and the methods) and look for exceptions in the quantitative data you collected.

Verify manually whether these anomalies represent design problems

Detailed Model Capture

- build up a detailed model of those parts of the system that will be important for your reengineering effort.
- It is important to understand that the intent of this pattern is not to improve the code base itself, but only to improve your understanding.
- Since the source code gives you only a very static view of the class hierarchy, it is useful to Step Through the Execution to learn what objects are instantiated and run time and how they interact.

Tie Code and Questions

Keep the questions and answers concerning your reengineering activities synchronized with the code by storing them directly in the source files

In principle there are two ways to annotate the code.

Comment-based Annotations.

This approach uses the commenting conventions of the programming language and as such is better suited for a text-oriented environment. A few conventions are needed to distinguish the normal comments from the annotations.

```
/* #to: John #by: SD #on: 3/12/99 *****
Screws up when we have nested IFs. */
```

Method-based annotations.

This approach exploits the possibility to query which method invokes a given method, a feature provided by many of today's programming environments. The idea is to declare a global method accepting a few strings as an argument and having an empty method body. Each time you want to annotate a particular piece of code, you invoke that method passing your annotations as a parameter.

```
this.annotateCode("#to: John #by: SD #on: 3/12/99",
"Screws up when we have nested IFs.");
```

Refactor to Understand

Iteratively refactor a part of a software system in order to validate and reflect your understanding of how it works

- Iteratively rename and refactor the code to introduce meaningful names and to make sure the structure of the code reflects what the system is actually doing.
- Run regression tests after each change if they are available, else compile often to check whether your changes make sense.
- Decide what to do with the code after you have refactored it.

Your primary goal here is to understand the system, not to improve the code. The changes you make to the code should therefore be treated as “experiments” to test your understanding of the code. As a consequence, you should make a copy of the code before you start.

The following guidelines will help you to find out where and how to apply these refactorings to improve the readability of the code.

Rename attributes to convey roles.
To find out about their roles, look at all the attribute accesses (including invocations of accessors methods). Afterwards, rename the attribute and its accessors according to its role, update all references and re-compile the system.

Rename methods to convey intent.
To retrieve the intent of a method that does not have an intention revealing name, investigate all invo cations and attribute uses, and deduce the method's responsibility. Afterwards, rename the method according to its intent, update all invocations and re-compile the system.

Rename classes to convey purpose.
Remove duplicated code.

Replace condition branches by methods.

If you encounter conditions with large branches, extract the leaves as new (private) methods. To name these methods, study the condition until you understand it well enough to choose an intention revealing name.

Refactor method bodies to a consistent level of abstraction.

Long method bodies with comments separating blocks of code violate the rule of the thumb that all statements in a single method body should have the same level of abstraction. Refactor such code by introducing a new (private) method for each separated block of code; name the method after the intent recorded in the comment.

Step Through the Execution

Understand how objects in the system collaborate by stepping through examples in a debugger.

- It is too time-consuming to step through every single statement of a run ning system.
- The assumption here is that you are focussed on some specific aspect of the system that is difficult to understand.
  - ❑ Set breakpoints to interrupt execution when the system enters the code you are interested in.
  - ❑ Change the internal state of the objects to see how alternative execution paths are triggered.

Look for the Contracts

Infer the proper use of a class interface by studying the way clients currently use it.

Look for common programming idioms that expose the way clients make use of the class interface.
Generalize your observations in the form of contracts, i.e., explicit declarations of what a class expects from its clients.

Learn from the Past

Obtain insights into the design by comparing subsequent versions of the system.

- Use the metrics or configuration management tool to find entities where functionality has been removed, because such entities are a sign of a consolidating design.
- Also, look for entities which change often as these may point you to an unstable part of the design.

Unstable design.

If you detect repeated growth and refactoring in the same portion of the system, that should be a sign that the design is unstable. It may indicate opportunities to redesign that portion of the system to bet ter accommodate the kinds of changes and extensions that habitually take place.

Legacy systems often do not have test procedures defined.

Changing parts of a system without introducing new bugs is a challenging task.



Readings

Chapter 6 to 10, Object-Oriented Reengineering Patterns by Serge Demeyer

TESTS: YOUR LIFE INSURANCE!

- Write Tests to Enable Evolution
- Grow Your Test Base Incrementally
- Use a Testing Framework
- Test the Interface, Not the Implementation
- Record Business Rules as Tests
- Write Tests to Understand

Detecting Duplicated Code

- Compare Code Mechanically
- Visualize Code as Dot plots

Redistribute Responsibilities

- Move Behavior Close to Data
- Eliminate Navigation Code
- Split Up GodClass

Migration Strategies

- Involve the Users
- Build Confidence
- Migrate Systems Incrementally
- Prototype the Target Solution
- Always Have a Running Version
- Regression Test After Every Change
- Make a Bridge to the New Town
- Present the Right Interface
- Distinguish Public from Published Interface
- Deprecate Obsolete Interfaces
- Conserve Familiarity
- Use Profiler Before Optimizing

Transform Conditionals to Polymorphism

- Transform Self Type Checks
- Transform Client Type Checks
- Transform Conditionals into Registration
- Factor out State
- Factor out Strategy
- Introduce Null Object

Write Tests to Enable Evolution

Protect your investment in the legacy code by imposing a systematic testing program.

Introduce a testing process based on tests that are automated, repeatable and stored.

Well-designed tests exhibit the following properties:

- **Automation**. Tests should run without human intervention.
- **Persistence**. Tests must be stored to be automatable. Each test documents its test data, the actions to perform, and the expected results.
- **Repeatability**. Confidence in the system is increased if tests can be repeated after any change is implemented. Whenever new functionality is added, new tests can be added to the pool of existing tests, thereby increasing the confidence in the system
- **Unit testing**. Tests should be associated to individual software components so that they identify clearly which part of the system they test
- **Independence.** Each test should minimize its dependencies on other tests. Dependent tests typically result in avalanche effects: when one test breaks, many others break as well.

TESTS: YOUR LIFE INSURANCE!

Grow Your Test Base Incrementally

Balance the costs and the benefits of tests by incrementally introducing just the tests you need at a given point in time.

Carefully assess your priorities and initially develop tests only for the most critical components. As you reengineer the system, intro duce tests for the new features, parts of the legacy that may be affected, and any bugs you identify along the way.

Use a Testing Framework

Encourage developers to write and use regression tests by providing a framework that makes it easy to develop, organize and run tests.

Use a testing framework that allows suites of tests to be composed from individual test cases.

TESTS: YOUR LIFE INSURANCE!

Test the Interface, Not the Implementation

Also Known As: Black-Box Testing

Build up reusable tests that focus on external behavior rather than on implementation details, and thereby will survive changes to the system.

- Legacy systems have many features that should continue to function as the system evolves.
- You cannot afford to spend too much time writing tests while reengineering the system. You do not want to waste effort in developing tests that will have to be changed as you change the system.

Record Business Rules as Tests

Keep the system in sync with the business rules it implements by encoding the rules explicitly as tests.

Write executable tests that record the business rules as test cases, actions, and tests over the results. When tests break, you know that things are out of sync.

Write Tests to Understand

Record your understanding of a piece of code in the form of executable tests, thus setting the stage for future changes.

- Tests can be applied to gain different levels of understanding. For example, black-box tests can help you to refine your understanding of roles and collaborations, whereas white-box tests can help you to gain understanding of the implementation of complex logic.
- The tests that you develop will help to enable future reengineering effort. Tests will force you to be precise about the creation and the use of the objects under test.

TESTS: YOUR LIFE INSURANCE!

Migration Strategies

- How can you be sure that the new system will be accepted by users?
- How do you migrate to the new system while the old system is being used?
- How can you test and evaluate the new system before it is finished?

It is not enough to reengineer a legacy system and then deploy it. In fact, if you try this, you will surely fail (for the same reasons that big Water fall projects in new territories often fail). You must be prepared to intro duce the new solution gradually, to gain the confidence and collaboration of the users, and you must adopt a strategy for migrating gradually and painlessly from the existing system, while it is still being deployed, to the new system.

Involve the Users

Also Known As: Engage Customers

Maximize acceptance of changes by involving the users at every step

Create an environment that will encourage contact between users and developers. Physical location is important.

Establish simple procedures for delivering intermediate results on a regular basis and obtaining feedback.

Early prototypes may help, especially to evaluate risky new technologies or approaches.

Build Confidence

Improve your chances of overall success by demonstrating results in regular increments.

Pick short intervals for delivering new results. At each step, try to agree together with the users what are the smallest results that can demonstrate real value.

By taking smaller steps, you reduce the risk that an individual step will fail.

Frequent, positive results help to build confidence.

Migrate Systems Incrementally

Also Known As: Chicken Little

Avoid complexity and risk of big-bang reengineering by deploying functionality in frequent increments

Deploy a first update of the legacy system as soon as you can, and migrate incrementally to the target system.

Steps

- Decompose the legacy system into parts.
- Choose one part to tackle at a time.
- Put tests in place for that part and the parts that depend on it
- Take appropriate steps to wrap, reengineer or replace the legacy component.
- Deploy the updated component and obtain feedback.
- Iterate.

Prototype the Target Solution

Evaluate the risk of migrating to a new target solution by building a prototype

Steps

- Identify the biggest technical risks for your reengineering project. Typically they will concern things like:

– choice of a new system architecture
– migration of legacy data to new system
– adequate performance — or performance gains — with new technology or platform (for example, demonstrating that a cer tain transaction throughput can be achieved)

- Decide whether to implement an **exploratory (i.e., throwaway)** pro totype that will service purely to evaluate the feasibility of a technical option, or rather an **evolutionary prototype** that will eventually evolve into the new target system

- The exploratory prototype does not need to be designed to address any other issues or questions, and will not be part of the migrated system. An evolutionary prototype, on the other hand, is intended to eventually replace a legacy component, and must therefore reflect the target architecture.

Always Have a Running Version

How do you convince your customer that you are on the right path?

This problem is difficult because:

- It can be hard to demo a software system under development, or to discuss problems with users since there is often no stable, running version of the system available.

- Integrating changes from multiple versions of a system can be slow and painful.

Institute a discipline of integrating new changes and developments on a daily basis.

Steps

- Have version management and configuration management systems in place.
- Make sure you have regression tests in place for the parts you are working on.
- Institute a discipline of short transactions for checking out system components and checking them back in again.
- Plan iterations to be as short as possible to allow changes to be integrated into a running system

Many practitioners advocate a process of continuous integration as a way to avoid a risky and painful big-bang integration

Regression Test After Every Change

Build confidence by making sure that whatever worked before still works

Run your regression test suite every time you think you have reached a stable state

If you consistently build up tests for defects you discover and new features, you will end up with a reusable test base that gives you confidence that your changes are sound, and helps you detect problems earlier.

Make a Bridge to the New Town

Migrate data from a legacy system by running the new system in parallel, with a bridge in between

Make a(data) bridge that will incrementally transfer data from the legacy system to the replacement system as new components are ready to take the data over from their legacy counterparts

Steps

- Identify legacy and replacement components that deal with the same logical data entities.
- Implement a “data bridge” which is responsible for redirecting read requests from the new component to the legacy data source, if the data have not already been migrated.
- The bridge is responsible for any necessary data conversion. The new component should not be aware of the bridge.
- Adapt the legacy component to redirect write requests to the new component, so that the new data stay up-to-date.
- When all the data have been transferred, remove the bridge and the legacy component.

Present the Right Interface

Also Known As: Semantic Wrapper

How should the new target system access legacy services during the migration process?

Identify the abstractions that you want to have in the new system, and wrap up the old software to emulate the new abstractions.

Present the Right Interface frees you from thinking in terms of the legacy design and makes it easier to consider alternative approaches.

Consider, **for example**, a procedural graphics library that will be used within an object-oriented system. It will be too costly and time-consuming to reimplement the library in an object-oriented way. It would be easier to wrap it as a utility class (i.e., as a class with static methods but no instances), but it would be wiser to write a slightly thicker wrapper that presents a truly object-oriented interface, but is implemented using the underlying procedural abstractions. In this way the new system will not be polluted by legacy abstractions.

Distinguish Public from Published Interface

Facilitate parallel development by distinguishing unstable “published interfaces” from stable “public interfaces”.

Since “published” interfaces are not supported by any programming language, you may have to use naming conventions, or abuse other features to achieve the desired effect.

In Java, consider declaring such interfaces as protected, or giving them package scope(undeclared). When the interfaces stabilize, you may redeclare them as being public.

In C++, consider declaring components with published interfaces private or protected, and declare as friends the clients that are permitted to use them. When the interfaces stabilize, redeclare the components as public, and delete the declarations of friends.

Deprecate Obsolete Interfaces

Give clients time to react to changes to public interfaces by flagging obsolete interfaces as “deprecated”.

Flag the old interface as being “deprecated”, thereby notifying clients that it will almost certainly be removed in the next upcoming release.

Steps

- You have determined that a public interface should be changed, but you do not want to break all clients. Implement the new interface, but “deprecate” the old one.
- The deprecation mechanism should inform clients that the interface has changed, and that a newer interface is recommended instead.
- Evaluate to what extent the deprecated interface continues to be used, and whether it can be permanently retired. Consider removing it in a future release.

Conserve Familiarity

The legacy system requires significant changes.
The users are not happy with the legacy system, but they understand it well.

- You can migrate incrementally to a new solution.
- Introduce only a constant, relatively low number of changes between each new release.
- Users do not have to change their work habits too much between releases.

Use Profiler Before Optimizing

- When you are reengineering software, you are likely to encounter many naive algorithms in the legacy code.
- It can be hard to predict what will impact performance, and you can lose a lot of time on pure supposition.
- Optimized code is often more complex than simple, naive code.

- Whenever you are tempted to optimize a “clearly inefficient” part of the system, first use a profiler to determine whether it is actually a bottleneck.
- Don’t optimize anything unless your profiler tells you it will make a difference.
- If you decide to go ahead, prepare benchmarks that will demonstrate the performance gains.
- You do not waste time optimizing something that will not make a difference to overall performance.

Detecting Duplicated Code

- Duplicated code hampers the introduction of changes, since every implemented variant of a piece of functionality will have to be changed.
- Since it is easy to miss some variants, bugs are likely to pop up in other places.
- Duplicated code replicates and scatters the logic of a system instead of grouping it into identifiable artifacts (classes, methods, packages).
- It leads to systems that are more difficult to understand and to change.

Detecting Duplicated Code consists of **two patterns**:

1. Compare Code Mechanically, which describes how we can detect duplicated code, and
2. Visualize Code as Dot plots, which shows how duplicated code can be better understood by simple matrix visualization.

Compare Code Mechanically

Discover duplicated code by comparing all the source code files line-by line.

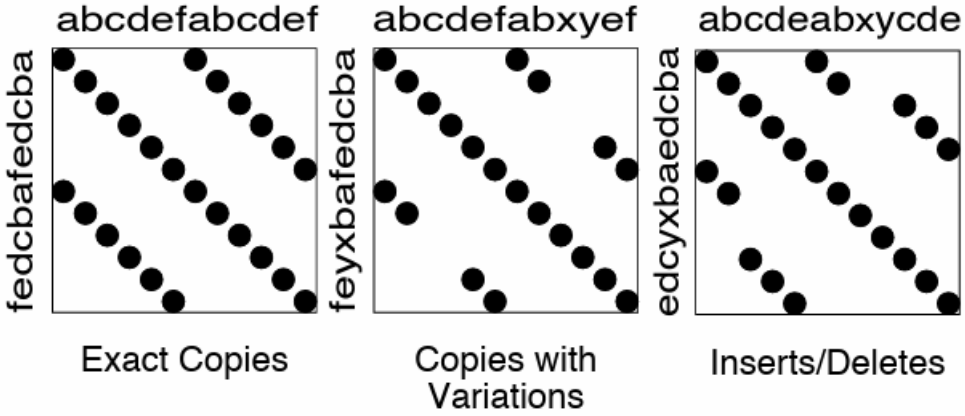
Steps

- Normalize the lines of code by removing comments, tabs and blanks.
- Remove lines that contain uninteresting code elements (e.g., just else or })
- Compare each line with all the other lines.
- Reduce search complexity by hashing:
- **Preprocessing:**
  - ❑ Compute the hash value for each line
  - ❑ Actual Comparison: Compare all lines in the same hash bucket

Visualize Code as Dot plots

Gain insight into the nature of the duplication by studying the patterns in the dot plots.

Visualize the code as a matrix in which the two axes represent two source code files (possibly the same file), and dots in the matrix occur where source code lines are duplicated.



Possible sequences of dot and their associated interpretations

Redistribute Responsibilities

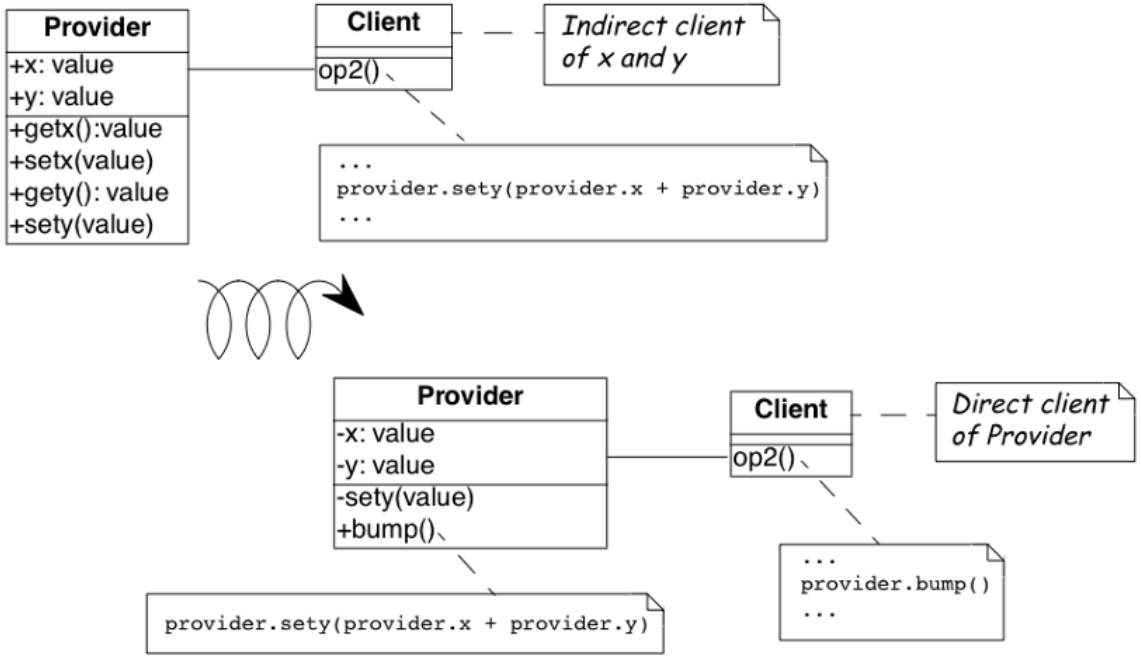
- This cluster deals with problems of misplaced responsibilities.
- The two extreme cases are **data containers**, classes that are nothing but glorified data structures and have almost no identifiable responsibilities, and **god classes**, procedural monsters that assume too many responsibilities.

Data containers (objects that just provide access to data, but no own behavior) are a simple and convenient way to share information between many subsystems.

God classes (big classes that do all of the work) are hard to extend, modify or subclass because such changes affect large numbers of other methods or instance variables.

Move Behavior Close to Data

Strengthen encapsulation by moving behavior from indirect clients to the class containing the data it operates on.



Classes that were mere data containers are transformed into real service providers

Redistribute Responsibilities

- Data containers are converted to service providers with clear responsibilities.
- The service providers become more useful to other clients.
- Clients are no longer responsible for implementing provider behavior.

It makes use of the refactorings **ExtractMethod** and **MoveMethod**, since the behavior in question will have to be extracted from a client method and then moved to a provider class.

1. Identify the client behavior that you want to move, i.e., the complete method or a part of a method that accesses provider data.
2. Create the corresponding method in the provider class, if it does not already exist.
3. Give an intention-revealing name to the new method.
4. In the client invoke the new provider method with the correct parameters.

Eliminate Navigation Code

Reduce the impact of changes by shifting responsibility down a chain of connected classes.

How do you reduce coupling due to classes that navigate through the object graph?

Changes in the interfaces of a class will affect not only direct clients, but also all the indirect clients that navigate to reach it.

Iteratively move behavior defined by an indirect client to the container of the data on which it operates.

Look for indirect clients that contain a lot of navigation code. Navigation code is of two kinds:

1. a sequence of attribute accesses, e.g.,a.b.c.d where b is an attribute of a, c is an attribute of b and d an attribute of c. The result of such a sequence can be assigned to variable or a method of the last object can be invoked, e.g.,a.b.c.d.op().
2. A sequence of accessor method calls. In Java and C++ such a sequence has the form object.m1().m2().m3() where object is an expression returning an object, m1 is a method of object, m2 a method of the object returned by the invocation of m1, m3 a method of the object returned by the invocation of m2 and so on.

Navigation code can be detected by simple pattern matching.

Split Up God Class

Split up a class with too many responsibilities into a number of smaller, cohesive classes.

- Evolution of the application is difficult because nearly every change touches this class, and affects multiple responsibilities.
- It is difficult to understand the different abstractions that are inter mixed in a god class. Most of the data of the multiple abstractions are accessed from different places.
- Identifying where to change a feature without impacting the other functionality or other objects in the system is difficult.
- Moreover, changes in other objects are likely to impact the god class, thus hampering the evolution of the system.
- It is nearly impossible to change a part of the behavior of a god class in a black-box way.

Pros: **Application control is no longer centralized in a single monolithic entity but distributed amongst entities that each assume a well defined set of responsibilities. The design evolves from a procedural design towards an object-oriented design based on autonomous interacting objects.**

Redistribute Responsibilities

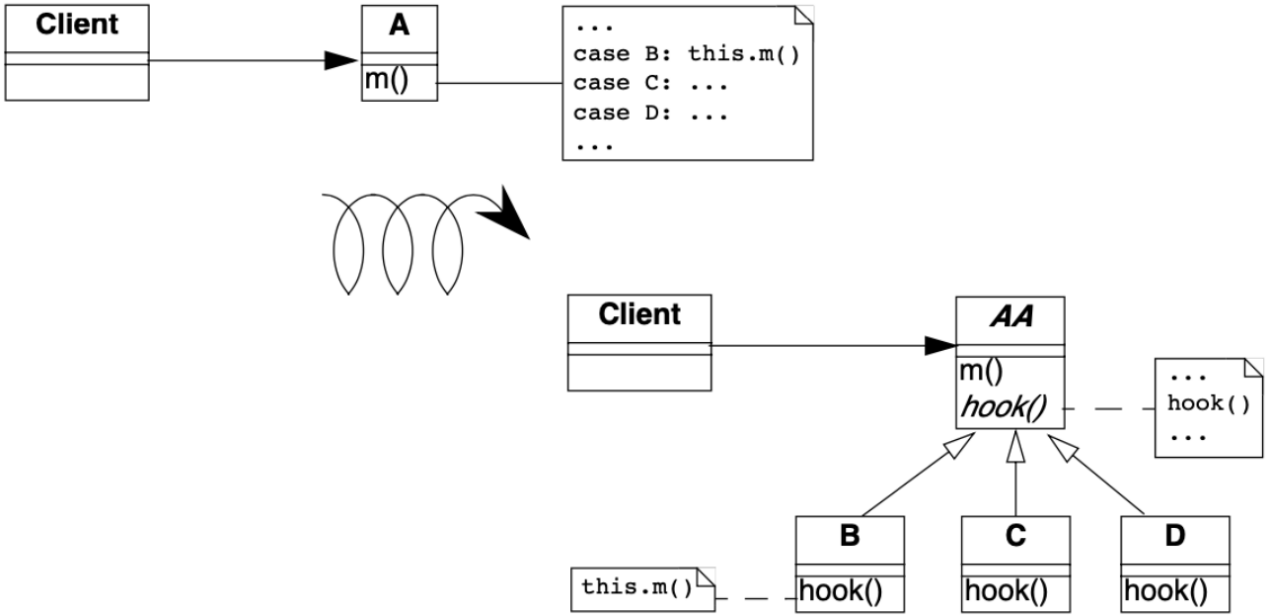
Transform Conditionals to Polymorphism

eliminate the large conditionals, thereby reducing coupling between classes, and improving flexibility in the face of future changes

Large conditionals are often a sign that behavior implemented by clients should probably be be shifted to the provider classes. Typically a new method will be introduced to the provider hierarchy, and the individual cases of the conditional statement will each move to one of the provider classes.

Transform Self Type Checks

Improve the extensibility of a class by replacing a complex conditional statement with a call to a hook method implemented by subclasses.



Transformation of explicit type check into self polymorphic method calls.

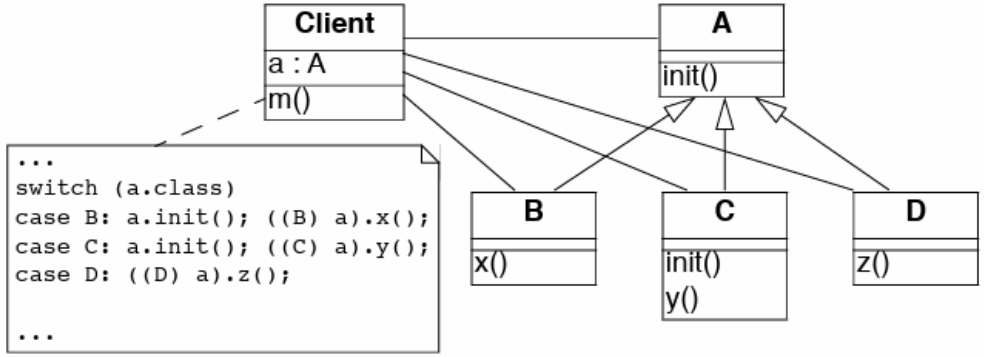
- New behaviors can now be added in a incremental manner, without having to change a set of methods of a single class containing all the behavior.
- A specific behavior can now be understood independently from the other variations.
- A new behavior represents its data independently from the others, thereby minimizing the possible interference and increasing the understandability of the separated behaviors.
- All behaviors now share a common interface, thereby improving their readability.

Transform Client Type Checks

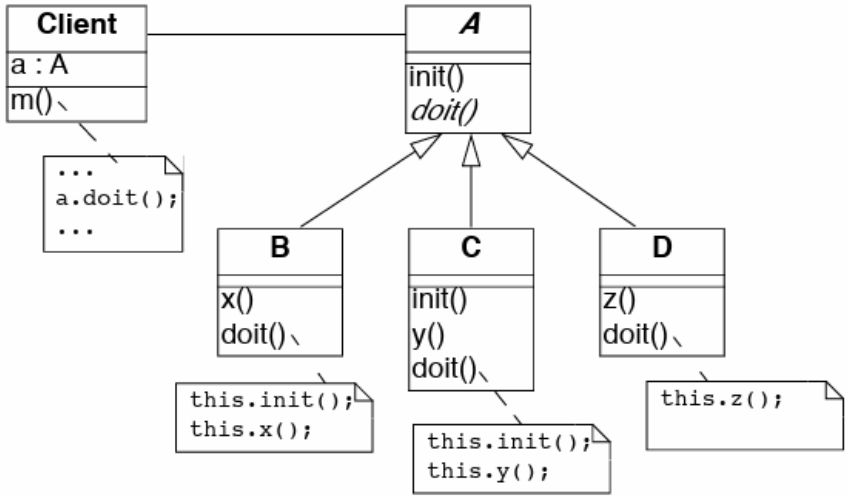
Reduce client/provider coupling by transforming conditional code that tests the type of the provider into a polymorphic call to a new provider method

How do you reduce the coupling between clients and providers of ser vices, where the clients explicitly check the type of providers and have the responsibility to compose providers code?

- Introduce a new method to the provider hierarchy.
- Implement the new method in each subclass of the provider hierarchy by moving the corresponding case of the clients conditional to that class.
- Replace the entire conditional in the client by a simple call to the new method.



Transformation of explicit type check used to determine which methods of a client should be invoked into polymorphic method calls.



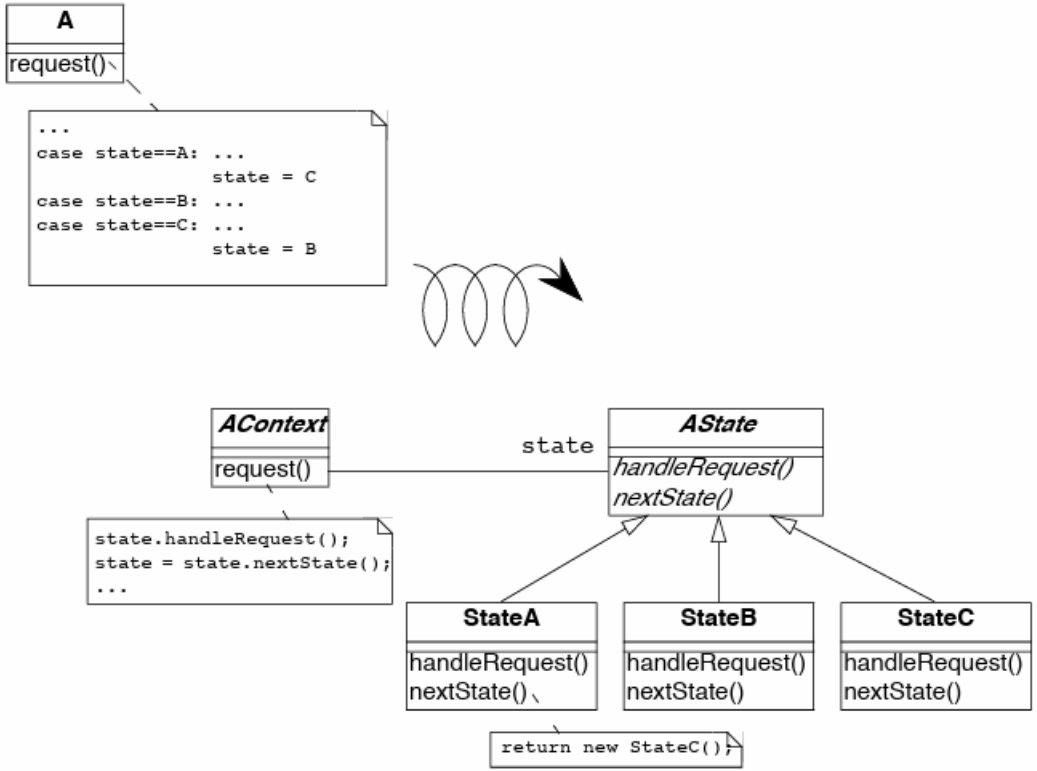
Factor out State

Eliminate complex conditional code over an object's state by applying the State design pattern.

- There are several complex conditional statements spread out over the methods of the object. Adding new behavior may affect these conditionals in subtle ways.
- Whenever new possible states are introduced, all the methods that test state have to be modified.

Apply the State pattern, i.e., encapsulate the state-dependent behavior into separate objects, delegate calls to these objects and keep the state of the object consistent by referring to the right instance of these state objects

Transformation to go from a state pattern simulated using explicit state conditional to a situation where the state pattern has been applied.

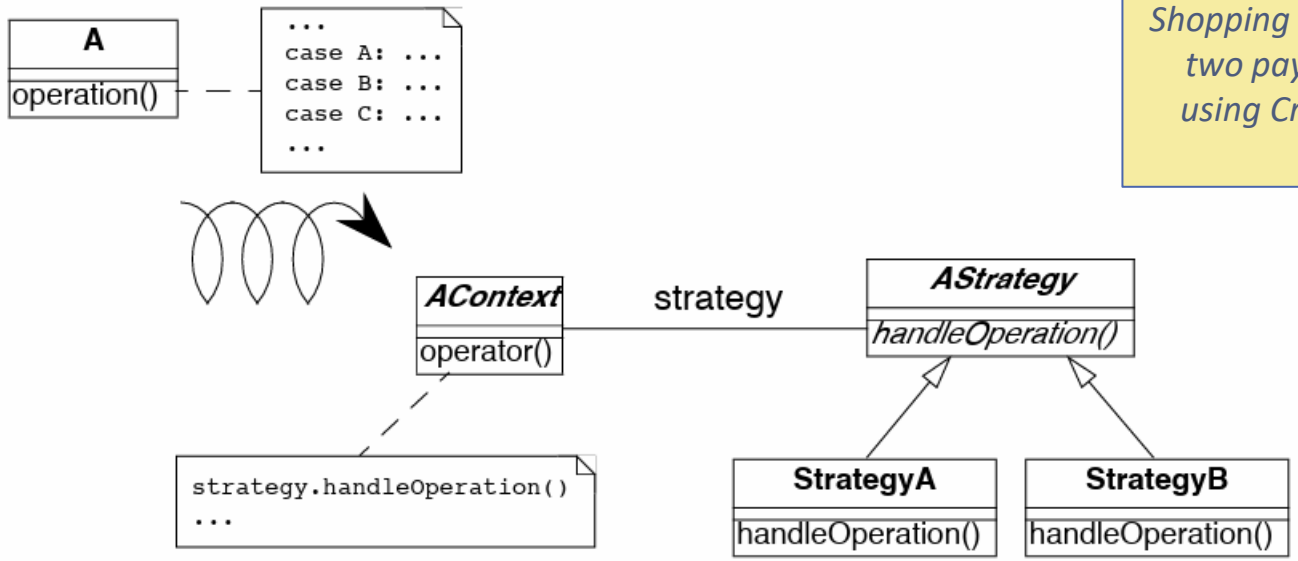


Transform Conditionals to Polymorphism

Factor out Strategy

Eliminate conditional code that selects a suitable algorithm by applying the Strategy design pattern.

Apply the Strategy pattern, i.e., Strategy pattern is used when we have multiple algorithm for a specific task and client decides the actual implementation to be used at runtime.



Shopping Cart where we have two payment strategies - using Credit Card or using PayPal.

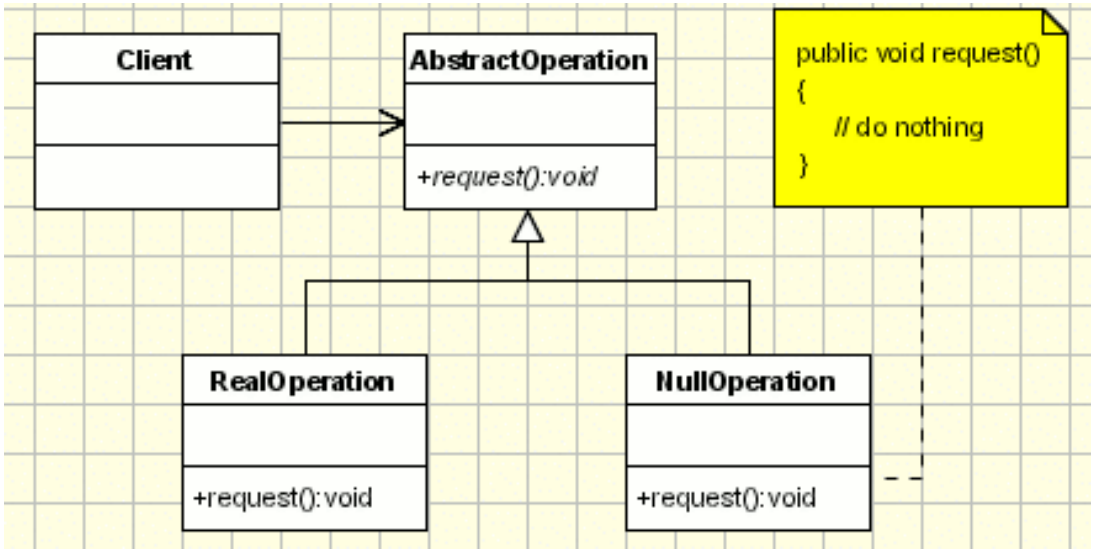
Introduce Null Object

Eliminate conditional code that tests for null values by applying the Null Object design pattern

- Client methods are always testing that certain values are not null before actually invoking their methods.
- Adding a new subclass to the client hierarchy requires testing null values before invoking some of the provider methods.

Apply the Null Object pattern, i.e., encapsulate the null behavior as a separate provider class so that the client class does not have to perform a null test.

The participants classes in this pattern are:
AbstractClass - defines abstract primitive operations that concrete implementations have to define.
RealClass - a real implementation of the AbstractClass performing some real actions.
NullClass - a implementation which do nothing of the abstract class, in order to provide a non-null object to the client.
Client - the client gets an implementation of the abstract class and uses it. It doesn't really care if the implementation is a null object or an real object since both of them are used in the same way.



Transform Conditionals into Registration

Improve the modularity of a system by replacing conditionals in clients by a registration mechanism.

- Having one single place to look for all the kinds of tools makes it easy to understand the system and easy to add new tools.
- However, every time you remove a tool, you have to remove one case in some conditional statement, else certain parts (tool clients) would still reflect the presence of the removed tools, leading to fragile systems.
- Then every time you add a new tool, you have to add a new conditional in all the tool clients.

Introduce a registration mechanism to which each tool is responsible for registering itself, and transform the tool clients to query the registration repository instead of performing conditionals.

That’s it